

Lightweight API Security Monitoring and Anomaly Detection

PHASE I REPORT

Submitted by

TEJUSHREE SANJEEVI KUMAR	230701360
VALLURU VARSHINI	230701369
VARSHA B	230701370

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

December 2026

ANNA UNIVERSITY, CHENNAI
BONAFIDE CERTIFICATE

Certified that this Report titled “**LIGHTWEIGHT API SECURITY MONITORING AND ANOMALY DETECTION: REAL-TIME DETECTION OF FIVE OWASP-BASED VULNERABILITY CATEGORIES WITH ENCRYPTED ACCESS TOKENS**” is the bonafide work of **TEJUSHREE SANJEEVI KUMAR (230701360)**, **VALLURU VARSHINI (230701369)**, **VARSHA B (230701370)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

“This project addresses the following Sustainable Development Goal(s): Goal 9 : Industry, Innovation and Infrastructure.”

SIGNATURE

Dr. J. Manoranjini,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

Mrs. Shyni,
Assistant Professor (SS),
Department of Computer Science
and Engineering,
Rajalakshmi Engineering
College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

Application Programming Interfaces (APIs) are the lifeblood of modern cloud and web services. APIs are often publicly exposed and therefore often targeted. Today's machine-learning approaches to API threat detection score traffic post-factum, explain every flagged anomaly, require enterprise infrastructure and cannot see authorization violations. The access tokens contained in each request are generally only signed and legible to the attacker who intercepts them. The goal of this project is to put forward a lightweight system to solve these problems as a whole. The detection is based on the five named categories in the OWASP API Security Top 10: broken authentication, unrestricted resource consumption, abuse of business flow, broken object-level authorization (BOLA) and broken function-level authorization (BFLA). Each request is classified inline, with fast per token in-memory counters for the behaviour-based categories, and direct ownership and privilege checks for BOLA and BFLA, so there is no machine learning inference needed on the detection path. An isolation forest is trained and kept for running an on-demand SHAP explanation for a request chosen by the analyst from a three-level dashboard. The credential layer is strengthened by issuing access tokens as JSON Web Encryption objects with AES-256-GCM and a short 20-minute expiration. The system correctly and immediately detected all five injected categories on a simulated FastAPI backend with five endpoint types, direct inspection verified that the token payload is unreadable without the server key, and the explanation experiment demonstrated that a BOLA request looks statistically normal, which justifies the hybrid design of rules and model.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini, Ph.D.**, Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Mrs. Shyni**, Assistant Professor (SS), Department of Computer Science and Engineering for her valuable guidance throughout the course of the project. We thank our Project Coordinator, **Mr. Karthik, B.E., M.E.**, Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

TEJUSHREE SANJEEVI KUMAR : 230701360

VALLURU VARSHINI : 230701369

VARSHA B : 230701370

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iii
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1.	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 OBJECTIVE	2
	1.3 EXISTING SYSTEM	3
	1.4 PROPOSED SYSTEM	4
2.	LITERATURE SURVEY	6
	2.1 LITERATURE REVIEW	6
	2.2 COMPARISON AND DISCUSSION	10
	2.3 CONCLUSION	11
3.	SYSTEM DESIGN	12
	3.1 SYSTEM ARCHITECTURE	12
	3.2 SYSTEM REQUIREMENTS	14
	3.2.1 SOFTWARE REQUIREMENTS	14
	3.2.2 HARDWARE REQUIREMENTS	15
	3.2.3 DATASET REQUIREMENTS	16
	3.2.4 DEPLOYMENT AND SCALING	17
	3.3 UML DIAGRAMS	17
	3.3.1 ACTIVITY DIAGRAM	17
	3.3.2 DATA FLOW DIAGRAM	19
	3.3.3 USE CASE DIAGRAM	21
	3.3.4 SEQUENCE DIAGRAM	22
	3.3.5 ENTITY RELATIONSHIP DIAGRAM	23
	3.4 DATABASE DESIGN	24
4.	PROJECT DESCRIPTION	26
	4.1 METHODOLOGIES	26

4.2	MODULES	27
4.2.1	API BACKEND PACKAGE	28
4.2.2	AUTHENTICATION AND TOKEN SECURITY PACKAGE	28
4.2.3	REAL-TIME DETECTION PACKAGE	31
4.2.4	DATA PERSISTENCE PACKAGE	33
4.2.5	EXPLAINABILITY PACKAGE	33
4.2.6	ANALYST DASHBOARD PACKAGE	34
4.2.7	SIMULATION AND EVALUATION UTILITIES	35
5.	IMPLEMENTATION AND RESULT DISCUSSION	37
5.1	IMPLEMENTATION RESULTS	37
5.1.1	IMPLEMENTATION OF THE ENCRYPTED TOKEN MODULE	37
5.1.2	IMPLEMENTATION OF REAL-TIME CATEGORY DETECTION	38
5.1.3	IMPLEMENTATION OF ON-DEMAND EXPLANATION	40
5.1.4	IMPLEMENTATION OF THE ANALYST DASHBOARD	41
5.2	DISCUSSION	42
6.	CONCLUSION AND FUTURE WORK	45
6.1	CONCLUSION	45
6.2	LIMITATIONS	45
6.3	FUTURE WORK	46
	REFERENCES	47

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Summary of the literature survey	9
3.1	Software requirements of the proposed system	14
3.2	Hardware requirements of the proposed system	16
3.3	Structure of the users table	24
3.4	Structure of the request log table	25
3.5	Structure of the category flag table	25
4.1	Endpoints of the simulated backend	28
4.2	Category detection rules	32
4.3	Features supplied to the Isolation Forest	33
4.4	Traffic scenarios generated by the simulator	36
5.1	Comparison of a conventional signed JWT and the proposed JWE token	38
5.2	Category detection results on the simulated traffic	39
5.3	Outcome of the on-demand explanation for two flagged requests	41
5.4	Test cases and observed outcomes	42
5.5	Comparison of the proposed system with the existing system	44

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
1.1	Overview of the Proposed System Architecture	4
3.1	Architecture diagram for the entire system	13
3.2	Activity diagram for the real-time request classification	18
3.3	Activity diagram for the analyst investigation workflow	19
3.4	Data flow diagram of the system (Level 0)	20
3.5	Data flow diagram of the system (Level 1)	21
3.6	Use case diagram of the system	22
3.7	Sequence diagram of the login, request classification and explanation flows	23
3.8	Entity relationship diagram of the database	24
4.1	Real-time category detection pipeline with on-demand SHAP explanation	27
4.2	Structure of the JWE compact serialization used for the access token	29
4.3	Token issuance and verification flow	30
4.4	Per-token in-memory tracking structure	31
4.5	Layout of the three-level analyst dashboard	35
5.1	Number of flagged requests in each category	39
5.2	Behaviour-based versus authorization-based flags	40
5.3	First level of the analyst dashboard populated with the recorded results	42

LIST OF ABBREVIATIONS

AES	Advanced Encryption Standard
API	Application programming interface
ASGI	Asynchronous Server Gateway Interface
BFLA	Broken function-level authorization
BOLA	Broken object-level authorization
DFD	Data flow diagram
GCM	Galois/Counter Mode
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IETF	Internet Engineering Task Force
IP	Internet Protocol
JSON	JavaScript Object Notation
JWA	JSON Web Algorithms
JWE	JSON Web Encryption
JWK	JSON Web Key
JWS	JSON Web Signature
JWT	JSON Web Token
LIME	Local Interpretable Model-agnostic Explanations
LOF	Local Outlier Factor
ML	Machine learning
NIST	National Institute of Standards and Technology
OAuth	Open Authorization
ORM	Object relational mapper
OWASP	Open Worldwide Application Security Project
PKCE	Proof Key for Code Exchange
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
SVM	Support vector machine
TLS	Transport Layer Security
UML	Unified Modeling Language
UUID	Universally unique identifier
WAF	Web application firewall

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Application Programming Interfaces (APIs) are the backbone of modern web, mobile and cloud-native software. All login, search, payment and profile updates through an application are ultimately requests to an API endpoint. This openness is why the API layer is one of the most attacked parts of a modern system. Unlike a traditional web page, an API exposes business functions and data objects directly. So an attacker who can reach an endpoint can probe it programmatically, at high speed and at very low cost.

The OWASP API Security Top 10 [2] is a report on the most common ways APIs are attacked by the security community. Broken object-level authorization, broken authentication, broken function-level authorization, unrestricted resource consumption and unrestricted access to sensitive business flows are on the list. These weaknesses vary in nature. Some of them like credential stuffing, token reuse or request flooding leave a visible trace in the traffic: the request rate increases, the same token is seen from many addresses or the payload is unusually large. Others, such as Broken Object-Level Authorization (BOLA) and Broken Function-Level Authorization (BFLA), create requests that are perfectly normal in shape but become malicious only because of who is making the request and what that caller is allowed to access.

To identify abnormal API traffic, unsupervised anomaly detection has been suggested, and more specifically the Isolation Forest algorithm, which has been combined with explainable AI techniques such as SHAP (SHapley Additive exPlanations) for analysts to understand why a request was flagged [1]. But an anomaly score alone doesn't tell an analyst what kind of attack is occurring, and explaining every flagged request doesn't scale as alert volume grows. Further, each subsequent request after login is made with an access token. If that token is a traditional signed JSON Web

Token (JWT), its payload is simply Base64url-encoded and is readable by anyone who intercepts it [16].

This project solves these three problems simultaneously, on a lightweight and reproducible platform. It classifies every incoming request in real-time against five named OWASP categories, reserves machine-learning explanation for a request that an analyst chooses to inspect, and hardens the credential layer by issuing encrypted, short-lived access tokens using JSON Web Encryption (JWE) [18].

1.2 OBJECTIVE

The objective of this project is to design and implement a lightweight API security monitoring system that detects five well-known API attack categories in real time, explains flagged requests on demand, and protects the confidentiality of the access tokens on which API traffic depends. To achieve this objective the following specific goals are set:

1. Build a simulated FastAPI backend with five endpoint types (login, search, checkout, profile update and data retrieval) and one administrator-only endpoint, so that realistic normal traffic and attack traffic can be generated and observed.
2. Anchor detection to five named categories drawn from the OWASP API Security Top 10: broken authentication, unrestricted resource consumption, business flow abuse, BOLA and BFLA, instead of relying on an open-ended anomaly score.
3. Classify each request inline, before the response is returned, using constant-time in-memory counters and direct authorization checks, so that machine-learning inference is not required on the detection path.
4. Retain an Isolation Forest model and a SHAP explainer that are invoked only when an analyst selects a specific flagged request, keeping the explanation cost bounded regardless of traffic volume.
5. Present the results through a three-level analyst dashboard that moves from category summaries to individual flagged requests and finally to an explanation of one request.

6. Harden the credential layer by issuing access tokens as JWE objects encrypted with AES-256-GCM and by giving them a short fixed expiry, and verify the confidentiality property by direct inspection of the issued tokens.
7. Evaluate the complete system on simulated traffic and report its limitations honestly.

1.3 EXISTING SYSTEM

Traditional API protection relies on Web Application Firewalls (WAFs) that apply signature-based rules, on API gateways that authenticate callers, and on static rate-limiting policies. These mechanisms suffer from high false-positive rates, cannot detect novel attack patterns and need manual configuration, which makes them a weak defence against evolving threats [1]. They also have no notion of object-level or function-level authorization, because a request that carries a valid credential and a well-formed payload does not match any signature.

Machine-learning approaches have therefore been proposed. The closest existing system, by Mittal et al. [1], combines Isolation Forest anomaly detection with SHAP explanations in a cloud-native streaming architecture and reports a precision of 0.94 and a recall of 0.91 on more than one million simulated fintech requests. Although the work is a significant step, it has several limitations from the viewpoint of this project. Its architecture depends on enterprise infrastructure such as stream-processing services and serverless functions, which is difficult to reproduce in a small team or a teaching laboratory. Scoring and explanation are performed for every flagged request, and its own analysis attributes 45 percent of the total latency to SHAP generation. Its error analysis shows that BOLA and BFLA attacks account for 34 percent and 28 percent of the missed attacks respectively, because they look statistically normal. Finally, its categories are not tied to a named security standard, and the confidentiality of the access tokens used by the protected APIs is not addressed.

On the credential side, most APIs issue signed-only JSON Web Tokens. A signed token cannot be forged without the signing key, but its payload is merely Base64url-encoded and is trivially readable by any party that observes the token in

transit, in a log file or at an intermediary proxy [16], [17]. The user identifier, the token identifier and the expiry time are therefore exposed even though the token cannot be altered.

1.4 PROPOSED SYSTEM

The proposed system is a light weight API security monitoring platform for two environments. Figure 1.1 shows the general organization. The upper, real-time environment is in the request path of a simulated FastAPI backend. The token security package is the first stop in every request . It issues encrypted access tokens at login, and decrypts and validates them on all subsequent calls . The request is then passed to the detection package. This package maintains two small per-token structures in memory: a deque of recent request timestamps and a set of recent client addresses. It checks five rules based on OWASP against these. Three of the rules are behavior based as they make use of the counters or the payload size while the last two are direct authorization comparisons which are not dependent on traffic volume or timing. If a rule matches, it saves a category flag and a human-readable reason into a SQLite database.

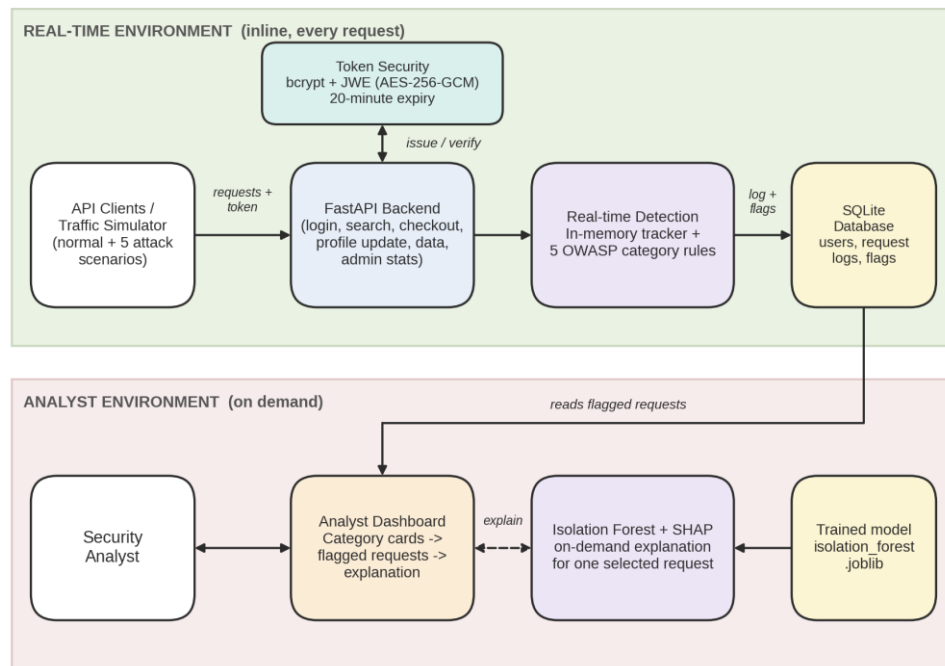


Figure 1.1. Overview of the Proposed System Architecture

The lower, analyst environment is demand-driven. A three-level web dashboard reads the stored flags and presents one card per category, a list of flagged requests of a selected category and, for a single selected request, a SHAP explanation computed from a separately trained Isolation Forest model. The explanation is not computed for every flagged request, purposely so that its cost does not scale with the amount of traffic. These violations are invalid due to the requester, not the request itself. For BOLA and BFLA, the rule that fired is displayed on the dashboard.

Therefore, the system is different from the existing approaches in four aspects. Detection is based on five named OWASP categories and is performed inline on every request, with constant-time checks, the machine-learning model is for explanation, not detection, and the credential layer is hardened with encrypted, short-lived tokens. The whole system is built on FastAPI, SQLAlchemy with SQLite and scikit-learn on a single laptop so that it can be reproduced without enterprise infrastructure.

The remainder of this report is organized as follows. The related literature is presented in Chapter 2. Chapter 3 describes the system design, the architecture, the requirements and the UML diagrams. Chapter 4 gives a detailed description of the modules of the project. Chapter 5 is dedicated to the implementation and the results obtained on simulated traffic. Finally chapter 6 concludes the report with the limitations and the future work.

CHAPTER 2

LITERATURE SURVEY

This chapter surveys the existing work on which the proposed system builds. It covers the closest existing system for API threat detection, the unsupervised anomaly-detection algorithms and explainability techniques that make up its machine-learning side, the practical limits of machine learning in intrusion detection, the OWASP taxonomy of API vulnerabilities, and the standards that govern token-based authentication and encryption. Each work is discussed together with the aspects that it leaves open for this project.

2.1 LITERATURE REVIEW

The paper by Mittal, Keshap and Hosabettu [1] is the closest existing system. It combines Isolation Forest anomaly detection with SHAP explanations for API threat detection in cloud environments and adds a policy-as-code engine that converts model outputs into enforcement actions. The system is built on stream processing, serverless inference and a time-series analytics store, and it is evaluated on more than one million simulated fintech requests with a precision of 0.94, a recall of 0.91 and an inference latency of about 120 ms.

The authors report that SHAP generation consumes 45 percent of the latency, and their error analysis shows that BOLA and BFLA attacks are among the most frequently missed, because such requests carry valid tokens and normal request rates. The shortcomings for this project are the dependence on enterprise-scale infrastructure, the explanation of every flagged request, the absence of a named attack taxonomy and the lack of any protection for the confidentiality of access tokens. Chandola, Banerjee and Kumar [3] provide the standard survey of anomaly detection. They classify the techniques into classification-based, nearest-neighbour, clustering, statistical, information-theoretic and spectral families and distinguish point, contextual and collective anomalies. The survey makes clear that every technique rests on an assumption about what normal behaviour looks like. A request that violates an access-control rule

but has a normal shape is therefore outside the assumption of all of families, which is the motivation behind the hybrid rule-and-model design of this project..

The Isolation Forest was introduced by Liu, Ting and Zhou [4], [5]. Instead of modeling the normal data, the algorithm constructs an ensemble of random binary trees that recursively divide the data on a random feature and split value. Anomalies are few and different. So they are isolated after less splits and have a shorter average path length. The method has linear time complexity, small memory footprint and there is no need for distance or density computation, which makes it attractive for lightweight system. But all it generates is a score. It does not provide information of the attack category of a request. It cannot detect requests of normal feature values.

Local Outlier Factor (LOF) of Breunig et al. [6] ranks each object by comparing its local density with its neighbors. The one-class support vector machine of Schölkopf et al. [7] estimates the support of normal distribution. Both are common baselines in unsupervised. Their cost depends on the amount of stored data, and they share the same limitation of the Isolation Forest, i.e., a detection of deviations only in feature space. Hariri, Carrasco Kind and Brunner [8] introduced the Extended Isolation Forest, which employs randomly oriented hyperplanes to eliminate the artifacts caused by axis-parallel splits in the anomaly-score map. We keep the standard Isolation Forest in this work because it is only used for explanation and not for detection.

Mirsky et al. [9] show that a lightweight network intrusion detection online can be performed outside an Isolation Forest design, using an ensemble of small autoencoders that process packet features incrementally, called Kitsune. But it's an important precedent for cheap, real-time detection. It scores traffic without regard to a named vulnerability taxonomy, and it operates at the network-packet level rather than at the level of API semantics such as object ownership and administrative functions.

For explainability, Ribeiro et al. [10] proposed LIME which approximates a model around a single prediction with an interpretable surrogate. Lundberg and Lee [11] proposed SHAP, an additive feature attribution framework based on Shapley values with properties of local accuracy, missingness and consistency. Lundberg et al. [12] developed TreeExplainer, which computes exact SHAP values for tree ensembles in

polynomial time. TreeExplainer makes it cheap to explain a single decision of an Isolation Forest. However, Slack et al. [13] showed that classifiers can be adversarially constructed to fool perturbation-based LIME and SHAP explanations. Thus, explanations should be taken only as a diagnostic aid, not proof.

[14] Sommer and Paxson investigate why successful machine learning experiments in intrusion detection are rarely deployed in production. They refer to the difficulty of finding really abnormal activity, the high cost of errors, the semantic gap between an anomaly and an actionable finding, and the difficulty of evaluation. They conclude that detections must be interpretable and tied to activities an operator understands. This supports the choice of this project to report named categories with a stored reason. Veeramachaneni et al. [15] suggest the analyst-in-the-loop system AI2, which combines unsupervised outlier detection, analyst feedback, and supervised learning. They show that the system needs to minimize the data the analyst has to review to operate successfully. The dashboard and the on-demand explanation of this project are based on the same principle, while the feedback loop is left as future work.

The OWASP API Security Top 10 [2] enumerates the most prevalent vulnerability classes in API deployments. The five categories selected for this project correspond to its entries for broken authentication, unrestricted resource consumption, unrestricted access to sensitive business flows, broken object level authorization and broken function level authorization. This taxonomy is often referenced in industry guidance, but it is rarely used as the direct basis for a real-time, rule-based detection architecture in the anomaly-detection literature, which tends to be more reliant on general-purpose statistical scoring.

On the credential side, JSON Web Token (JWT) [16] defines a compact and URL-safe representation of claims, and JSON Web Signature (JWS) [17] protects the integrity of the claims with a signature or a message authentication code. A signed token does not guaranty confidentiality as its payload is merely Base64url-encoded. JSON Web Encryption (JWE) [18] adds confidentiality to the family and defines a compact serialization of five segments: a protected header, an encrypted key, an initialization vector, the ciphertext and an authentication tag. The algorithm identifiers are defined in

JSON Web Algorithms (JWA) [19] including the direct key agreement mode. Encryption keys are represented by JSON Web Key (JWK) [20].

The AES-256-GCM used in this project is the Advanced Encryption Standard with Galois/Counter mode . GCM is an authenticated encryption mode that provides confidentiality and tamper detection in a single primitive [21]. The AES cipher itself is standardized in FIPS PUB 197 [22]. The present implementation only partially follows the key management recommendations from NIST Special Publication 800-57 [23] (e.g. limiting the cryptoperiod of a key) and are listed as future work.

Contextualizing Token Based Security in the OAuth 2.0 Ecosystem. The OAuth 2.0 authorization framework [24] popularized short-lived access tokens and refresh tokens . The threat model [25] and the formal security analysis of Fett, Küsters and Schmitz [26] catalog attacks on authorization flows. Proof Key for Code Exchange [27] and token introspection [28] address related concerns. OpenID Connect [29] adds identity assertions on top of OAuth 2.0. At the transport layer, TLS 1.3 [30] encrypts tokens in transit, which is complementary, but not a substitute, for payload-level encryption for tokens that may be written to logs or go through intermediaries. Much of this guidance is focused on detecting the misuse of a compromised token after the fact, and relatively little of it is focused on what a token reveals if it is intercepted before the misuse is detected.

Table 2.1. Summary of the literature survey

Ref.	Work	Contribution	Limitation for this project
[1]	Mittal et al., 2025	Isolation Forest + SHAP with policy-as-code on cloud infrastructure	Enterprise infrastructure, every anomaly explained, BOLA/BFLA missed, tokens not addressed
[3]	Chandola et al., 2009	Survey and taxonomy of anomaly detection	Assumes anomalies differ in feature space
[4], [5]	Liu et al., 2008, 2012	Isolation Forest	Score only, no attack category
[6], [7]	Breunig et al.; Schölkopf et al.	LOF; one-class SVM	Cost grows with data, feature-space deviations only
[8]	Hariri et al., 2021	Extended Isolation Forest	Still a detection score, no taxonomy

Ref.	Work	Contribution	Limitation for this project
[9]	Mirsky et al., 2018	Kitsune: ensemble of autoencoders	Packet level, no API semantics
[10]-[12]	Ribeiro et al.; Lundberg et al.	LIME, SHAP, TreeExplainer	Costly if applied to every alert
[13]	Slack et al., 2020	Adversarial attacks on LIME and SHAP	Explanations can be misleading
[14], [15]	Sommer and Paxson; Veeramachaneni et al.	Limits of ML in IDS; analyst-in-the-loop	No API-specific, category-based design
[16]-[20]	IETF RFCs 7519, 7515, 7516, 7518, 7517	JWT, JWS, JWE, JWA, JWK	Signed tokens are readable; JWE rarely applied
[21]-[23]	McGrew and Viega; NIST	AES-GCM, AES, key management	Key rotation and storage remain deployment issues
[24]-[30]	OAuth 2.0 family, OIDC, TLS 1.3	Authorization and transport security	Focus on misuse after compromise

2.2 COMPARISON AND DISCUSSION

The works in the survey fall into two categories that are rarely connected. The anomaly-detection literature [1], [3]-[9] is about finding unusual traffic and at best explaining it. But it does not tell an analyst which named vulnerability is being exploited, and it cannot see authorization violations. The token-security literature [16]-[30] defines strong mechanisms for issuing and protecting credentials. It is not related to the detection of attacks against the APIs that use those credentials. A practical monitoring system needs both, because a real attack on an API typically includes both a suspicious traffic pattern and a stolen, shared or replayed token.

Three lessons are distilled from the survey. Firstly, the nearest existing system finds that Isolation Forest and SHAP work for API traffic but its own error analysis shows that authorization violations escape a traffic-shape detector. The project takes this finding as a design requirement and uses direct ownership and privilege checks for BOLA and BFLA. Second, operational studies [14], [15] argue that detection results should be linked to activities the analyst understands and the amount of information presented should be limited, so this project reports five named categories and computes explanations only on demand. Third, the mechanism for confidential tokens is already

provided by the standards [18]-[21] and to use it, only a small change is needed in the token issuance step.

2.3 CONCLUSION

Although the reviewed works have studied anomaly detection, explainability and token security in depth, there is no single system that combines real-time detection tied to a named API vulnerability taxonomy, on-demand explanation that keeps the explanation cost low, explicit handling of authorization violations and confidentiality of access tokens, on a platform reproducible by a small team. The proposed system aims to fill this gap. The system design, its modules and the results obtained on simulated traffic are described in the following chapters.

CHAPTER 3

SYSTEM DESIGN

The design of the proposed system is presented in this chapter. It describes the architecture and its layers, the software, hardware and data requirements and the UML diagrams that specify the behavior of the system: activity diagrams, data flow diagrams, a use case diagram, a sequence diagram and an entity relationship diagram. The database design is explained at the end of the chapter.

3.1 SYSTEM ARCHITECTURE

The system is based on a layered architecture (see Figure 3.1) each layer has a single responsibility and only communicates with its neighbors. The design is driven by three principles: detect quickly with simple checks, explain lazily and only when asked, and protect the credential that every request carries.

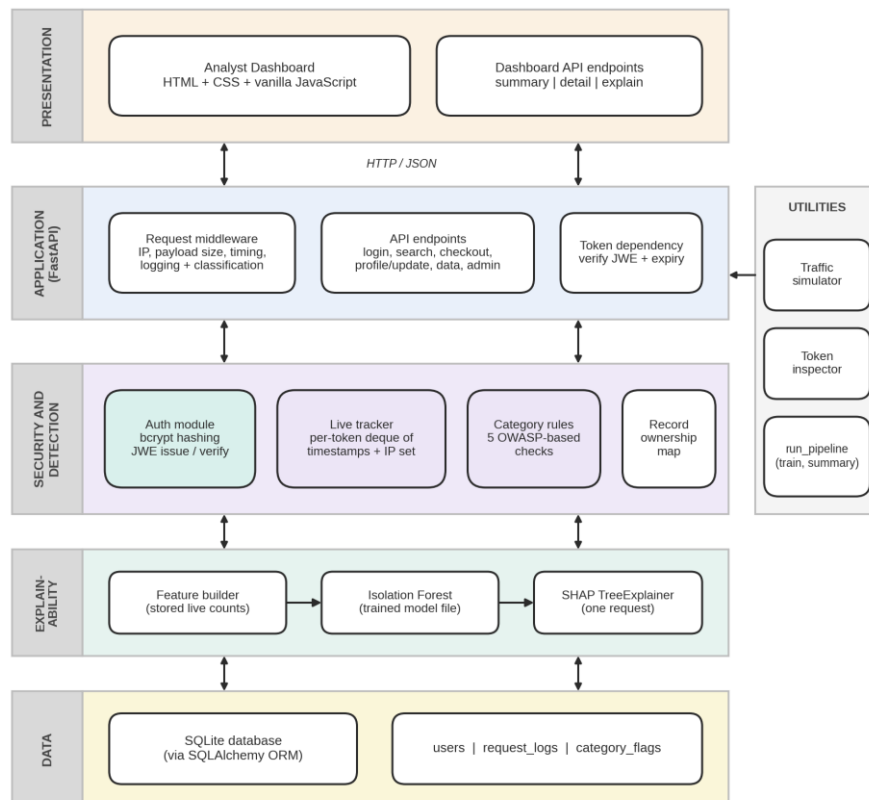


Figure 3.1. Architecture diagram for the entire system

The presentation layer is composed of the analyst dashboard, a server-rendered HTML page with vanilla JavaScript and a small set of dashboard API endpoints which return the category summary, the flagged requests of a category and the explanation of one request. The application layer is the FastAPI app. A request middleware listens for all calls and the endpoints implement the simulated business functions. Each protected endpoint depends on a token, so there is no custom authentication logic in any endpoint.

The detection and security layer is made up of the components that make the system unique. The authentication module hashes passwords using bcrypt, and generates and validates the JWE tokens. Live tracker for each token, keeping a deque of request timestamps and a set of recent client addresses, category rules assess the five conditions based on OWASP, and a small map of record-ownership provides something concrete for the BOLA check to compare to.

The explainability layer includes the feature builder, the trained Isolation Forest model and the SHAP TreeExplainer. It is not part of the request path, and only called by the explain endpoint. The data layer is a SQLite database and is accessed via SQLAlchemy. There are three tables for users, request logs and category flags. The system is completed with three utilities: a traffic simulator, which generates normal and attack traffic, a token inspector, which shows the token structure, and a command-line script, which trains the model and generates the category summary.

3.2 SYSTEM REQUIREMENTS

This section discusses the requirements of the proposed system. It describes the software and hardware necessary to build and run it, the data used for evaluation, and how the system can be deployed and scaled.

3.2.1 SOFTWARE REQUIREMENTS

Python is selected as the development language since the web framework, the cryptography libraries and the machine-learning stack are all available for it. We used Python 3.12, because the latest interpreter versions did not yet have pre-built packages for the data-science libraries used on the development machine. The libraries used and their purposes are listed in Table 3.1.

Table 3.1. Software requirements of the proposed system

Software component	Version	Purpose
Python	3.12	Overall development language of the system
FastAPI	0.115.0	Web framework for the endpoints, dependencies and middleware
Uvicorn	0.30.6	ASGI server that runs the FastAPI application
SQLAlchemy	2.0.35	Object relational mapper used to define and query the tables
SQLite	Bundled with Python	Single-file embedded database for users, request logs and category flags
python-jose	3.3.0	JSON Web Encryption (dir, A256GCM) for the access tokens
cryptography	41.0.7	Cryptographic back end for python-jose
passlib and bcrypt	1.7.4 and 4.0.1	Salted password hashing at login
python-multipart	0.0.9	Parsing of the login form
pandas	2.2.2	Building the feature table for the Isolation Forest
scikit-learn	Release compatible with Python 3.12	Isolation Forest model
SHAP	Release compatible with Python 3.12	TreeExplainer for on-demand explanations
joblib	Installed with scikit-learn	Saving and loading the trained model file
Visual Studio Code, pip and venv	Current	Development environment and dependency isolation

3.2.2 HARDWARE REQUIREMENTS

The system is designed to be lightweight. The detection path only uses constant-time operations on small in-memory structures, and the Isolation Forest is trained on a small feature table, so no specialized hardware is required. The server, the traffic simulator and the dashboard can be run together on a standard laptop. The proposed specifications are given in Table 3.2.

Table 3.2. Hardware requirements of the proposed system

Hardware component	Suggested specification	Purpose
Processor	Multi-core x86-64 processor (4 or more cores)	Running the server, simulator and model training concurrently
RAM	8 GB or higher (4 GB minimum)	Python runtime, SHAP and the in-memory tracker
Storage	2 GB of free space	Virtual environment, libraries, database and model file
Operating system	Windows 10/11, Linux or macOS	Development and execution of the system
Display and browser	Any modern web browser	Viewing the analyst dashboard
Network	Local loopback interface	Communication between the simulator, server and dashboard

3.2.3 DATASET REQUIREMENTS

The system is tested on synthetic data generated by the traffic simulator against the live local backend. There's no real user traffic, and there's no real attack against a real system. When the server starts it automatically seeds three fake accounts: a regular demo user, a second user that is used as victim in BOLA scenario and an administrator. A fixed ownership rule is used (e.g. records 1-50 belong to the demo user while records 51-100 belong to the second user) so that the BOLA check has a concrete rule to compare to.

The simulator generates normal usage traffic with a human-like pace and injects 5 deliberate attack scenarios, one for each identified category. Each scenario logs in separately and therefore uses its own token. Preventing the behavioral counters of one scenario contaminating the next.

All requests are logged with endpoint, method, client address, the token identifier, timestamp, the payload size and response time. The counts that were computed for it at request time are stored with it so it can be explained later.

3.2.4 DEPLOYMENT AND SCALING

Currently the system runs in one Uvicorn process on one machine, with the tracker in the process' memory and the data in a local SQLite file. This suffices for a demonstration and the simulated traffic used here. For a production deployment, two changes are needed. The per-token tracker would have to be moved to a shared low-latency store like Redis so multiple worker processes see the same counters, and SQLite would be replaced by a server database. Also, packaging the model and the dashboard and the backend into a container would offer isolation of the demonstration environment from the host. These changes do not affect the detection logic and we list them as future work.

3.3 UML DIAGRAMS

In this section, the design of the proposed system is illustrated using UML diagrams. The diagrams show the flow of activities and data through the system and the interactions of its components. They define the requirements that must be satisfied by the implementation. The system has a real-time detection path and an on-demand analyst path, both of which are explained in the diagrams.

3.3.1 ACTIVITY DIAGRAM

An activity diagram depicts the flow of actions of a system, including decisions and parallel branches. Figure 3.2 shows the activity diagram of the realtime detection path. It then gets the client address, the X-Forwarded-For header if available, and the size of the payload from the request that comes in along with its bearer token. The token is then decrypted, the expiry checked, and an invalid or expired token request is rejected. If the token is valid, the in-memory tracker is updated and the five category checks are executed in parallel. If any rule is matched, category flags with their reasons are stored and the request log with its live counts is persisted before the response is returned.

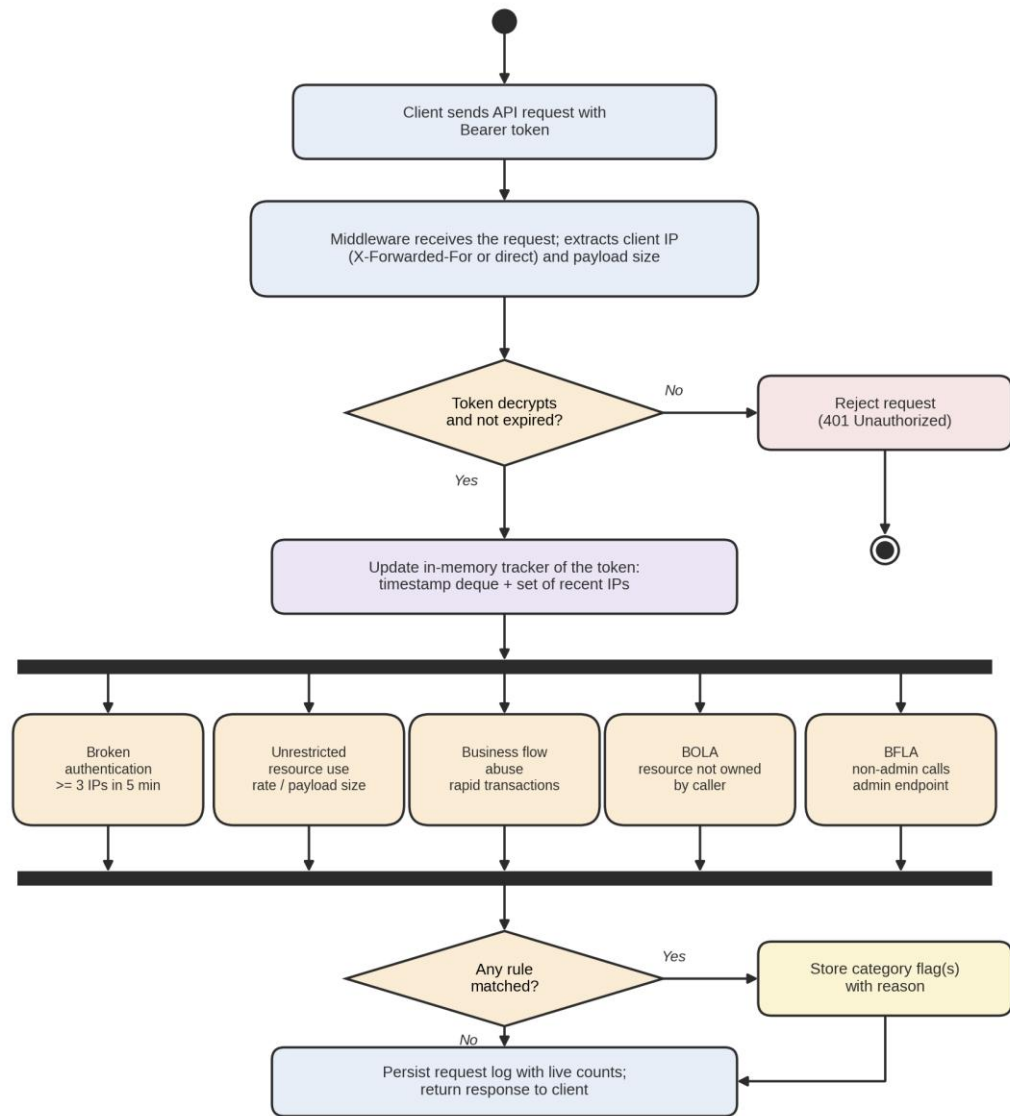


Figure 3.2. Activity diagram for the real-time request classification

The analyst path activity diagram is shown in Figure 3.3. The analyst opens the dashboard and sees 1 card per category. The analyst selects a category and sees every flagged request within it. The analyst selects 1 request. For the three behaviour-based categories the system calculates the Isolation Forest decision function and the SHAP contributions for that single request and displays them as horizontal bars. For BOLA and BFLA it shows only the rule reason as these violations cannot be explained from the traffic shape. The analyst then investigates the incident.

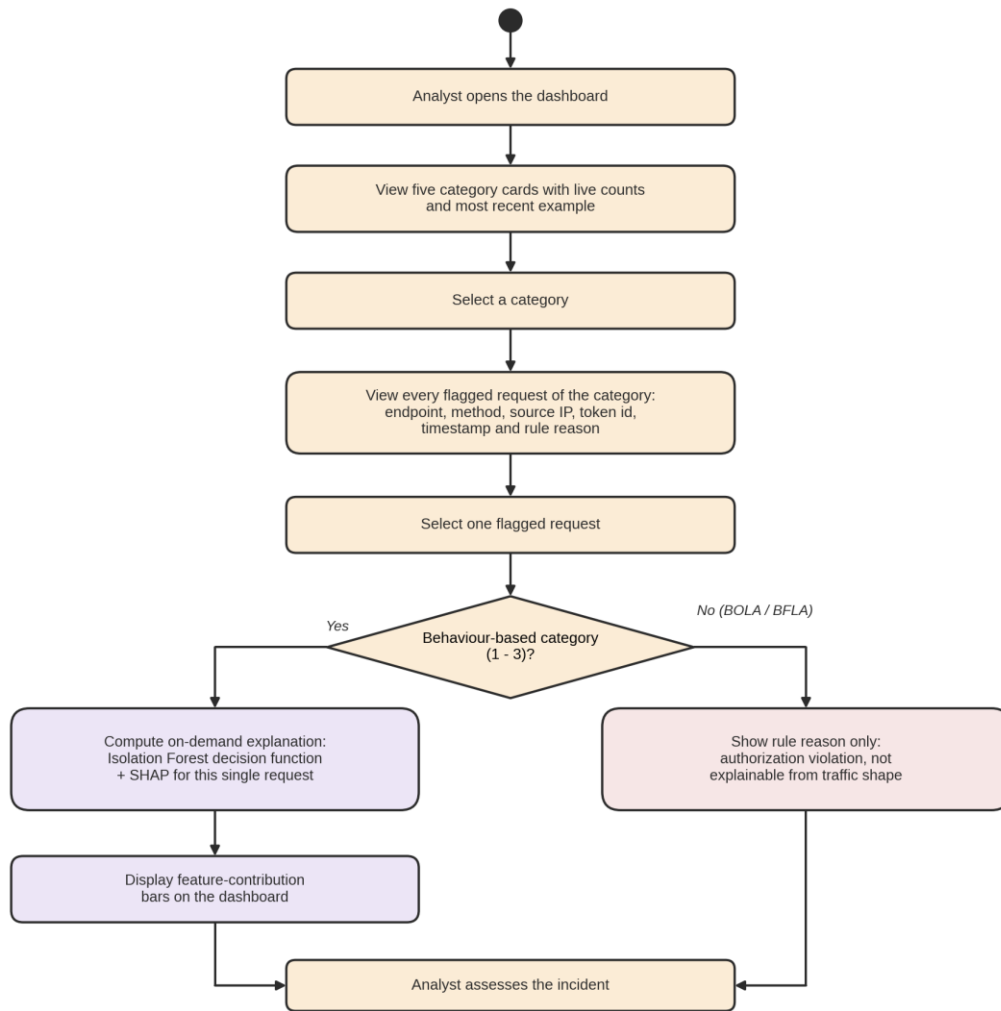


Figure 3.3. Activity diagram for the analyst investigation workflow

3.3.2 DATA FLOW DIAGRAM

A data flow diagram is a diagram of the flow of data between external entities, processes and data stores of a system. Data flow diagrams can be drawn at multiple levels. Figure 3.4: Level 0 or context diagram The whole system is a single process that exchanges data with two external entities. The API client sends credentials and requests with a bearer token and receives encrypted access tokens and responses and the security analyst sends selections and explanation requests and receives category summaries, flagged requests and explanations.

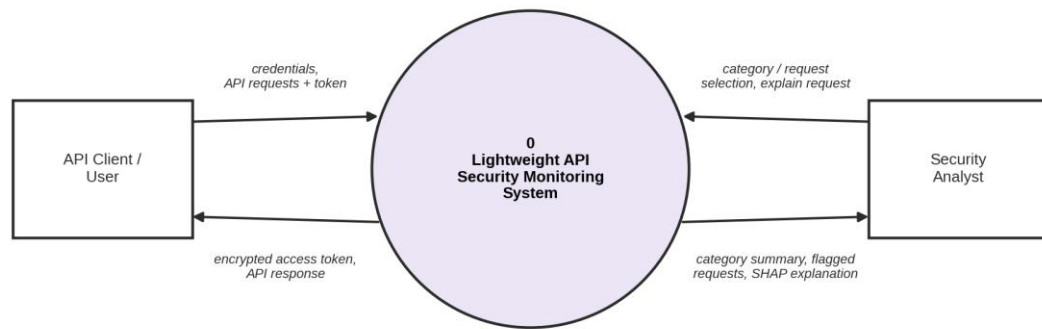


Figure 3.4. Data flow diagram of the system (Level 0)

Figure 3.5 is the level 1 diagram. The circles are the processes, the open rectangles are the data stores and the arrows are the data in transfer. The process 1.0 authenticates the user against the users store and issues the token. Process 2.0 validates the token, processes the request, and writes the request log. Process 3.0 Tracks and classifies the behavior of the request and writes any category flag. Process 4.0 reads the request logs and category flags to display the dashboard. Process 5.0 loads the trained model and explains one selected request based on its stored features. The detection path is processes 2.0 and 3.0, and that is the only part that is executed on every request. The model is only touched by process 5.0.

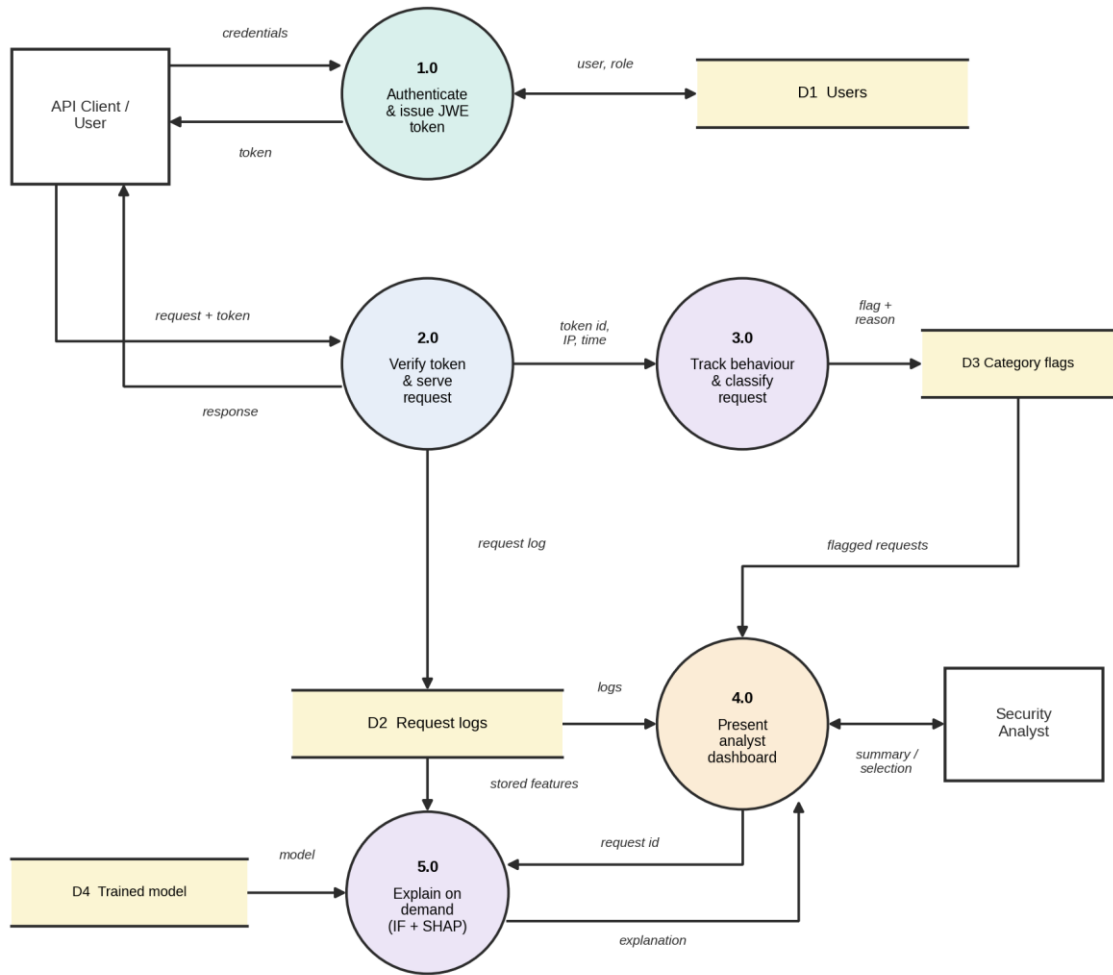


Figure 3.5. Data flow diagram of the system (Level 1)

3.3.3 USE CASE DIAGRAM

System offers functions to four actors, as shown in the use case diagram in Figure 3.6. The API user logs in and receives an encrypted token, then calls the business endpoints to retrieve records. The administrator also accesses the administrative statistics endpoint. The simulated attacker creates the attack traffic that tests the detection rules. The security analyst looks at the category summary and the flagged requests, asks for an on-demand explanation and trains the Isolation Forest model. The explanation of the flagged requests extends the view of the real time classification use case in all business calls.

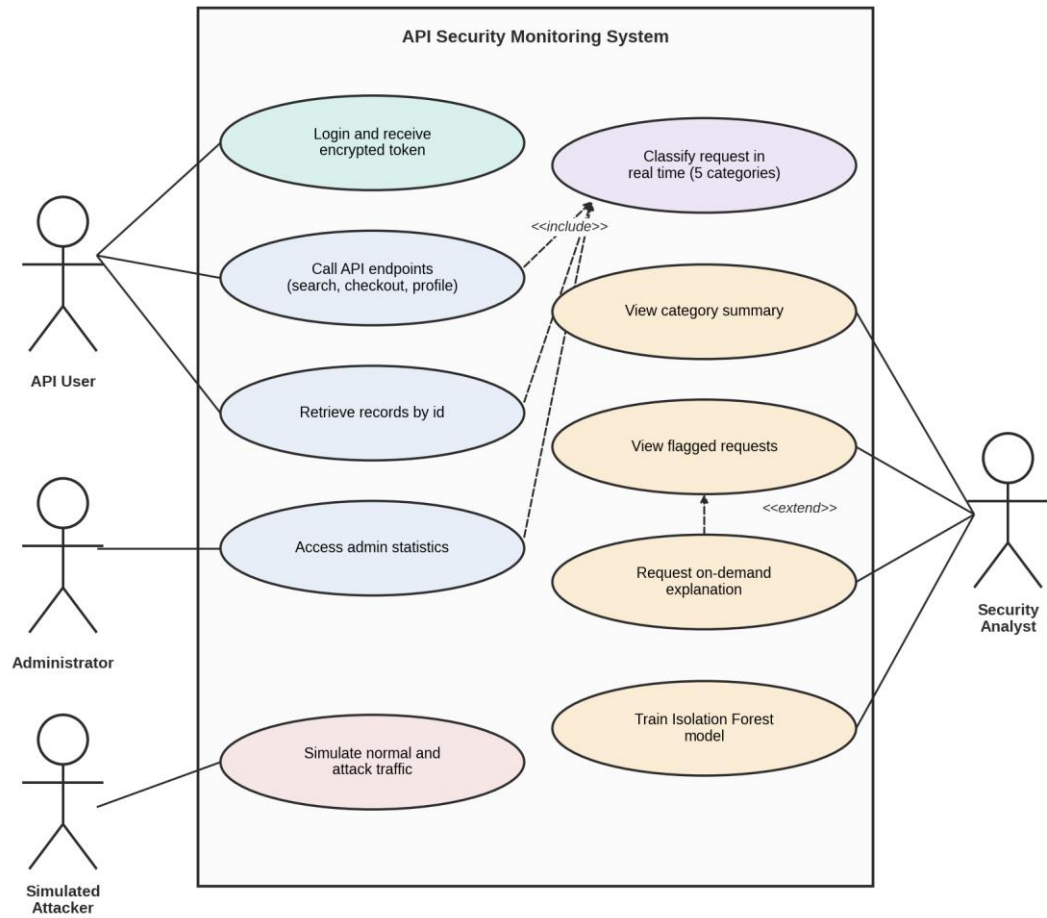


Figure 3.6. Use case diagram of the system

3.3.4 SEQUENCE DIAGRAM

Fig. 3.7 shows the sequence diagram representing the order of interactions between components. The password hash is checked, and the claims are encrypted and sent back in a token with five segments in response to the login message. Later on the middleware asks the authentication module to decrypt the token and receives the claims, updates the tracker, evaluates the five rules, stores the request log and the flags in the database and returns the response. Later the analyst fetches the counts and details from the database and upon selecting one request the explainer reads the features of that request stored and returns the decision function and the SHAP bars.

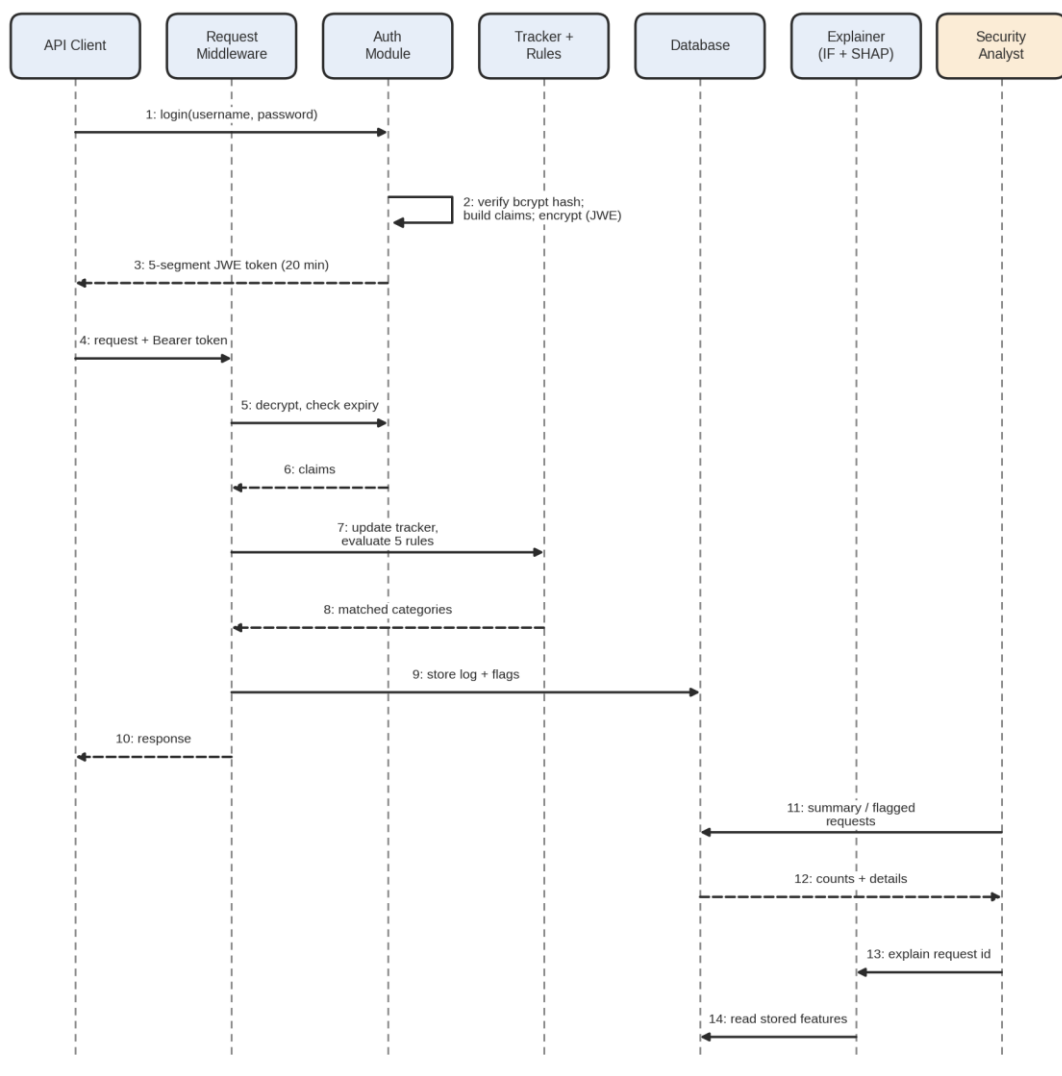


Figure 3.7. Sequence diagram of the login, request classification and explanation flows

3.3.5 ENTITY RELATIONSHIP DIAGRAM

Figure 3.8. Entity Relationship Diagram of the Persistent Data. A user makes many requests. Each logged request can have zero or more category flags associated with it. The per-token tracker is intentionally not persisted; it is a working structure of the detection path, and only its calculated counts are persisted with each request log.

The per-token tracker (deque + IP set) lives in memory and is not persisted.

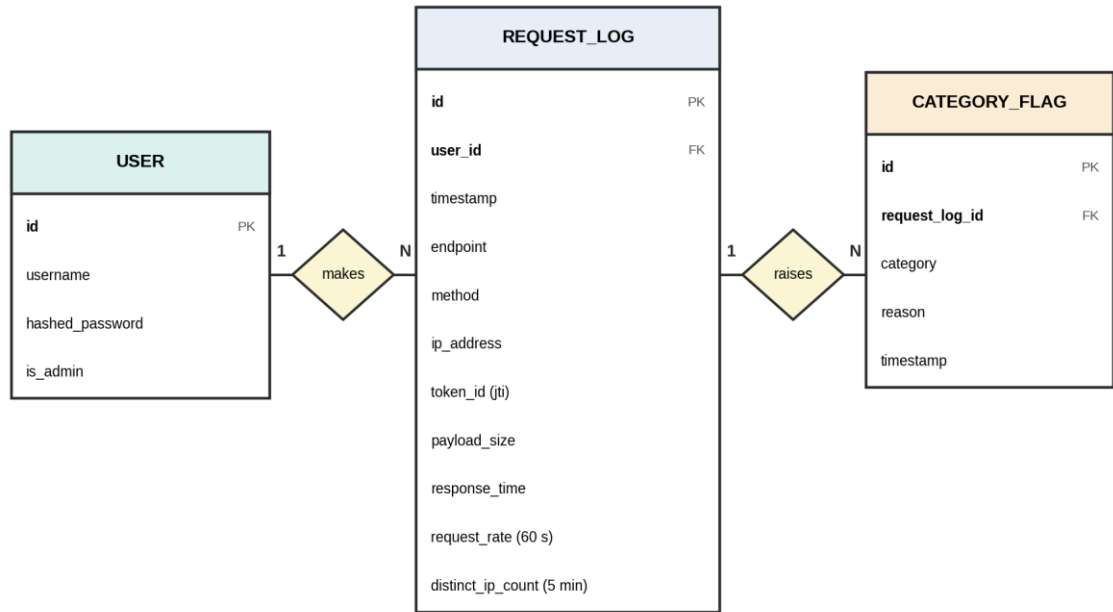


Figure 3.8. Entity relationship diagram of the database

3.4 DATABASE DESIGN

The persistent data is stored in 3 tables and are described in Tables 3.3 to 3.5. The column names are on the logical level.

Table 3.3. Structure of the users table

Column	Type	Description
id	Integer, primary key	Unique identifier of the user; it is also the subject claim of the token
username	String, unique	Login name of the user (demo, victim, admin)
hashed_password	String	Salted bcrypt hash of the password; the plain password is never stored
is_admin	Boolean	Marks administrators; copied into the token claims for the BFLA check

Table 3.4. Structure of the request log table

Column	Type	Description
id	Integer, primary key	Identifier of the logged request
user_id	Integer, foreign key	User that owns the token used
timestamp	Date and time	Time at which the request arrived
endpoint	String	login, search, checkout, profile_update, data_retrieval or admin_stats
method	String	HTTP method of the request
ip_address	String	Client address (X-Forwarded-For header or direct connection)
token_id	String	Unique token identifier (jti) of the token used
payload_size	Integer	Size of the request payload in bytes
response_time	Float	Time taken to serve the request
request_rate	Integer	Requests by the token in the last 60 seconds, computed at request time
distinct_ip_count	Integer	Distinct addresses of the token in the last 5 minutes, computed at request time

Table 3.5. Structure of the category flag table

Column	Type	Description
id	Integer, primary key	Identifier of the flag
request_log_id	Integer, foreign key	Request that matched the rule
category	String	broken_authentication, unrestricted_resource_use, business_flow_abuse, broken_object_authorization or broken_function_authorization
reason	String	Human-readable reason, for example the number of requests in the window
timestamp	Date and time	Time at which the flag was stored

CHAPTER 4

PROJECT DESCRIPTION

4.1 METHODOLOGIES

This section describes in detail the methodology followed in the project. The system is designed as a set of loosely coupled packages that communicate through well defined interfaces. The approach was refined through an iterative process with feedback from the review. The current design that checks each request inline against named OWASP categories and reserves machine-learning explanation for one request at a time replaces the previous design that scored all traffic in batches and ranked individual anomalies.

Figure 4.1 depicts the lifecycle of a request in the resultant design. A request enters one of the endpoints of the simulated backend first. The fast in-memory tracker updates the per-token request and address counters and the five-category rule classifier compares the request against the threshold and authorization checks . If a rule matches then a category flag is stored and the response is sent back to the client. The detection adds no machine-learning latency to the response, since the classifier uses only constant-time counters and direct comparisons. The dashboard displays the feature contributions from the trained Isolation Forest and SHAP only when an analyst clicks a flagged request.

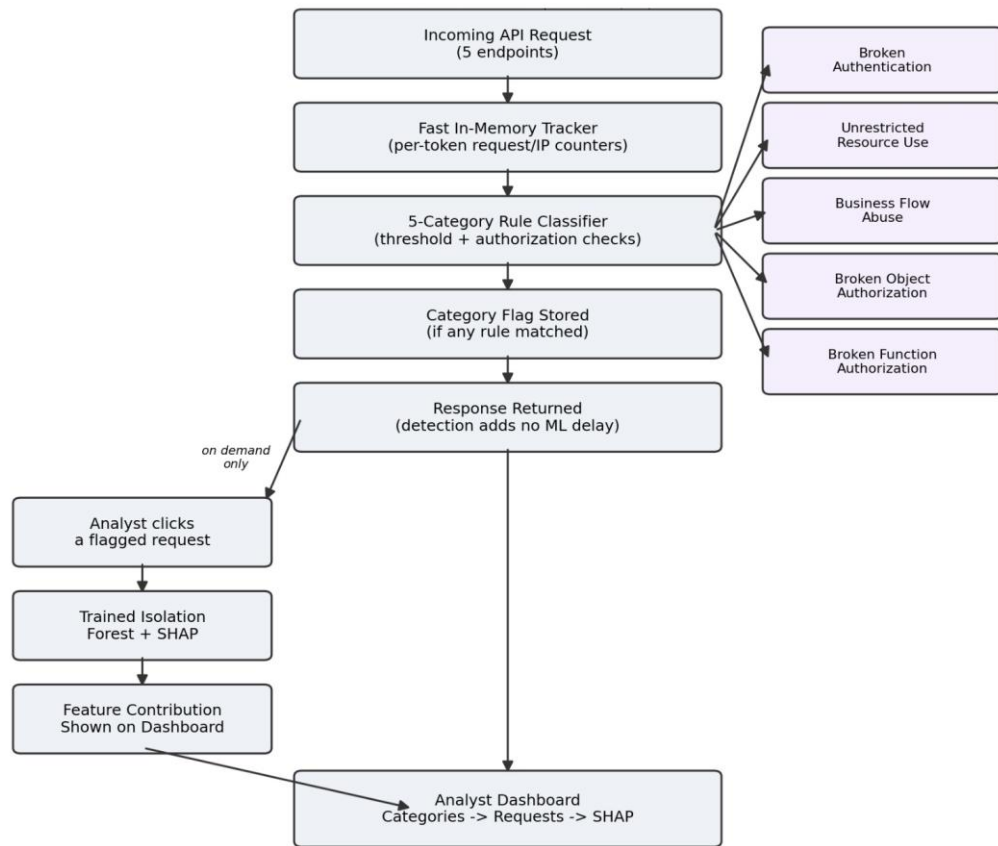


Figure 4.1. Real-time category detection pipeline with on-demand SHAP explanation

4.2 MODULES

There are seven packages in the system. Some of them are active on every request, some only when the analyst asks them. The packages are:

- 1) Package API Backend
- 2) Authentication and Token Security Bundle
- 3) Realtime Detection Package
- 4) Data Persistence Package
- 5) Explainability Pack
- 6) Analyst Dashboard Package (
- 7) Simulation and Evaluation Tools

4.2.1 API BACKEND PACKAGE

This package, the simulated application, is protected by the rest of the system. It is a FastAPI application with 5 endpoint types, login, search, checkout, profile update, data retrieval and one administrator only endpoint that was added to exercise the BFLA rule. The endpoints are given in Table 4.1. Each protected endpoint declares token verification as a dependency, so there is no custom authentication logic in any endpoint. Each call is wrapped by a request middleware: it reads the client address from the X-Forwarded-For header, measures the payload size and measures the response time and calls the classification for header when the header is present and from the direct connection otherwise (also how a deployment behind a proxy would work). When the server starts, it seeds three fake accounts and record ownership is given by a fixed rule, records 1 to 50 for the demo user and 51 to 100 for the second user.

Table 4.1. Endpoints of the simulated backend

Endpoint id	Path	Function	Related category
login	/login	Verifies the password and issues the JWE token	Broken authentication
search	/search	Simulated search request	Resource consumption
checkout	/checkout	Simulated transactional call with a payload	Resource use, business flow
profile_update	/profile/update	Simulated profile modification	Business flow abuse
data_retrieval	/data?record_id=	Returns a record by its identifier	BOLA
admin_stats	/admin/stats	Administrator-only statistics	BFLA

4.2.2 AUTHENTICATION AND TOKEN SECURITY PACKAGE

This package issues and validates access tokens. The token is not readable if obtained by anyone else. Instead the package returns the token as a JSON Web Encryption object. When a user logs in, the server verifies the password against its bcrypt hash, and builds an object of claims with the user identifier, a unique token identifier (a UUID, called the jti), an expiry timestamp and an administrator flag which

is utilized by the BFLA check. The claims are encrypted with the jose library, using the dir key management algorithm, where the 32 byte symmetric key is used directly without any key-wrapping step, and the A256GCM content encryption algorithm. The five-part compact serialization of Figure 4.2 is the output, and the access token returned to the client.

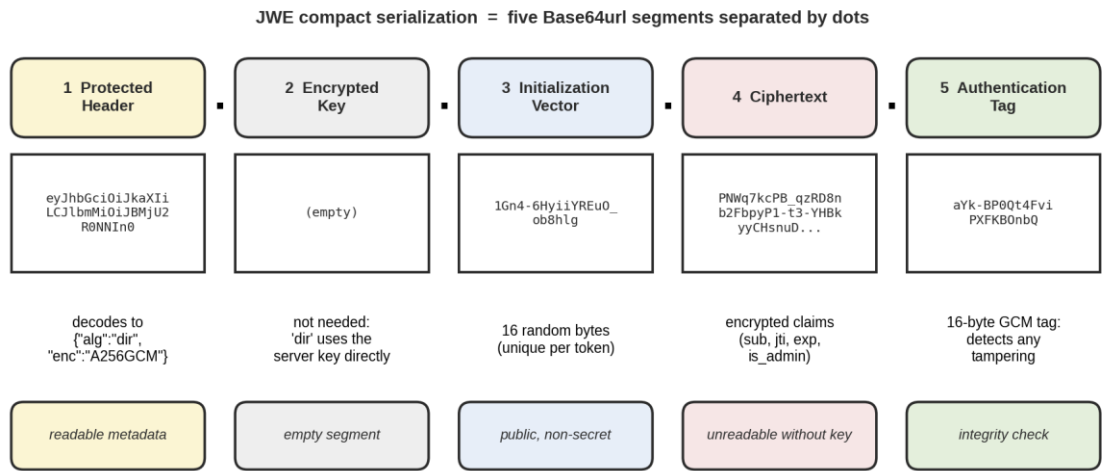


Figure 4.2. Structure of the JWE compact serialization used for the access token

You can read only the header and it doesn't contain any user specific information. It just tells the algorithm and the way of encryption. In direct mode, the encrypted key part is empty. The claims are in the ciphertext. The authentication tag allows to detect any modification of the token. The key that the server has is never sent to the client. Figure 4.3 shows the verification flow along with the issuance flow.

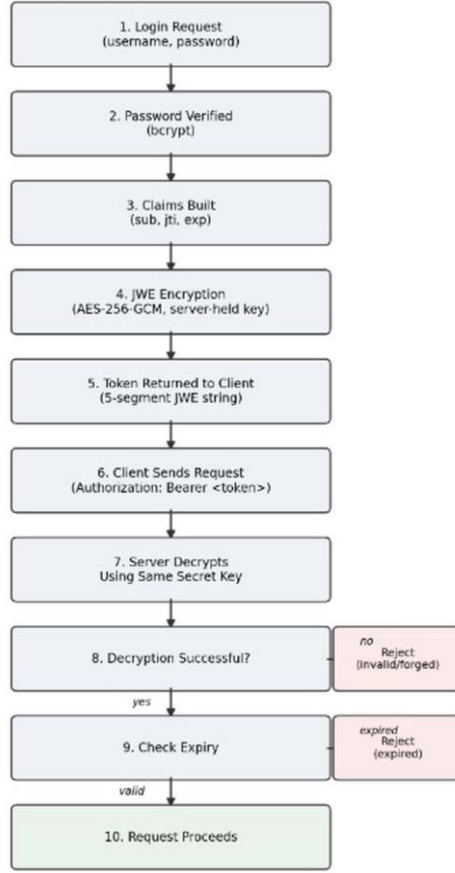


Figure 4.3. Token issuance and verification flow

The server decrypts the presented token with the same key for each subsequent request. If the decryption fails because the token was tampered with, forged or issued with a different key, the request is rejected immediately. If this is successful, the expiry timestamp is checked against the current time and the expired token is rejected even if it is decrypted correctly. Formally, a token t is only accepted if Equation 4.1 holds where Dec is the AES-256-GCM decryption with the server key K .(4.1).

$$\text{valid}(t) = (\text{Dec}_K(t) \neq \perp) \wedge (t.\text{exp} > \text{now}) \quad (4.1)$$

The tokens are issued with a fixed expiration of 20 minutes. This is a conscious design decision to limit the amount of time an intercepted token is useful to an attacker, regardless of whether the interception is ever noticed. A simplified version of the short lived credential principle is used, without a refresh mechanism.

4.2.3 REAL-TIME DETECTION PACKAGE

This package is the heart of the system and it sorts every request into one of five categories. Two of the categories rely on short-term behavioral history: the number of recent requests made by a token and the number of distinct addresses that it has been seen from. If the entire history of requests was queried for each check, the result would be a slowdown, so the package keeps a small rolling window in memory for each token as depicted in Figure 4.4. The window is a deque of recent timestamps and a set of recent client addresses. New requests are added to the end and requests older than the window are removed at the other end. So each check is a constant time operation regardless of the amount of traffic logged. The structures are protected by a lock so that they remain safe during concurrent serving of requests. The two quantities are defined in Equation 4.2 where T is the set of timestamps and I is the set of addresses of token t .

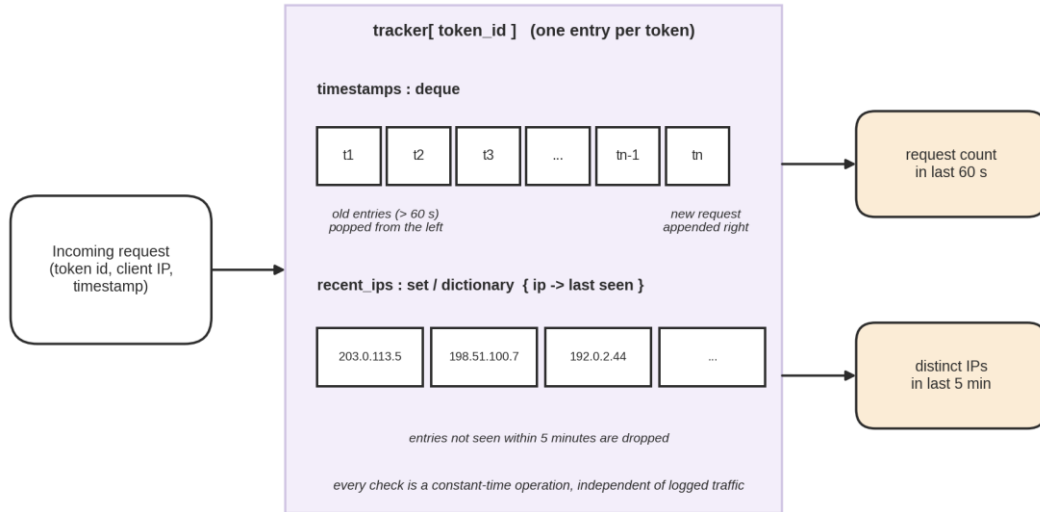


Figure 4.4. Per-token in-memory tracking structure

$$r_{60}(t) = |\{\tau \in T_t : now - \tau \leq 60s\}|, \quad d_5(t) = |\{ip \in I_t : now - seen(ip) \leq 300s\}| \quad (4.2)$$

The five rules are summarized in Table 4.2. The first three are based on behavior and use the counters described above, while the last two are based on authorization and depend on a direct comparison with the real permissions of the request, regardless of the volume or timing of the traffic. The BOLA rule checks the owner of the record being

requested and compares it to the subject of the token. The BFLA rule checks the administrator claim of the token when an administrator-only endpoint is invoked. In the current implementation, the thresholds are fixed constants.

Table 4.2. Category detection rules

Category	OWASP entry	Trigger condition	Data used
Broken authentication	API2:2023	A token is seen from three or more distinct IP addresses within a 5-minute window, indicating token sharing, replay or theft	In-memory address set
Unrestricted resource consumption	API4:2023	A token exceeds a request-rate threshold within a 60-second window, or a single request payload exceeds a size threshold	Timestamp deque, payload size
Business flow abuse	API6:2023	Transactional endpoints (checkout and profile_update) are called faster than manual human interaction would plausibly allow	Timestamp deque per endpoint
Broken object-level authorization	API1:2023	The target resource identifier does not belong to the authenticated caller	Ownership map, token subject
Broken function-level authorization	API5:2023	A token without administrative privilege calls an administrator-only endpoint	Administrator claim

Algorithm 4.1. Real-time request classification

```

for each incoming request r with token t:
  claims <- Decrypt(t); reject r if invalid or expired
  tracker.update(claims.jti, now, r.ip)
  rate <- tracker.count(claims.jti, 60 s)
  ips <- tracker.distinct_ips(claims.jti, 300 s)
  flags <- empty set
  if ips >= 3: flags += BROKEN_AUTH
  if rate > R_MAX or r.payload > P_MAX: flags += RESOURCE_USE
  if r.endpoint in {checkout, profile_update}
    and endpoint_rate > B_MAX: flags += BUSINESS_FLOW
  if r.record_id exists
    and owner(r.record_id) != claims.sub: flags += BOLA
  if r.endpoint is admin-only
    and not claims.is_admin: flags += BFLA
  store request log with (rate, ips); store flags with reasons
  return response

```

In Algorithm 4.1 the parameters R_MAX , P_MAX and B_MAX are the fixed thresholds for the request rate, the payload size and the transactional rate respectively.

4.2.4 DATA PERSISTENCE PACKAGE

This package defines the SQLAlchemy models and the SQLite database discussed in Section 3.4. Each processed request is persisted as a row in a request log, together with the live counts calculated for it at request time, so that a request can be explained precisely as it appeared when it arrived, without recomputing rolling windows. Category flags are stored in a separate table which points back to the request log. This allows a single request to be flagged in multiple categories. You don't need to install and maintain any database server, just use a single file database.

4.2.5 EXPLAINABILITY PACKAGE

This package maintains the machine-learning model of the system and its role is different from existing systems. The model does not drive the detection, it powers an on-demand explanation. The underlying features for the behaviour-based categories are payload size, response time, request rate and distinct address count, as listed in Table 4.3, which are also suitable inputs to an Isolation Forest. The feature table is built from the stored request logs with pandas, and the model is trained via a command-line script and saved as a model file.

Table 4.3. Features supplied to the Isolation Forest

Feature	Meaning	Related categories
payload_size	Size of the request payload	Unrestricted resource consumption
response_time	Time taken to serve the request	Resource consumption
request_rate	Requests by the token in the last 60 seconds	Resource consumption, business flow abuse
distinct_ip_count	Distinct addresses of the token in the last 5 minutes	Broken authentication

The Isolation Forest isolates observations by recursive random partitioning. The anomaly score of an observation x in a sample of n observations is given by Equation 4.3, where $E(h(x))$ is the average path length of x over the trees and the normalizing

constant $c(n)$ is given by Equation 4.4, with $H(i)$ the harmonic number. Scores close to one suggest anomalies. In the scikit-learn implementation, the decision function is negative for observations that the model thinks are anomalous and positive for those that the model thinks are normal.

$$s(x, n) = 2^{-E(h(x))/c(n)} \quad (4.3)$$

$$c(n) = 2H(n-1) - \frac{2(n-1)}{n} \quad (4.4)$$

The system calculates the decision function and a SHAP explanation restricted to the feature vector of that single request, when the analyst selects a particular request. The SHAP value of feature i , as shown in Eq. 4.5, is the average marginal contribution of the feature over all subsets S of the feature set F , and the TreeExplainer computes it exactly and efficiently for tree ensembles [12]. The cost is bounded no matter the traffic volume as the explanation is only computed on demand.

$$\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f_{S \cup \{i\}}(x_{S \cup \{i\}}) - f_S(x_S)] \quad (4.5)$$

This mechanism does not intend to explain the violations of BOLA and BFLA. Typically a BOLA request is not distinguishable from normal traffic by size, timing, or rate of the payload, and invalidity is based not on what the request looks like, but on who is making the request. The rule reason is shown for these categories in the dashboard. This observation is experimentally validated in Section 5.1.3.

4.2.6 ANALYST DASHBOARD PACKAGE

Figure 4.5 depicts the navigation of a three-level web dashboard that displays the results to the security analyst. The dashboard is a vanilla JS server-rendered HTML page which calls dedicated API endpoints. The first level displays a single card per category, with a live count of flagged requests and the most recent example. When you select a category the second level opens, showing you all flagged requests in that category, their endpoint, method, source address, token identifier, timestamp and the detailed reason why the rule fired. There are no hidden limits, so if a category has three flagged requests

you see all three at once. The third level is accessed by selecting one of the flagged requests, and calculates and displays its explanation on demand as horizontal bars proportional to the contribution of each feature.

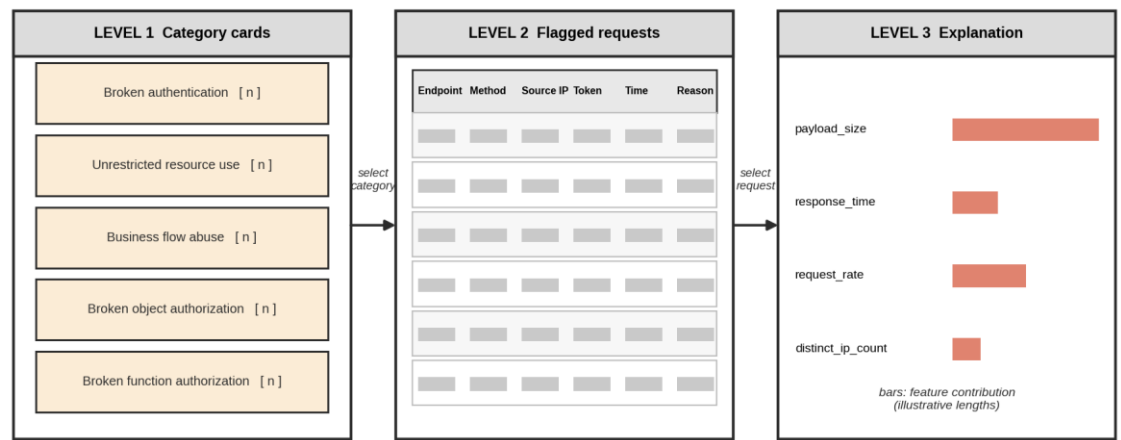


Figure 4.5. Layout of the three-level analyst dashboard

4.2.7 SIMULATION AND EVALUATION UTILITIES

Three scripts are used to support the demonstration and evaluation. The traffic simulator sends HTTP requests to the local server to create normal traffic and the five attack scenarios summarized in Table 4.4. Normal traffic is throttled at a human-like rate, and uses only the caller’s own records, and therefore does not trigger the BOLA rule. Each attack scenario logs in separately to get a fresh token and a fresh set of counters. The token inspector grabs a token from the running server and shows its structure – first without the key, then with it. The command-line pipeline script trains the Isolation Forest and generates the category summary.

Table 4.4. Traffic scenarios generated by the simulator

Scenario	Behaviour of the simulator	Expected category
Normal usage	Human-paced requests across the endpoints, using only records owned by the caller	None
Token reuse	The same token is presented with X-Forwarded-For addresses from several distinct IPs	Broken authentication
Request flooding	A rapid burst of requests and an oversized checkout payload	Unrestricted resource use
Rapid transactions	A rapid burst of profile update calls	Business flow abuse
Foreign record access	Requests for record identifiers owned by another user	BOLA
Privilege misuse	A non-administrator token calls the administrator endpoint	BFLA

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION RESULTS

Chapter 4 describes the implementation of the modules of the system and the analysis of the results. All results are obtained by using the full system on the development laptop. The server is started with Uvicorn , the traffic simulator generates the normal and attack traffic against it, and the pipeline script trains the model and generates the category summary. All data is synthetic as explained in Section 3.2.3 and the results should therefore be read as a functional verification of the design and not as a performance benchmark.

5.1.1 IMPLEMENTATION OF THE ENCRYPTED TOKEN MODULE

The token module's confidentiality property was verified by directly inspecting a token generated by the login endpoint. The token inspector takes a token from the running server and decodes it twice, first as an outsider who does not possess the server key, and then as the server. The outcome of the inspection is presented in the following listing.

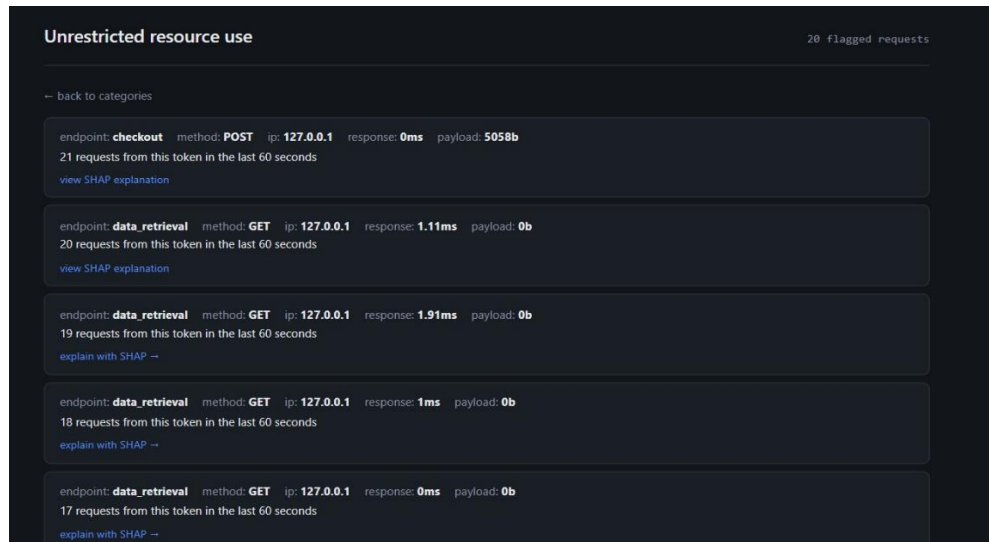


Figure5.1. Output of the token inspection script

Four properties confirmed by the inspection. The token has five segments separated by dots, as per the JWE compact serialization and not the three segments of a signed JWT. The header section is readable, as expected, because it only names the algorithm and the encryption method and does not reveal any user-specific information. The decoded portion of the ciphertext (without the server key) produces unreadable binary output with no apparent structure. Decryption with the correct key held by the server always recovers the original claims, the user identifier, the unique token identifier and the expiry timestamp. The decoded lengths also match the design: the authentication tag and the initialization vector are both 16 bytes, and the ciphertext is the same size as the serialized claims, as is the case for GCM. Table 5.1 summarizes the difference between the token issued by the system and a conventional signed JWT.

5.1.2 IMPLEMENTATION OF REAL-TIME CATEGORY DETECTION

The traffic simulator was run against the live backend to evaluate the detection path. The simulator first generated normal usage traffic at a human-like pacing and then injected the five attack scenarios of Table 4.4, one for each category, each with an independent login so that the behavioral counters were not shared across scenarios. 60 requests were recorded and categorized. Table 5.2 gives a summary of categories produced by the pipeline script. Figure 5.1 plots the same counts.

Category	Flagged count	Representative reason stored with the latest flag
Broken authentication	3	Token used from 5 distinct IPs in the last 5 minutes
Unrestricted resource use	12	21 requests from this token in the last 60 seconds
Business flow abuse	12	8 calls to <code>profile_update</code> in the last 60 seconds, faster than manual use
Broken object authorization	3	Requested a record not owned by the authenticated user
Broken function authorization	1	Non-admin token attempted an admin-only action

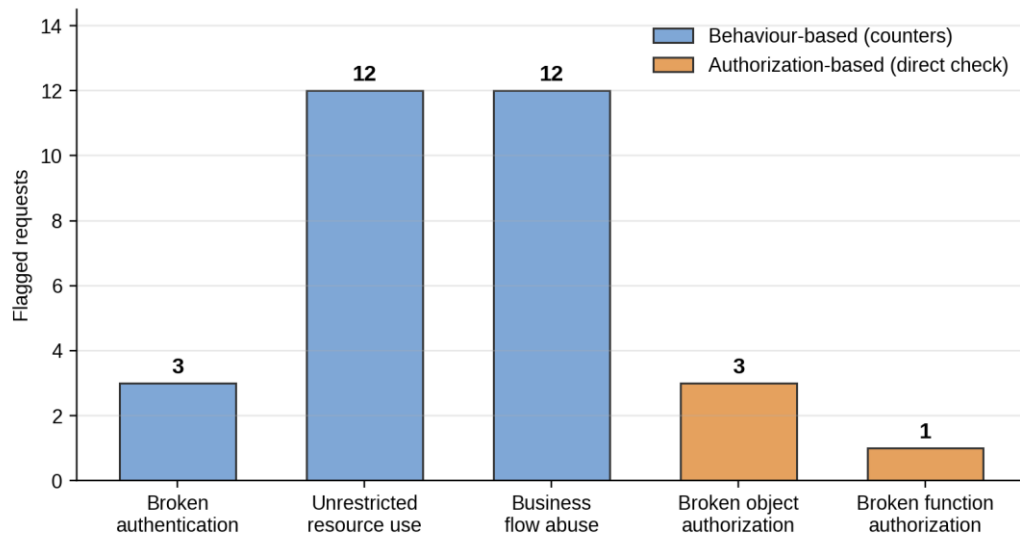
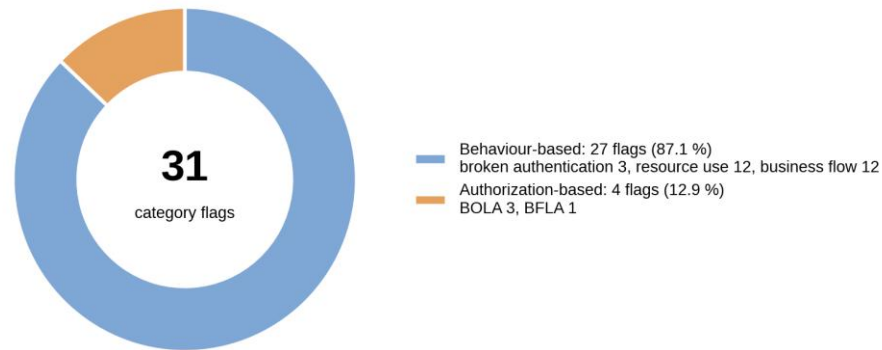


Figure 5.1. Number of flagged requests in each category

All five target categories were immediately correctly detected and the stored reason for each flag directly reflects the rule that fired. The authorization-based counts match the amount injected exactly: three requests for records owned by another user created three BOLA flags, and one call to the administrator endpoint with a normal token created a BFLA flag. The broken authentication flags started to appear when the token was seen from three different addresses. The counts are counts of flags , not separate requests . A request can be flagged in multiple categories , e.g. when a quick burst of profile updates also exceeds the general rate threshold . As shown in Figure 5.2, the behaviour-based rules raised 27 out of the 31 flags and the authorization-based rules raised 4.



5.1.3 IMPLEMENTATION OF ON-DEMAND EXPLANATION

After training on the logged traffic the on-demand explanation was requested for two flagged requests to test if the Isolation Forest would independently consider the same requests anomalous. The results are summarized in Table 5.3. The decision function for the request flagged as BOLA was positive, meaning that the model considered the request statistically normal in terms of payload size, timing and rate. Conversely, the explanation computed for the oversized-payload request that had been flagged as unrestricted resource use yielded a strongly negative score, with the payload size as the dominating contributing feature.

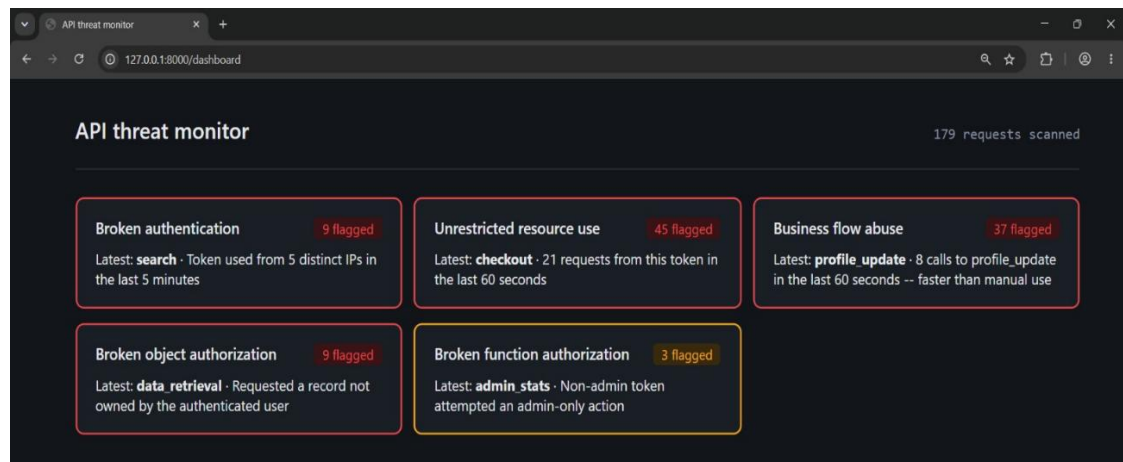


Figure 5.2. Behaviour-based versus authorization-based flags

It is an important result since it confirms the hybrid design. A BOLA request is normal-shaped and only wrong in that it is asking for data that belongs to someone else. So an anomaly detector that only looks at the shape of the traffic will never be able to detect it. This observation agrees with the error analysis of the nearest existing system [1] where BOLA and BFLA were some of the most commonly missed attacks. It motivates direct ownership and privilege checks for these two categories and proves the correctness of the on-demand explanation mechanism for the behavior-based categories.

5.1.4 IMPLEMENTATION OF THE ANALYST DASHBOARD

The results are shown to the analyst on the dashboard as described in Section 4.2.6. The first level of the dashboard populated with the counts and latest examples recorded in Table 5.2 is shown in Figure 5.3. Selecting a card shows all of the flagged requests of that category, with endpoint, method, source address, token identifier, timestamp and reason. Selecting a request computes its explanation.

The figure is a rendering of the layout with recorded values and not a screenshot of the running page.



Figure 5.3. First level of the analyst dashboard populated with the recorded results

5.2 DISCUSSION

In this section we discuss the results obtained on the simulated traffic. The exercised test cases and observed results are listed in Table 5.4. Each instance worked fine.

Table 5.4. Test cases and observed outcomes

No.	Test case	Expected result	Observed result
1	Log in with valid credentials	Five-segment JWE token returned	Five segments observed
2	Read the token without the key	Header readable, ciphertext unreadable	As expected
3	Decrypt the token with the server key	Claims recovered	sub, jti and exp recovered
4	Human-paced normal traffic	No category flag	No flags in the normal phase
5	Token used from several IP addresses	Broken authentication flag	3 flags
6	Request flood and oversized payload	Resource consumption flag	12 flags
7	Rapid profile updates	Business flow abuse flag	12 flags
8	Request for a record owned by another user	BOLA flag	3 flags
9	Ordinary token calls the admin endpoint	BFLA flag	1 flag
10	Explain the oversized-payload request	Anomalous, payload dominant	Strongly negative score
11	Explain the BOLA request	Model sees a normal request	Positive decision function
12	Category summary command	Five categories with counts	60 requests, five categories

First, the design avoids machine learning in the detection path: each category is determined by constant-time counters or by a direct comparison, so detection cost does not grow with the amount of logged traffic. The latency of the detection was not benchmarked in this phase and is listed as future work. Second, the hybrid design is not only convenient but also necessary, because the explanation experiment shows that the

authorization violations are invisible to the traffic-shape model. Third, token confidentiality does not require client side changes, as the encrypted token is used exactly like a signed token.

Table 5.5 shows the comparison of the proposed system with the closest existing system [1]. The proposed system does not claim to outperform the Isolation Forest and SHAP systems for general anomaly detection. Its goal is to focus on a fixed and named set of high impact vulnerability classes, with fast deterministic checks, leaving machine-learning for diagnostic explanations only.

Table 5.5. Comparison of the proposed system with the existing system

Aspect	Existing system	Proposed system
Detection basis	General anomaly score	Five named OWASP categories
Detection timing	Periodic or streaming scoring	Real-time, inline for each request
Explanation scope	Every flagged anomaly	On demand, for one analyst-selected request
Authorization violations (BOLA/BFLA)	Not addressed	Explicit rule-based checks
Detection-path ML inference	Required	Not required
Infrastructure	Stream processing and serverless cloud services	FastAPI, SQLite and scikit-learn on one laptop
Token confidentiality	Not addressed	JWE (AES-256-GCM) with 20-minute expiry

The evaluation has clear limitations. The attack scenarios were designed by the developers together with the thresholds, so the perfect match between the injected and detected counts only shows that the design works as intended and not that real attackers will adhere to the thresholds. The dashboard was validated by testing its interfaces and layout, not through a user study . The tests were performed on simulated traffic and not on real production traffic . Chapter 6 states the limitations clearly and motivates the future work..

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

In this report we have presented the problem of API monitoring against attacks. We have designed and implemented a lightweight system, whose detection is anchored to five named categories of the OWASP API Security Top 10. Each incoming request is categorized in real time, before the response is returned, using in-memory counters with constant-time complexity for the behaviour-based categories, and direct authorization checks for object-level and function-level violations. We keep an Isolation Forest model and a SHAP explainer, but only to explain a request selected by the analyst, so the explanation cost remains bounded regardless of the traffic volume. Results are presented in a three-level dashboard, ranging from category cards to flagged requests to an explanation.

The credential layer on which all the traffic depended was also hardened. Access tokens are issued as JSON Web Encryption objects using AES-256-GCM and a short fixed expiry. Direct inspection showed that an intercepted token reveals nothing except its algorithm header, while the server is still able to recover the claims. The system correctly and immediately identified all five targeted categories on simulated traffic. We also found that a BOLA request is scored normally by the Isolation Forest, confirming that authorization violations cannot be detected from the traffic shape and the hybrid design of rules and model is justified.

6.2 LIMITATIONS

There are some limitations that should be noted. The evaluation was performed on simulated, not actual production traffic and real attackers may not follow the specific thresholds used here. Detection is limited to the five targeted categories, thus an attack outside this taxonomy such as an injection attack would not be flagged, as no general purpose anomaly score is computed on the detection path.

The thresholds are fixed constants and not adaptively calibrated to the observed traffic. The analyst dashboard currently lacks authentication, does not persist analyst feedback (e.g. marking an incident resolved), and does not auto-refresh. For the tokens the implementation uses a single static key without rotation, so a compromised key would expose all the tokens issued with it, there is no refresh-token rotation, and the key is held as a static configuration value and not in a secrets manager.

6.3 FUTURE WORK

There are several extensions planned for the next phase of the project:

1. Adaptive thresholds. The fixed thresholds will be replaced by calibrated values on the observed traffic of each endpoint.
2. Feedback from analyst. The dashboard will store analyst decisions, e.g. marking an incident as resolved, and will be protected by authentication and refresh automatically, using the same pattern as analyst-in-the-loop systems [15].
3. Token hardening Key rotation, addition of refresh tokens and a secrets manager will be added, in accordance with the NIST key management recommendations [23].
4. Using integration tokens. Token-usage behavior (reuse across addresses, request velocity per token) will be incorporated directly into the detection model, so that traffic-level detection and token security work as one pipeline.
5. Deployment and scale. We will move the tracker to a shared store to be able to use several worker processes, the database will be replaced by a server database, the system will be packaged in a container and we will benchmark the detection latency.
6. Real-world evaluation. The system will be tested on real traffic in controlled and isolated environment. Further OWASP categories will be added.
7. Strength of explanations. The robustness of the on-demand explanations to adversarial manipulation will be investigated, as perturbation-based explanations can be misled [13].

REFERENCES

1. A. Mittal, P. Keshap, and A. Hosabettu, “AI-driven real-time API security: Explainable threat detection for cloud environments,” in *Proc. Int. Carnahan Conf. Security Technology (ICCST)*, 2025, doi: 10.1109/ICCST63435.2025.11293949.
2. OWASP Foundation, “OWASP API Security Top 10,” 2023. [Online]. Available: <https://owasp.org/API-Security/>
3. V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Comput. Surv.*, vol. 41, no. 3, Art. no. 15, 2009.
4. F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.
5. F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation-based anomaly detection,” *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, 2012.
6. M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, “LOF: Identifying density-based local outliers,” in *Proc. ACM SIGMOD*, 2000, pp. 93–104.
7. B. Schölkopf *et al.*, “Estimating the support of a high-dimensional distribution,” *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
8. S. Hariri, M. Carrasco Kind, and R. J. Brunner, “Extended isolation forest,” *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 4, pp. 1479–1489, 2021.
9. Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, “Kitsune: An ensemble of autoencoders for online network intrusion detection,” in *Proc. NDSS*, 2018.
10. M. T. Ribeiro, S. Singh, and C. Guestrin, “‘Why should I trust you?’: Explaining the predictions of any classifier,” in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.
11. S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
12. S. M. Lundberg *et al.*, “From local explanations to global understanding with explainable AI for trees,” *Nature Mach. Intell.*, vol. 2, pp. 56–67, 2020.
13. D. Slack *et al.*, “Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods,” in *Proc. AAAI/ACM AIES*, 2020, pp. 180–186.
14. R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. IEEE Symp. Security and Privacy*, 2010, pp. 305–316.
15. K. Veeramachaneni *et al.*, “AI2: Training a big data machine to defend,” in *Proc. IEEE Big Data Security*, 2016, pp. 49–54.
16. M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” IETF RFC 7519, May 2015.
17. M. Jones, J. Bradley, and N. Sakimura, “JSON Web Signature (JWS),” IETF RFC 7515, May 2015.

18. M. Jones and J. Hildebrand, “JSON Web Encryption (JWE),” IETF RFC 7516, May 2015.
19. M. Jones, “JSON Web Algorithms (JWA),” IETF RFC 7518, May 2015.
20. M. Jones, “JSON Web Key (JWK),” IETF RFC 7517, May 2015.
21. D. McGrew and J. Viega, “The Galois/Counter Mode of Operation (GCM),” NIST Submission, 2004.
22. National Institute of Standards and Technology, “Advanced Encryption Standard (AES),” FIPS PUB 197, November 2001.
23. E. Barker, “Recommendation for Key Management: Part 1 – General,” NIST Special Publication 800-57 Part 1 Revision 5, May 2020.
24. D. Hardt, “The OAuth 2.0 Authorization Framework,” IETF RFC 6749, Oct. 2012.
25. T. Lodderstedt, M. McGloin, and P. Hunt, “OAuth 2.0 Threat Model and Security Considerations,” IETF RFC 6819, January 2013.
26. D. Fett, R. Küsters, and G. Schmitz, “A comprehensive formal security analysis of OAuth 2.0,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2016, pp. 1204–1215.
27. N. Sakimura, J. Bradley, and N. Agarwal, “Proof Key for Code Exchange by OAuth Public Clients (PKCE),” IETF RFC 7636, September 2015.
28. J. Richer, Ed., “OAuth 2.0 Token Introspection,” IETF RFC 7662, October 2015.
29. N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, “OpenID Connect Core 1.0,” OpenID Foundation, November 2014.
30. E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” IETF RFC 8446, August 2018.

Lightweight API Security Monitoring and Anomaly Detection: Real-Time Detection of Five OWASP-Based Vulnerability Categories

Mrs. M.Shyni

Assistant Professor (SS)
Rajalakshmi Engineering College
Chennai, India
shyni.m@rajalakshmi.edu.in

Varsha B

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
prathikshathila@gmail.com

Tejushree Sanjeevi Kumar

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
tejushreebv@gmail.com

Valluru Varshini

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
valluruvarshini9@gmail.com

Abstract—API anomaly detection utilizing machine learning with Isolation Forest and SHAP has been shown to work successfully in an enterprise context. However, conventional methods usually score the traffic after the fact and explain each of the detected anomalies, which is not scalable when the number of alerts grows. This paper describes a lightweight solution that focuses on just five well defined attack types from OWASP API Security Top 10: broken authentication, unrestricted resource consumption, business flow abuse, broken object-level authorization (BOLA) and broken function-level authorization (BFLA). Each request is analyzed against these five types of attacks in real-time by using simple deterministic rules (Algorithm 1) instead of extracting machine learning inference on the detection path, and rule checking itself takes just 0.21 μ s on average, and total hot path time is 1.50 μ s. In the case study conducted with labelled data (80 normal requests, 60 combined-demo requests, and independent attacks in each class), we observed that four out of five classes obtained perfect marks in precision, recall, and F1 (1.00 in each case). The fifth class, business flow abuse, achieved a high recall score of 1.00 but low precision of 0.16 (F1 = 0.28). This low precision value was only encountered when past data was examined. Another observation was that we discovered authorization violations, for instance, BOLA, which were marked as statistically normal under Isolation Forest. This indicated that violations won't be exposed through traffic pattern anomaly detection.

I. INTRODUCTION

Modern cloud-based services are built around Application Programming Interfaces (APIs) which have become the target of cybercriminals because of their exposure on the public Internet. Earlier research has found out that some methods for unknown anomaly detection like Isolation Forest have been successfully used along with SHAP to detect strange API traffic in an enterprise environment [1]. Nevertheless, such systems usually work like offline solutions: the traffic is monitored, analyzed, and explained afterward and the detected anomalies are accounted for completely. This paper adopts a novel technique that is influenced by two practical concerns. Instead of depending upon a general anomaly score to determine what qualifies as “unusual,” detection is based on five specific attack categories from the OWASP API Security Top 10 white paper

[2]. One very widely used industry reference to the most popular ways that APIs get exploited is presented. Second, rather than scoring traffic in batches, each incoming request is identified using fast threshold tests, thus eliminating the requirement for active learning inference as part of the detection process. One machine learning model, Isolation Forest, is retained, but instead of being incorporated into the detection process, the model is used to power an on-demand explanation solution that is only invoked when an analyst decodes the flagged request. As a result, the ability to explain outcomes using SHAP is still preserved while excluding the computation of the explanations for every flagged event. The remainder of this paper is structured as follows: Section II provides a survey of works related to the topic; Section III describes the architecture of the system and outlines five target categories; Section IV discusses implementation issues; Section V presents an experimental study; Section VI compares the proposed method with other existing solutions; Section VII discusses limitations; Section VIII provides directions for future research.

II. RELATED WORK

Anomaly detection comprises various techniques used for monitoring the behavior of applications, including statistical methods, distance-based techniques, density-based methods, and data learning, all relying on different assumptions about the normal state of operation and the availability of labels [3]. The isolation forest technique is based on separating the comprehended subjects through a number of random partitions without assessing the distribution of normal data [4], [5]. For the purpose of comparison, it is also essential to mention such unsupervised techniques as local outlier factor (LOF) [6] and single-class support vector machine [7]. Advanced versions of the isolation forest algorithm make use of the hyperplanes that are randomly oriented to prevent potential issues with the axes alignment [8]. In the current research, the classic isolation forest method is employed; it should be noted that it is employed only

for the purpose of needing the explanation, rather than the task of detection. There is a similar application developed by Mittal et al. that uses the isolation forest method coupled with the shap library to detect threats in APIs and reports 0.94 precision and 0.91 recall achieved with the help of simulation of fintech companies traffic streams [1]. The presented model, like other models that use the anomaly detection technique. The OWASP API Security Top 10 [2] documents the most common API vulnerability classes observed in practice, including Broken Object Level Authorization, Broken Authentication, Broken Function Level Authorization, and Unrestricted Resource Consumption. Although widely referenced in industry security guidance, this taxonomy is less commonly used as the direct basis for a real-time, rule-based detection architecture in the anomaly-detection literature, which more often relies on general-purpose statistical scoring.

For local interpretability, LIME approximates a model's behavior around an individual prediction using an interpretable surrogate [10], whereas SHAP provides an additive feature-attribution framework [11]. TreeExplainer introduced efficient, tree-specific attribution algorithms [12]. Explanations require caution, as Slack et al. demonstrate that adversarially constructed classifiers can mislead perturbation-based LIME and SHAP explanations [13].

Operational intrusion detection also requires attention to analyst workload. Sommer and Paxson discuss why successful machine-learning experiments do not automatically translate into effective deployed intrusion detection [14]. The analyst-in-the-loop AI2 system combines outlier detection with analyst feedback and supervised learning [15], providing context for the dashboard and feedback loop discussed as future work in Section VIII.

III. PROPOSED METHODOLOGY

A. System Overview

The system proposed focuses on five categories chosen from the OWASP API Security Top 10 based on their applicability to the given type of endpoints in the simulated backend. The selected categories are: broken authentication, unregulated resource consumption, business flow misuse, broken object-level authorization (BOLA), and broken function-level authorization (BFLA). The incoming request is evaluated against all the categories in an inline fashion before the actual response is produced. Additionally, another Isolation Forest model is used only to provide an on-demand SHAP explanation

for a particular flagged request when necessary.

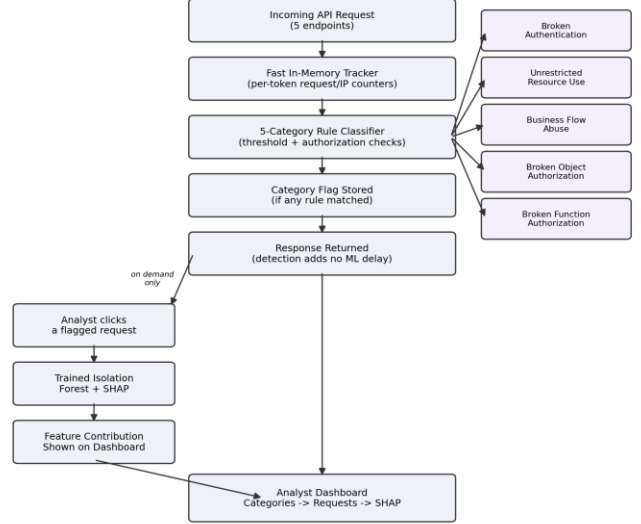


Fig. 1. Real-time category detection pipeline (center/right) with on-demand SHAP explanation (left).

B. Fast, In-Memory Behavioral Tracking

Two out of five categories heavily rely on recent behavior history such as the number of requests made by the token or the number of different IP addresses it has been associated with. To eliminate the need for constant recomputation of these figures by querying through request records considering increasing traffic rates, there are small rolling windows of short-term memory maintained per token that store a deque of timestamps and a set of IP addresses recently used. Thus, this allows making each verification within a constant time frame.

C. Category Detection Rules

The Twentieth Category: Detection Rules

- **Broken authentication:** This rule gets triggered when the same authentication token comes from three or more different IP addresses within a period of five minutes, which is an indication of token sharing, replays or even token theft.

- **Unrestricted resource consumption:** This rule gets activated when a token goes beyond a certain threshold of requests within a minute or when it goes over the allowed request size.

- **Business flow abuse:** This rule comes into play when certain action endpoint (checkout and profile_update) is invoked excessively when compared to other endpoints.

- **Broken object-level authorization:** This rule gets triggered when a request targets a resource identifier which does not belong to authenticated principal, which can be verified by means of the ownership check.

- **Broken function-level authorization:** This rule gets activated when a token without admin rights tries to call an admin endpoint.

In summary, the three former rules are behaviorbased, while two latter rely on the fact that requests permissions are compared against their real rights.

Algorithm 1: Per-request category classification

Input: endpoint e , payload size s , requests_last_60s r , distinct_ips_last_5min d , bola_flag b , bfla_flag f .
Output: set of matched categories (0, 1, or many).
1. $flags \leftarrow \{\}$
2. if $d \geq 3$: $flags \leftarrow flags \cup \{broken_authentication\}$
3. if $r \geq 10$: $flags \leftarrow flags \cup \{unrestricted_resource_use\}$
else if $s \geq 2000$: $flags \leftarrow flags \cup \{unrestricted_resource_use\}$
4. if $e \in \{checkout, profile_update\}$ and $r \geq 5$:
 $flags \leftarrow flags \cup \{business_flow_abuse\}$
5. if b : $flags \leftarrow flags \cup \{broken_object_authorization\}$
6. if f : $flags \leftarrow flags \cup \{broken_function_authorization\}$
7. return $flags$

D. On-Demand SHAP Explanation

For categories 1-3, the salient behavioral aspects of the requests, i.e., payload size, response time, request rate, and unique IP count, are suitable input for a trained Isolation Forest model. Rather than automatically generating a SHAP explanation for each flagged request, it is only generated when an analyst chooses a specific request from the dashboard using the decision function of the model and SHAP explainer specific to that request's feature vector. Thus, the need for computation is reduced while keeping the possibility of using explanations as a diagnostic tool. BOLA and BFLA violations, however, cannot be explained using this kind of approach: BOLA requests cannot be differentiated in most instances from normal requests based on the equivalent criteria of size and time of the request. It is violated because of the person submitting the request, and not because of what the request is.

E. Analyst Dashboard

The output will be demonstrated through a dashboard on the web with three levels. The first level will consist of one card for every category, showing the number of flagged applications and the latest instance as well. The second level will give all flagged requests under a certain category when pressing the category. Such requests will include all necessary data: endpoint, method, source IP, token ID, date & time, and the reason of the rule execution. The third level will show the SHAP justification of the flagged request if it is requested.

F. Computational Complexity

The in-memory tracker (described in Section III-B) is built such that its resource cost per request does not increase with the traffic volume, unlike the previous solution that recalculated token behavior by going through the request log for each check. Each timestamp added to a token deque is removed at most once when it goes outside the 60-second period, which means that the total number of operations W done on N requests is limited by $W(N) \leq N$ (1), resulting in the amortized request cost of $W(N)/N = O(1)$ (2). The distinct IP address table is operated similarly and has the amortized cost of $O(m)$, where m is the number of IP addresses associated with that token in the 5-minute window; m was at most 5 in all tests in Section V. Together with a constant number of operations shown in Algorithm 1, this leads to $O(1)$ amortized cost under the assumption that m remains small, which is exactly what is demonstrated in Table II (1.5 μ s).

IV. IMPLEMENTATION

The implementation of the entire system is done through Python with the use of FastAPI for the backend layer of API, SQLAlchemy and SQLite as the persistent storage, and through the use of scikit-learn packages to include the Isolation Forest model for instant explanations. The feature of tracking the behavior in real-time is done through thread-safe rolling windows for every token, using deques for keeping time stamps and dictionaries for IP addresses recently used that helps avoid repeated database lookups on the path for detection. The access tokens are created by the means of use of JSON Web Encryption method with the support of the algorithm AES-256-GCM and are of short expiry.

V. RESULTS

A. Detection Outcomes on Live Traffic

The evaluation was performed with the help of simulated traffic against a backend working in real-time: normal traffic was created to simulate direct interaction by human users and was followed by five intentionally generated cases for every targeted class. Separate credentials were used to prevent the sharing of statistical information across different cases. Thus, in total, there were 60 requests with their statuses obtained.

The summary of the results is given in Table I: all five classes were identified correctly, showing the reasons for the detection perfectly correlated with the rules applied. For those on the basis of BOLA, SHAP explanation was computed to find out whether the Isolation Forest algorithm would generate detection without being trained on the data previously. It turned out that the detection score is a normal one indicating that the request is normal considering payload dimension, timing, and frequency. Thus, the conclusion can be made that violations of the authorization nature cannot be detected through only traffic shapes. In contrast, the explanation of one of the operating cases, the oversized payload one, was negative showing that the payload dimension was the most possible reason confirming the efficiency of the explanation mechanism for the cases based on the categories

TABLE I
CATEGORY DETECTION RESULTS

Category	Count	Representative Reason
Broken authentication	3	Token used from 5 distinct IPs in 5 minutes
Unrestricted resource use	12	21 requests from one token in 60 seconds
Business flow abuse	12	8 calls to profile_update in 60 seconds
Broken object authorization	3	Requested a record not owned by the caller
Broken function authorization	1	Non-admin token called an admin-only endpoint

B. Classification Latency

In order to validate that classification is adequately quick for the detection route as asserted in Section III, the cost was

measured directly while preventing Algorithm 1 from using the in-memory tracker that comes before this algorithm in the live middleware. The results of the measurements conducted in 20,000 timed calls are displayed in Table II.

TABLE II
CLASSIFICATION LATENCY (μ S, $N = 20,000$)

Stage	Mean	Median	P95	P99
Rule evaluation only (Alg. 1)	0.21	0.17	0.32	0.38
Full hot path (tracker + Alg. 1)	1.50	1.36	2.15	3.41

Rule checks by themselves require approximately 0.21 microseconds, while the full hot path, inclusive of updating of the in-memory tracker (adding and eliminating of the deque of timestamps per token and the IP set) lasts about 1.50 microseconds. Both these values are negligible compared to the usual time required by network and database I/O for making requests, while no trained Isolation Forest model is needed for performing detection at request time, confirming that rule detection is not dependent on any machine-learning inference made at that time.

C. Precision, Recall, and F1

The numbers in Table I indicate that there were a total of five detected categories under traffic handling humans, but that these do not provide the number of false positives on their own. A separate analysis was conducted under ground truth conditions with 80 normal requests and five separate attack types, with one being conducted for each category using a different token for each. True positives, false positives, and false negatives were determined for each category as detailed below. The precision, recall, and F1 scores are calculated according to the following formulae:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

The results are given in Table III for each of the categories tested.

TABLE III
PRECISION, RECALL, AND F1 PER CATEGORY

Category	Prec.	Recall	F1
Broken authentication	1.00	1.00	1.00
Unrestricted resource use	1.00	1.00	1.00
Business flow abuse	0.16	1.00	0.28
Broken object authorization	1.00	1.00	1.00
Broken function authorization	1.00	1.00	1.00

Perfect recall and precision were obtained in four of the categories. Except Business flow abuse, which had a low precision (0.16), resulting from false positives only: as this evaluation involved the deliberate compression of time (with requests being sent back to back and without the intervals of several seconds that simulate human behavior in the demonstration described in Section V-A), a few ordinary tokens gathered enough requests over 60 seconds in the shared

counter and managed to cross the threshold for business flow abuse based upon a `profile_update` or `checkout` request. This situation could not happen in the real-life demonstration (Table I) because no false positives were recorded, as the real-life demonstration made it possible to keep the shared request-counting window low enough to avoid abuse threshold crossing. Thus, this finding is explicitly stated rather than put aside: it allows one to conclude that the threshold applied to business flow abuse is sensitive to elapsed time and supports the ideas of severity threshold adjustment and proper calibration outlined in Section VII.

D. Explaining the Business-Flow Precision Result

The 0.16 accuracy recorded in Section V-C can be calculated directly from the way in which the benchmark was created. There are ten normal tokens that make $K=8$ requests quickly, where there are 4 endpoint types and only 2 types contribute to the business-flow rule (`checkout` and `profile_update`), meaning the per-request likelihood of getting to the endpoint is $p=2/4=0.5$. Since all K requests happen in the same 60-second interval, the shared request counter reaches the limits $\theta=5$ at the 5th request; therefore, every other business-flow-endpoint call from that token, requests 5-8 will be flagged. The expected number of these calls for each token is basically p times the number of requests which can be served. Thus, when $T = 10$ tokens are taken into account the expected number of false positive becomes $E[\text{FP}] = T \cdot p \cdot \max(0, K - \theta + 1)$ (6). When one puts $T = 10$, $p = 0.5$, $K = 8$, $\theta = 5$ into this formula we receive $E[\text{FP}] = 10 \cdot 0.5 \cdot 4 = 20$ which is very close to our practical observation of 21 (Table III). Given the fact that $\text{TP} = 4$ is constant due to the scenario the precision we can expect is $\text{TP} / (\text{TP} + E[\text{FP}]) = 4 / 24 \approx 0.17$ (7). This final value is just one negative result different than the actual value of 0.16 (4/25). This means that it can be said for sure that lower precision is not the defect of implementation but the result of analytics.

E. Throughput Bound Under Load

Since the hot path specified in Table II ($c = 1.50 \mu$ s per request) occurs synchronously before the response is returned, continuous request rates approaching its inverse lead to a queue delay on top of c . Assuming the detection process is modeled as a single service station with deterministic service time c and arrival rate R , the volume of traffic can be represented as

$$\rho = R \cdot c \quad (8)$$

and according to Pollaczek-Khinchine formula, the mean waiting time for M/D/1 queue can be calculated as

$$W_q = \rho / (2(1 - \rho)) \cdot c, \text{ for } \rho < 1 \quad (9)$$

thus the expected total time of detection is

$$c + W_q = c \cdot (1 + \rho / (2(1 - \rho))) \quad (10)$$

In order for this to stay within 10% of c only ($\rho / (2(1 - \rho)) \leq 0.1$), we have to have $\rho \leq 0.167$, or $R \leq 0.167/c \approx 111,000$ requests per second for the investigated single core — many times greater than any expected API loads for the simulated backend, pointing to the fact that the detection process will unlikely be the bottleneck for the system.

VI. COMPARISON WITH EXISTING SYSTEMS

The differences are summarized in Table IV. The system does not claim superior performances over existing Isolation Forest-SHAP systems regarding anomaly detection. Instead, the goal of the current system is to focus on a set of different pre-defined classes of critical vulnerabilities with fast, deterministic checks and using machine learning to provide reasons for the occurrence and not to detect anomalies themselves.

TABLE IV
COMPARISON WITH EXISTING SYSTEM

Aspect	Existing System	Proposed System
Detection basis	General anomaly score	Five named OWASP categories
Detection timing	Periodic batch scoring	Real-time, inline per request
Explanation scope	Every flagged anomaly	On-demand, per analyst request
Authorization violations	Not addressed	Explicit rule-based checks
Detection-path ML inference	Required	Not required (rule-based)

VII. LIMITATIONS

Certain limitations have to be recognized. First, instead of using real-world production traffic, evaluation was done using simulated traffic; attackers may not behave the same way as envisioned based on the values used in the study. Next, detection is restricted only to the five chosen categories. An attack falling outside this range, for example, an injection attack, would not be considered, as there is no general-purpose anomaly score on the detection route. Third, the category thresholds, such as request-rate limits, are viewed as constants; these can lead to a decrease in the precision when assessing business flow abuse based on the results of the evaluation done under compressed time, as illustrated in Section V-C; while no false positives were found in real life.

ACKNOWLEDGMENT

Fig. 1 in this paper was generated using Claude (Anthropic), an AI tool. All other content, including the implementation, experiments, and results, is the authors' own work.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we introduced a novel API threat detection system that ensures real-time detection and is based on OWASP API Security Top 10 principles. The proposed system encompasses a combination of conventional detection methodologies based on API-specific behavioral and authorization practices with the use of SHAP-based explanations for cases when analysts want to investigate any specific incident. We conducted experiments to prove that our detection methodology worked correctly and instantly detected malicious activities in all five categories. The experiments confirmed that our system successfully detect

anomalies in traffic flow, however, in terms of detection of authorization issues like BOLA, we emphasize that traffic anomaly techniques alone cannot detect breaches thus leading us to hybrid approach that we suggested in this paper. Future work will be focused on calibration of adaptive thresholds, development of analyst feedback system for marking incidents as resolved, provision of real-time reports, involvement of token behavior in detection process, provision of token ciphering system as separate topic and testing of our system in real setups.

REFERENCES

- [1] A. Mittal, P. Keshap, and A. Hosabettu, "AI-driven real-time API security: Explainable threat detection for cloud environments," in *Proc. 2025 IEEE Int. Carnahan Conf. Security Technology (ICCST)*, Oct. 2025, pp. 1–7, doi: 10.1109/ICCST63435.2025.11293949.
- [2] OWASP Foundation, "OWASP API Security Top 10," 2023. [Online]. Available: <https://owasp.org/API-Security/>
- [3] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, Dec. 2008, pp. 413–422.
- [5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 1–39, 2012.
- [6] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, May 2000, pp. 93–104.
- [7] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [8] S. Hariri, M. C. Kind, and R. J. Brunner, "Extended isolation forest," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 4, pp. 1479–1489, 2021.
- [9] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," *arXiv preprint arXiv:1802.09089*, 2018.
- [10] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?": Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Aug. 2016, pp. 1135–1144.
- [11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [12] S. M. Lundberg *et al.*, "Explainable AI for trees: From local explanations to global understanding," *arXiv preprint arXiv:1905.04610*, 2019.
- [13] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods," in *Proc. AAAI/ACM Conf. AI, Ethics, and Society*, Feb. 2020, pp. 180–186.
- [14] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. 2010 IEEE Symp. Security and Privacy*, May 2010, pp. 305–316.
- [15] K. Veeramachaneni, I. Arnaldo, V. Korrapati, C. Bassias, and K. Li, "AP2: Training a big data machine to defend," in *Proc. 2016 IEEE 2nd Int. Conf. Big Data Security on Cloud (BigDataSecurity)*, Apr. 2016, pp. 49–54.

dick deadeye

Paper1_IEEE_format humanized p1--.docx

 Research PB PB A'Sharqiyah University, Oman

Document Details

Submission ID

trn:oid::1:3666128610

Submission Date

Oct 1, 2026, 8:37 PM GMT+4

Download Date

Oct 1, 2026, 8:41 PM GMT+4

File Name

Paper1_IEEE_format_humanized_p1--.docx

File Size

200.7 KB

5 Pages**4,101 Words****22,394 Characters**

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.



0 AI-generated 0%

Likely generated or likely generated and revised by AI.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Lightweight API Security Monitoring and Anomaly Detection: Real-Time Detection of Five OWASP-Based Vulnerability Categories

Mrs. Shyni

Assistant Professor (SS)
Rajalakshmi Engineering College
Chennai, India
shyni.m@rajalakshmi.edu.in

Varshabala

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
prathikshathila@gmail.com

Tejushree Sanjeevi Kumar

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
tejushreebv@gmail.com

Valluru Varshini

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
valluruvarshini9@gmail.com

Abstract—API anomaly detection utilizing machine learning with Isolation Forest and SHAP has been shown to work successfully in an enterprise context. However, conventional methods usually score the traffic after the fact and explain each of the detected anomalies, which is not scalable when the number of alerts grows. This paper describes a lightweight solution that focuses on just five well defined attack types from OWASP API Security Top 10: broken authentication, unrestricted resource consumption, business flow abuse, broken object-level authorization (BOLA) and broken function-level authorization (BFLA). Each request is analyzed against these five types of attacks in real-time by using simple deterministic rules (Algorithm 1) instead of extracting machine learning inference on the detection path, and rule checking itself takes just 0.21 μ s on average, and total hot path time is 1.50 μ s. In the case study conducted with labelled data (80 normal requests, 60 combined-demo requests, and independent attacks in each class), we observed that four out of five classes obtained perfect marks in precision, recall, and F1 (1.00 in each case). The fifth class, business flow abuse, achieved a high recall score of 1.00 but low precision of 0.16 (F1 = 0.28). This low precision value was only encountered when past data was examined. Another observation was that we discovered authorization violations, for instance, BOLA, which were marked as statistically normal under Isolation Forest. This indicated that violations won't be exposed through traffic pattern anomaly detection.

I. INTRODUCTION

Modern cloud-based services are built around Application Programming Interfaces (APIs) which have become the target of cybercriminals because of their exposure on the public Internet. Earlier research has found out that some methods for unknown anomaly detection like Isolation Forest have been successfully used along with SHAP to detect strange API traffic in an enterprise environment [1]. Nevertheless, such systems usually work like offline solutions: the traffic is monitored, analyzed, and explained afterward and the detected anomalies are accounted for completely. This paper adopts a novel technique that is influenced by two practical concerns. Instead of depending upon a general anomaly score to determine what qualifies as “unusual,” detection is based on five specific attack categories from the OWASP API Security Top 10 white paper

[2]. One very widely used industry reference to the most popular ways that APIs get exploited is presented. Second, rather than scoring traffic in batches, each incoming request is identified using fast threshold tests, thus eliminating the requirement for active learning inference as part of the detection process. One machine learning model, Isolation Forest, is retained, but instead of being incorporated into the detection process, the model is used to power an on-demand explanation solution that is only invoked when an analyst decodes the flagged request. As a result, the ability to explain outcomes using SHAP is still preserved while excluding the computation of the explanations for every flagged event. The remainder of this paper is structured as follows: Section II provides a survey of works related to the topic; Section III describes the architecture of the system and outlines five target categories; Section IV discusses implementation issues; Section V presents an experimental study; Section VI compares the proposed method with other existing solutions; Section VII discusses limitations; Section VIII provides directions for future research.

II. RELATED WORK

Anomaly detection comprises various techniques used for monitoring the behavior of applications, including statistical methods, distance-based techniques, density-based methods, and data learning, all relying on different assumptions about the normal state of operation and the availability of labels [3]. The isolation forest technique is based on separating the comprehended subjects through a number of random partitions without assessing the distribution of normal data [4], [5]. For the purpose of comparison, it is also essential to mention such unsupervised techniques as local outlier factor (LOF) [6] and single-class support vector machine [7]. Advanced versions of the isolation forest algorithm make use of the hyperplanes that are randomly oriented to prevent potential issues with the axes alignment [8]. In the current research, the classic isolation forest method is employed; it should be noted that it is employed only

for the purpose of needing the explanation, rather than the task of detection. There is a similar application developed by Mittal et al. that uses the isolation forest method coupled with the shap library to detect threats in APIs and reports 0.94 precision and 0.91 recall achieved with the help of simulation of fintech companies traffic streams [1]. The presented model, like other models that use the anomaly detection technique. The OWASP API Security Top 10 [2] documents the most common API vulnerability classes observed in practice, including Broken Object Level Authorization, Broken Authentication, Broken Function Level Authorization, and Unrestricted Resource Consumption. Although widely referenced in industry security guidance, this taxonomy is less commonly used as the direct basis for a real-time, rule-based detection architecture in the anomaly-detection literature, which more often relies on general-purpose statistical scoring.

For local interpretability, LIME approximates a model's behavior around an individual prediction using an interpretable surrogate [10], whereas SHAP provides an additive feature-attribution framework [11]. TreeExplainer introduced efficient, tree-specific attribution algorithms [12]. Explanations require caution, as Slack et al. demonstrate that adversarially constructed classifiers can mislead perturbation-based LIME and SHAP explanations [13].

Operational intrusion detection also requires attention to analyst workload. Sommer and Paxson discuss why successful machine-learning experiments do not automatically translate into effective deployed intrusion detection [14]. The analyst-in-the-loop AI2 system combines outlier detection with analyst feedback and supervised learning [15], providing context for the dashboard and feedback loop discussed as future work in Section VIII.

III. PROPOSED METHODOLOGY

A. System Overview

The system proposed focuses on five categories chosen from the OWASP API Security Top 10 based on their applicability to the given type of endpoints in the simulated backend. The selected categories are: broken authentication, unregulated resource consumption, business flow misuse, broken object-level authorization (BOLA), and broken function-level authorization (BFLA). The incoming request is evaluated against all the categories in an inline fashion before the actual response is produced. Additionally, another Isolation Forest model is used only to provide an on-demand SHAP explanation

for a particular flagged request when necessary.

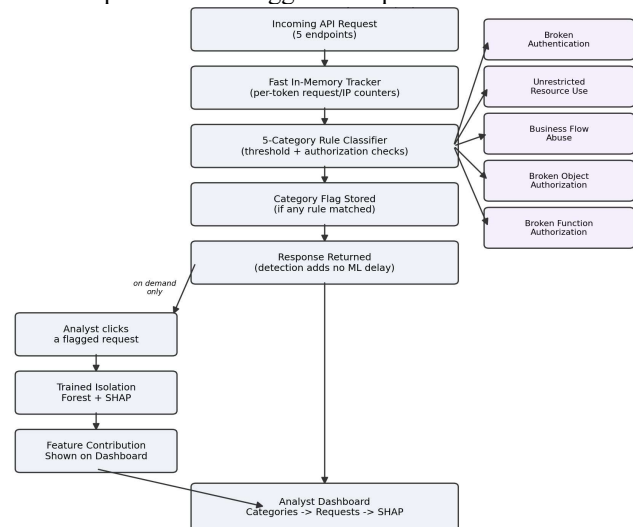


Fig. 1. Real-time category detection pipeline (center/right) with on-demand SHAP explanation (left).

B. Fast, In-Memory Behavioral Tracking

Two out of five categories heavily rely on recent behavior history such as the number of requests made by the token or the number of different IP addresses it has been associated with. To eliminate the need for constant recomputation of these figures by querying through request records considering increasing traffic rates, there are small rolling windows of short-term memory maintained per token that store a deque of timestamps and a set of IP addresses recently used. Thus, this allows making each verification within a constant time frame.

C. Category Detection Rules

The Twentieth Category: Detection Rules

- **Broken authentication:** This rule gets triggered when the same authentication token comes from three or more different IP addresses within a period of five minutes, which is an indication of token sharing, replays or even token theft.

- **Unrestricted resource consumption:** This rule gets activated when a token goes beyond a certain threshold of requests within a minute or when it goes over the allowed request size.

- **Business flow abuse:** This rule comes into play when certain action endpoint (checkout and profile_update) is invoked excessively when compared to other endpoints.

- **Broken object-level authorization:** This rule gets triggered when a request targets a resource identifier which does not belong to authenticated principal, which can be verified by means of the ownership check.

- **Broken function-level authorization:** This rule gets activated when a token without admin rights tries to call an admin endpoint.

In summary, the three former rules are behaviorbased, while two latter rely on the fact that requests permissions are compared against their real rights.

Algorithm 1: Per-request category classification

Input: endpoint e , payload size s , requests_last_60s r , distinct_ips_last_5min d , bola_flag b , bfla_flag f .

Output: set of matched categories (0, 1, or many).

```

1. flags  $\leftarrow \{\}$ 
2. if  $d \geq 3$ : flags  $\leftarrow$  flags  $\cup$  {broken_authentication}
3. if  $r \geq 10$ : flags  $\leftarrow$  flags  $\cup$  {unrestricted_resource_use}
   else if  $s \geq 2000$ : flags  $\leftarrow$  flags  $\cup$  {unrestricted_resource_use}
4. if  $e \in \{\text{checkout, profile\_update}\}$  and  $r \geq 5$ :
   flags  $\leftarrow$  flags  $\cup$  {business_flow_abuse}
5. if  $b$ : flags  $\leftarrow$  flags  $\cup$  {broken_object_authorization}
6. if  $f$ : flags  $\leftarrow$  flags  $\cup$  {broken_function_authorization}
7. return flags

```

D. On-Demand SHAP Explanation

For categories 1-3, the salient behavioral aspects of the requests, i.e., payload size, response time, request rate, and unique IP count, are suitable input for a trained Isolation Forest model. Rather than automatically generating a SHAP explanation for each flagged request, it is only generated when an analyst chooses a specific request from the dashboard using the decision function of the model and SHAP explainer specific to that request's feature vector. Thus, the need for computation is reduced while keeping the possibility of using explanations as a diagnostic tool. BOLA and BFLA violations, however, cannot be explained using this kind of approach: BOLA requests cannot be differentiated in most instances from normal requests based on the equivalent criteria of size and time of the request. It is violated because of the person submitting the request, and not because of what the request is.

E. Analyst Dashboard

The output will be demonstrated through a dashboard on the web with three levels. The first level will consist of one card for every category, showing the number of flagged applications and the latest instance as well. The second level will give all flagged requests under a certain category when pressing the category. Such requests will include all necessary data: endpoint, method, source IP, token ID, date & time, and the reason of the rule execution. The third level will show the SHAP justification of the flagged request if it is requested.

F. Computational Complexity

The in-memory tracker (described in Section III-B) is built such that its resource cost per request does not increase with the traffic volume, unlike the previous solution that recalculated token behavior by going through the request log for each check. Each timestamp added to a token deque is removed at most once when it goes outside the 60-second period, which means that the total number of operations W done on N requests is limited by $W(N) \leq N$ (1), resulting in the amortized request cost of $W(N)/N = O(1)$ (2). The distinct IP address table is operated similarly and has the amortized cost of $O(m)$, where m is the number of IP addresses associated with that token in the 5-minute window; m was at most 5 in all tests in Section V. Together with a constant number of operations shown in Algorithm 1, this leads to $O(1)$ amortized cost under the assumption that m remains small, which is exactly what is demonstrated in Table II (1.5 μ s).

IV. IMPLEMENTATION

The implementation of the entire system is done through Python with the use of FastAPI for the backend layer of API, SQLAlchemy and SQLite as the persistent storage, and through the use of scikit-learn packages to include the Isolation Forest model for instant explanations. The feature of tracking the behavior in real-time is done through thread-safe rolling windows for every token, using deques for keeping time stamps and dictionaries for IP addresses recently used that helps avoid repeated database lookups on the path for detection. The access tokens are created by the means of use of JSON Web Encryption method with the support of the algorithm AES-256-GCM and are of short expiry.

V. RESULTS**A. Detection Outcomes on Live Traffic**

The evaluation was performed with the help of simulated traffic against a backend working in real-time: normal traffic was created to simulate direct interaction by human users and was followed by five intentionally generated cases for every targeted class. Separate credentials were used to prevent the sharing of statistical information across different cases. Thus, in total, there were 60 requests with their statuses obtained.

The summary of the results is given in Table I: all five classes were identified correctly, showing the reasons for the detection perfectly correlated with the rules applied. For those on the basis of BOLA, SHAP explanation was computed to find out whether the Isolation Forest algorithm would generate detection without being trained on the data previously. It turned out that the detection score is a normal one indicating that the request is normal considering payload dimension, timing, and frequency. Thus, the conclusion can be made that violations of the authorization nature cannot be detected through only traffic shapes. In contrast, the explanation of one of the operating cases, the oversized payload one, was negative showing that the payload dimension was the most possible reason confirming the efficiency of the explanation mechanism for the cases based on the categories

TABLE I
CATEGORY DETECTION RESULTS

Category	Count	Representative Reason
Broken authentication	3	Token used from 5 distinct IPs in 5 minutes
Unrestricted resource use	12	21 requests from one token in 60 seconds
Business flow abuse	12	8 calls to profile_update in 60 seconds
Broken object authorization	3	Requested a record not owned by the caller
Broken function authorization	1	Non-admin token called an admin-only endpoint

B. Classification Latency

In order to validate that classification is adequately quick for the detection route as asserted in Section III, the cost was

measured directly while preventing Algorithm 1 from using the in-memory tracker that comes before this algorithm in the live middleware. The results of the measurements conducted in 20,000 timed calls are displayed in Table II.

TABLE II
CLASSIFICATION LATENCY (μ S, N = 20,000)

Stage	Mean	Median	P95	P99
Rule evaluation only (Alg. 1)	0.21	0.17	0.32	0.38
Full hot path (tracker + Alg. 1)	1.50	1.36	2.15	3.41

Rule checks by themselves require approximately 0.21 microseconds, while the full hot path, inclusive of updating of the in-memory tracker (adding and eliminating of the deque of timestamps per token and the IP set) lasts about 1.50 microseconds. Both these values are negligible compared to the usual time required by network and database I/O for making requests, while no trained Isolation Forest model is needed for performing detection at request time, confirming that rule detection is not dependent on any machine-learning inference made at that time.

C. Precision, Recall, and F1

The numbers in Table I indicate that there were a total of five detected categories under traffic handling humans, but that these do not provide the number of false positives on their own. A separate analysis was conducted under ground truth conditions with 80 normal requests and five separate attack types, with one being conducted for each category using a different token for each. True positives, false positives, and false negatives were determined for each category as detailed below. The precision, recall, and F1 scores are calculated according to the following formulae:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

$$\text{F1} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

The results are given in Table III for each of the categories tested.

TABLE III
PRECISION, RECALL, AND F1 PER CATEGORY

Category	Prec.	Recall	F1
Broken authentication	1.00	1.00	1.00
Unrestricted resource use	1.00	1.00	1.00
Business flow abuse	0.16	1.00	0.28
Broken object authorization	1.00	1.00	1.00
Broken function authorization	1.00	1.00	1.00

Perfect recall and precision were obtained in four of the categories. Except Business flow abuse, which had a low precision (0.16), resulting from false positives only: as this evaluation involved the deliberate compression of time (with requests being sent back to back and without the intervals of several seconds that simulate human behavior in the demonstration described in Section V-A), a few ordinary tokens gathered enough requests over 60 seconds in the shared

counter and managed to cross the threshold for business flow abuse based upon a profile_update or checkout request. This situation could not happen in the real-life demonstration (Table I) because no false positives were recorded, as the real-life demonstration made it possible to keep the shared request-counting window low enough to avoid abuse threshold crossing. Thus, this finding is explicitly stated rather than put aside: it allows one to conclude that the threshold applied to business flow abuse is sensitive to elapsed time and supports the ideas of severity threshold adjustment and proper calibration outlined in Section VII.

D. Explaining the Business-Flow Precision Result

The 0.16 accuracy recorded in Section V-C can be calculated directly from the way in which the benchmark was created. There are ten normal tokens that make $K=8$ requests quickly, where there are 4 endpoint types and only 2 types contribute to the business-flow rule (checkout and profile_update), meaning the per-request likelihood of getting to the endpoint is $p=2/4=0.5$. Since all K requests happen in the same 60-second interval, the shared request counter reaches the limits $\theta=5$ at the 5th request; therefore, every other business-flow-endpoint call from that token, requests 5-8 will be flagged. The expected number of these calls for each token is basically p times the number of requests which can be served. Thus, when $T = 10$ tokens are taken into account the expected number of false positive becomes $E[\text{FP}] = T \cdot p \cdot \max(0, K - \theta + 1)$ (6). When one puts $T = 10$, $p = 0.5$, $K = 8$, $\theta = 5$ into this formula we receive $E[\text{FP}] = 10 \cdot 0.5 \cdot 4 = 20$ which is very close to our practical observation of 21 (Table III). Given the fact that $\text{TP} = 4$ is constant due to the scenario the precision we can expect is $\text{TP} / (\text{TP} + E[\text{FP}]) = 4 / 24 \approx 0.17$ (7). This final value is just one negative result different than the actual value of 0.16 (4/25). This means that it can be said for sure that lower precision is not the defect of implementation but the result of analytics.

E. Throughput Bound Under Load

Since the hot path specified in Table II ($c = 1.50 \mu$ s per request) occurs synchronously before the response is returned, continuous request rates approaching its inverse lead to a queue delay on top of c . Assuming the detection process is modeled as a single service station with deterministic service time c and arrival rate R , the volume of traffic can be represented as

$$\rho = R \cdot c \quad (8)$$

and according to Pollaczek-Khinchine formula, the mean waiting time for $M/D/1$ queue can be calculated as

$$W_q = \rho / (2(1 - \rho)) \cdot c, \text{ for } \rho < 1 \quad (9)$$

thus the expected total time of detection is

$$c + W_q = c \cdot (1 + \rho / (2(1 - \rho))) \quad (10)$$

In order for this to stay within 10% of c only ($\rho / (2(1 - \rho)) \leq 0.1$), we have to have $\rho \leq 0.167$, or $R \leq 0.167/c \approx 111,000$ requests per second for the investigated single core — many times greater than any expected API loads for the simulated backend, pointing to the fact that the detection process will unlikely be the bottleneck for the system.

VI. COMPARISON WITH EXISTING SYSTEMS

The differences are summarized in Table IV. The system does not claim superior performances over existing Isolation Forest-SHAP systems regarding anomaly detection. Instead, the goal of the current system is to focus on a set of different pre-defined classes of critical vulnerabilities with fast, deterministic checks and using machine learning to provide reasons for the occurrence and not to detect anomalies themselves.

TABLE IV
COMPARISON WITH EXISTING SYSTEM

Aspect	Existing System	Proposed System
Detection basis	General anomaly score	Five named OWASP categories
Detection timing	Periodic batch scoring	Real-time, inline per request
Explanation scope	Every flagged anomaly	On-demand, per analyst request
Authorization violations	Not addressed	Explicit rule-based checks
Detection-path ML inference	Required	Not required (rule-based)

VII. LIMITATIONS

Certain limitations have to be recognized. First, instead of using real-world production traffic, evaluation was done using simulated traffic; attackers may not behave the same way as envisioned based on the values used in the study. Next, detection is restricted only to the five chosen categories. An attack falling outside this range, for example, an injection attack, would not be considered, as there is no general-purpose anomaly score on the detection route. Third, the category thresholds, such as request-rate limits, are viewed as constants; these can lead to a decrease in the precision when assessing business flow abuse based on the results of the evaluation done under compressed time, as illustrated in Section V-C; while no false positives were found in real life.

ACKNOWLEDGMENT

Fig. 1 in this paper was generated using Claude (Anthropic), an AI tool. All other content, including the implementation, experiments, and results, is the authors' own work.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we introduced a novel API threat detection system that ensures real-time detection and is based on OWASP API Security Top 10 principles. The proposed system encompasses a combination of conventional detection methodologies based on API-specific behavioral and authorization practices with the use of SHAP-based explanations for cases when analysts want to investigate any specific incident. We conducted experiments to prove that our detection methodology worked correctly and instantly detected malicious activities in all five categories. The experiments confirmed that our system successfully detect

anomalies in traffic flow, however, in terms of detection of authorization issues like BOLA, we emphasize that traffic anomaly techniques alone cannot detect breaches thus leading us to hybrid approach that we suggested in this paper. Future work will be focused on calibration of adaptive thresholds, development of analyst feedback system for marking incidents as resolved, provision of real-time reports, involvement of token behavior in detection process, provision of token ciphering system as separate topic and testing of our system in real setups.

REFERENCES

- [1] A. Mittal, P. Keshap, and A. Hosabettu, "AI-driven real-time API security: Explainable threat detection for cloud environments," in *Proc. 2025 IEEE Int. Carnahan Conf. Security Technology (ICCST)*, Oct. 2025, pp. 1–7, doi: 10.1109/ICCST63435.2025.11293949.
- [2] OWASP Foundation, "OWASP API Security Top 10," 2023. [Online]. Available: <https://owasp.org/API-Security/>
- [3] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, Dec. 2008, pp. 413–422.
- [5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 1–39, 2012.
- [6] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, May 2000, pp. 93–104.
- [7] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [8] S. Hariri, M. C. Kind, and R. J. Brunner, "Extended isolation forest," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 4, pp. 1479–1489, 2021.
- [9] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," *arXiv preprint arXiv:1802.09089*, 2018.
- [10] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Aug. 2016, pp. 1135–1144.
- [11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [12] S. M. Lundberg *et al.*, "Explainable AI for trees: From local explanations to global understanding," *arXiv preprint arXiv:1905.04610*, 2019.
- [13] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooing LIME and SHAP: Adversarial attacks on post hoc explanation methods," in *Proc. AAAI/ACM Conf. AI, Ethics, and Society*, Feb. 2020, pp. 180–186.
- [14] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. 2010 IEEE Symp. Security and Privacy*, May 2010, pp. 305–316.
- [15] K. Veeramachaneni, I. Arnaldo, V. Korrapati, C. Bassias, and K. Li, "AP2: Training a big data machine to defend," in *Proc. 2016 IEEE 2nd Int. Conf. Big Data Security on Cloud (BigDataSecurity)*, Apr. 2016, pp. 49–54.

Paper1_IEEE_format humanized p1--.docx

by deadeye dick

Submission date: 01-Oct-2026 08:38PM (UTC+0400)

Submission ID: 3040740640

File name: Paper1_IEEE_format_humanized_p1--.docx (200.73K)

Word count: 4101

Character count: 22394

Lightweight API Security Monitoring and Anomaly Detection: Real-Time Detection of Five OWASP-Based Vulnerability Categories

Mrs. Shyni

¹⁵ Assistant Professor (SS)
Rajalakshmi Engineering College
Chennai, India
shyni.m@rajalakshmi.edu.in

Varshabala

² Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
prathikshathila@gmail.com

Tejushree Sanjeevi Kumar

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
tejushreebv@gmail.com

Valluru Varshini

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
valluruvarshini9@gmail.com

Abstract—API anomaly detection utilizing machine learning with Isolation Forest and SHAP has been shown to work successfully in an enterprise context. However, conventional methods usually score the traffic after the fact and explain each of the detected anomalies, which is not scalable when the number of alerts grows. This paper describes a lightweight ¹¹ solution that focuses on just five well defined attack types from OWASP API Security Top 10: broken authentication, ³ restricted resource consumption, business flow abuse, broken object-level authorization (BOLA) and broken function-level authorization (BFLA). Each request is analyzed against these five types of attacks in real-time by using simple deterministic rules (Algorithm 1) instead of extracting machine learning inference on the detection path, and rule checking itself takes just 0.21 μ s on average, and total hot path time is 1.50 μ s. In the case study conducted with labelled data (80 normal requests, 60 combined-demo requests, and independent attacks in each class), we observed that four out of five classes obtained perfect marks in precision, recall, and F1 (1.00 in each case). The fifth class, business flow abuse, achieved a high recall score of 1.00 but low precision of 0.16 (F1 = 0.28). This low precision value was only encountered when past data was examined. Another observation was that we discovered authorization violations, for instance, BOLA, which were marked as statistically normal under Isolation Forest. This indicated that violations won't be exposed through traffic pattern anomaly detection.

I. INTRODUCTION

Modern cloud-based services are built around Application Programming Interfaces (APIs) which have become the target of cybercriminals because of their exposure on the public Internet. Earlier research has found out that some methods for unknown anomaly detection like Isolation Forest have been successfully used along with SHAP to detect strange API traffic in an enterprise environment [1]. Nevertheless, such systems usually work like offline solutions: the traffic is monitored, analyzed, and explained afterward and the detected anomalies are accounted for completely. This paper adopts a novel technique that is influenced by two practical concerns. Instead of depending upon a general anomaly score to determine what ⁵ qualifies as “unusual,” detection is based on five specific attack categories from the OWASP API Security Top 10 white paper

[2] One very widely used industry reference to the most popular ways that APIs get exploited is presented. Second, rather than scoring traffic in batches, each incoming request is identified using fast threshold tests, thus eliminating the requirement for active learning inference as part of the detection process. One machine learning model, Isolation Forest, is retained, but instead of being incorporated into the detection process, the model is used to power an on-demand explanation solution that is only invoked when an analyst decodes the flagged request. As a result, the ability to explain outcomes using SHAP is still preserved while excluding the ⁴ computation of the explanations for every flagged event. The remainder of this paper is structured as follows: Section II provides a survey of works related to the topic; Section III describes the ¹ architecture of the system and outlines five target categories; Section IV discusses implementation issues; Section V presents an experimental study; Section VI compares the proposed method with other existing solutions; Section VII discusses limitations; Section VIII provides directions for future research.

II. RELATED WORK

Anomaly detection comprises various techniques used for monitoring the behavior of applications, including statistical methods, distance-based techniques, density-based methods, and data learning, all relying on different assumptions about the normal state of operation and the availability of labels [3]. The isolation forest technique is based on separating the comprehended subjects through a number of random partitions without assessing the distribution of normal data [4], [5]. For the purpose of comparison, ⁹ is also essential to mention such unsupervised techniques as local outlier factor (LOF) [6] and single-class support vector machine [7]. Advanced versions of the isolation forest algorithm make use of the hyperplanes that are randomly oriented to prevent potential issues with the axes alignment [8]. In the current research, the classic isolation forest method is employed; it should be noted that it is employed only

for the purpose of needing the explanation, rather than the task of detection. There is a similar application developed by Mittal et al. that uses the isolation forest method coupled with the shap library to detect threats in APIs and reports 0.94 precision and 0.91 recall achieved with the help of simulation of fintech companies traffic streams [1]. The presented model, like other models that use the anomaly detection technique. The OWASP API Security Top 10 [2] documents the most common API vulnerability classes observed in practice, including Broken Object Level Authorization, Broken Authentication, Broken Function Level Authorization, and Unrestricted Resource Consumption. Although widely referenced in industry security guidance, this taxonomy is less commonly used as the direct basis for a real-time, rule-based detection architecture in the anomaly-detection literature, which more often relies on general-purpose statistical scoring.

For local interpretability, LIME approximates a model's behavior around an individual prediction using an interpretable surrogate [10], whereas SHAP provides an additive feature-attribution framework [11]. TreeExplainer introduced efficient, tree-specific attribution algorithms [12]. Explanations require caution, as Slack et al. demonstrate that adversarially constructed classifiers can mislead perturbation-based LIME and SHAP explanations [13].

Operational intrusion detection also requires attention to analyst workload. Sommer and Paxson discuss why successful machine-learning experiments do not automatically translate into effective deployed intrusion detection [14]. The analyst-in-the-loop AI2 system combines outlier detection with analyst feedback and supervised learning [15], providing context for the dashboard and feedback loop discussed as future work in Section VIII.

III. PROPOSED METHODOLOGY

A. System Overview

The system proposed focuses on five categories chosen from the OWASP API Security Top 10 based on their applicability to the given type of endpoints in the simulated backend. The selected categories are: broken authentication, unregulated resource consumption, business flow misuse, broken object-level authorization (BOLA), and broken function-level authorization (BFLA). The incoming request is evaluated against all the categories in an inline fashion before the actual response is produced. Additionally, another Isolation Forest model is used only to provide an on-demand SHAP explanation

for a particular flagged request when necessary.



Fig. 1. Real-time category detection pipeline (center/right) with on-demand SHAP explanation (left).

B. Fast, In-Memory Behavioral Tracking

Two out of five categories heavily rely on recent behavior history such as the number of requests made by the token or the number of different IP addresses it has been associated with. To eliminate the need for constant recomputation of these figures by querying through request records considering increasing traffic rates, there are small rolling windows of short-term memory maintained per token that store a deque of timestamps and a set of IP addresses recently used. Thus, this allows making each verification within a constant time frame.

C. Category Detection Rules

The Twentieth Category: Detection Rules

- Broken authentication: This rule gets triggered when the same authentication token comes from three or more different IP addresses within a period of five minutes, which is an indication of token sharing, replays or even token theft.

- Unrestricted resource consumption: This rule gets activated when a token goes beyond a certain threshold of requests within a minute or when it goes over the allowed request size.

- Business flow abuse: This rule comes into play when certain action endpoint (checkout and profile_update) is invoked excessively when compared to other endpoints.

- Broken object-level authorization: This rule gets triggered when a request targets a resource identifier which does not belong to authenticated principal, which can be verified by means of the ownership check.

- Broken function-level authorization: This rule gets activated when a token without admin rights tries to call an admin endpoint.

In summary, the three former rules are behaviorbased, while two latter rely on the fact that requests permissions are compared against their real rights.

Algorithm 1: Per-request category classification

Input: endpoint e , payload size s , requests_last_60s r , distinct_ips_last_5min d , bola_flag b , bfla_flag f .
Output: set of matched categories (0, 1, or many).
1. flags $\leftarrow \{\}$
2. if $d \geq 3$: flags $\leftarrow$ flags $\cup$ {broken_authentication}
3. if $r \geq 10$: flags $\leftarrow$ flags $\cup$ {unrestricted_resource_use}
else if $s \geq 2000$: flags $\leftarrow$ flags $\cup$ {unrestricted_resource_use}
4. if $e \in \{\text{checkout, profile_update}\}$ and $r \geq 5$:
flags $\leftarrow$ flags $\cup$ {business_flow_abuse}
5. if b : flags $\leftarrow$ flags $\cup$ {broken_object_authorization}
6. if f : flags $\leftarrow$ flags $\cup$ {broken_function_authorization}
7. return flags

D. On-Demand SHAP Explanation

For categories 1-3, the salient behavioral aspects of the requests, i.e., payload size, response time, request rate, and unique IP count, are suitable input for a trained Isolation Forest model. Rather than automatically generating a SHAP explanation for each flagged request, it is only generated when an analyst chooses a specific request from the dashboard using the decision function of the model and SHAP explainer specific to that request's feature vector. Thus, the need for computation is reduced while keeping the possibility of using explanations as a diagnostic tool. BOLA and BFLA violations, however, cannot be explained using this kind of approach: BOLA requests cannot be differentiated in most instances from normal requests based on the equivalent criteria of size and time of the request. It is violated because of the person submitting the request, and not because of what the request is.

E. Analyst Dashboard

The output will be demonstrated through a dashboard on the web with three levels. The first level will consist of one card for every category, showing the number of flagged applications and the latest instance as well. The second level will give all flagged requests under a certain category when pressing the category. Such requests will include all necessary data: endpoint, method, source IP, token ID, date & time, and the reason of the rule execution. The third level will show the SHAP justification of the flagged request if it is requested.

F. Computational Complexity

The in-memory tracker (described in Section III-B) is built such that its resource cost per request does not increase with the traffic volume, unlike the previous solution that recalculated token behavior by going through the request log for each check. Each timestamp added to a token deque is removed at most once when it goes outside the 60-second period, which means that the total number of operations W done on N requests is limited by $W(N) \leq N$ (1), resulting in the amortized request cost of $W(N)/N = O(1)$ (2). The distinct IP address table is operated similarly and has the amortized cost of $O(m)$, where m is the number of IP addresses associated with that token in the 5-minute window; m was at most 5 in all tests in Section V. Together with a constant number of operations shown in Algorithm 1, this leads to $O(1)$ amortized cost under the assumption that m remains small, which is exactly what is demonstrated in Table II (1.5 μ s).

The implementation of the entire system is done through Python with the use of FastAPI for the backend layer of API, SQLAlchemy and SQLite as the persistent storage, and through the use of scikit-learn packages to include the Isolation Forest model for instant explanations. The feature of tracking the behavior in real-time is done through thread-safe rolling windows for every token, using deques for keeping time stamps and dictionaries for IP addresses recently used that helps avoid repeated database lookups on the path for detection. The access tokens are created by the means of use of JSON Web Encryption method with the support of the algorithm AES-256-GCM and are of short expiry.

V. RESULTS**A. Detection Outcomes on Live Traffic**

The evaluation was performed with the help of simulated traffic against a backend working in real-time: normal traffic was created to simulate direct interaction by human users and was followed by five intentionally generated cases for every targeted class. Separate credentials were used to prevent the sharing of statistical information across different cases. Thus, in total, there were 60 requests with their statuses obtained.

The summary of the results is given in Table I: all five classes were identified correctly, showing the reasons for the detection perfectly correlated with the rules applied. For those on the basis of BOLA, SHAP explanation was computed to find out whether the Isolation Forest algorithm would generate detection without being trained on the data previously. It turned out that the detection score is a normal one indicating that the request is normal considering payload dimension, timing, and frequency. Thus, the conclusion can be made that violations of the authorization nature cannot be detected through only traffic shapes. In contrast, the explanation of one of the operating cases, the oversized payload one, was negative showing that the payload dimension was the most possible reason confirming the efficiency of the explanation mechanism for the cases based on the categories

TABLE I
CATEGORY DETECTION RESULTS

Category	Count	Representative Reason
Broken authentication	3	Token used from 5 distinct IPs in 5 minutes
Unrestricted resource use	12	21 requests from one token in 60 seconds
Business flow abuse	12	8 calls to profile_update in 60 seconds
Broken object authorization	3	Requested a record not owned by the caller
Broken function authorization	1	Non-admin token called an admin-only endpoint

B. Classification Latency

In order to validate that classification is adequately quick for the detection route as asserted in Section III, the cost was

measured directly while preventing Algorithm 1 from using the in-memory tracker that comes before this algorithm in the live middleware. The results of the measurements conducted in 20,000 timed calls are displayed in Table II.

TABLE II
CLASSIFICATION LATENCY (μ S, $N = 20,000$)

Stage	Mean	Median	P95	P99
Rule evaluation only (Alg. 1)	0.21	0.17	0.32	0.38
Full hot path (tracker + Alg. 1)	1.50	1.36	2.15	3.41

Rule checks by themselves require approximately 0.21 microseconds, while the full hot path, inclusive of updating of the in-memory tracker (adding and eliminating of the deque of timestamps per token and the IP set) lasts about 1.50 microseconds. Both these values are negligible compared to the usual time required by network and database I/O for making requests, while no trained Isolation Forest model is needed for performing detection at request time, confirming that rule detection is not dependent on any machine-learning inference made at that time.

C. Precision, Recall, and F1

The numbers in Table I indicate that there were a total of five detected categories under traffic handling humans, but that these do not provide the number of false positives on their own. A separate analysis was conducted under ground truth conditions with 80 normal requests and five separate attack types, with one being constructed for each category using a different token for each. True positives, false positives, and false negatives were determined for each category as detailed below. The precision, recall, and F1 scores are calculated according to the following formulae:

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN)$$

$$F1 = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$$

The results are given in Table III for each of the categories tested.

TABLE III
PRECISION, RECALL, AND F1 PER CATEGORY

Category	Prec.	Recall	F1
Broken authentication	1.00	1.00	1.00
Unrestricted resource use	1.00	1.00	1.00
Business flow abuse	0.16	1.00	0.28
Broken object authorization	1.00	1.00	1.00
Broken function authorization	1.00	1.00	1.00

Perfect recall and precision were obtained in four of the categories. Except Business flow abuse, which had a low precision (0.16), resulting from false positives only: as this evaluation involved the deliberate compression of time (with requests being sent back to back and without the intervals of several seconds that simulate human behavior in the demonstration described in Section V-A), a few ordinary tokens gathered enough requests over 60 seconds in the shared

counter and managed to cross the threshold for business flow abuse based upon a profile_update or checkout request. This situation could not happen in the real-life demonstration (Table I) because no false positives were recorded, as the real-life demonstration made it possible to keep the shared request-counting window low enough to avoid abuse threshold crossing. Thus, this finding is explicitly stated rather than put aside: it allows one to conclude that the threshold applied to business flow abuse is sensitive to elapsed time and supports the ideas of severity threshold adjustment and proper calibration outlined in Section VII.

D. Explaining the Business-Flow Precision Result

The 0.16 accuracy recorded in Section V-C can be calculated directly from the way in which the benchmark was created. There are ten normal tokens that make $K=8$ requests quickly, where there are 4 endpoint types and only 2 types contribute to the business-flow rule (checkout and profile_update), meaning the per-request likelihood of getting to the endpoint is $p=2/4=0.5$. Since all K requests happen in the same 60-second interval, the shared request counter reaches the limits $\theta=5$ at the 5th request; therefore, every other business-flow-endpoint call from that token, requests 5-8 will be flagged. The expected number of these calls for each token is basically p times the number of requests which can be served. Thus, when $T = 10$ tokens are taken into account the expected number of false positive becomes $E[FP] = T \cdot p \cdot \max(0, K - \theta + 1)$ (6). When one puts $T = 10$, $p = 0.5$, $K = 8$, $\theta = 5$ into this formula we receive $E[FP] = 10 \cdot 0.5 \cdot 4 = 20$ which is very close to our practical observation of 21 (Table III). Given the fact that $TP = 4$ is constant due to the scenario the precision we can expect is $TP / (TP + E[FP]) = 4 / 24 \approx 0.17$ (7). This final value is just one negative result different than the actual value of 0.16 (4/25). This means that it can be said for sure that lower precision is not the defect of implementation but the result of analytics.

E. Throughput Bound Under Load

Since the hot path specified in Table II ($c = 1.50 \mu$ s per request) occurs synchronously before the response is returned, continuous request rates approaching its inverse lead to a queue delay on top of c . Assuming the detection process is modeled as a single service station with deterministic service time c and arrival rate R , the volume of traffic can be represented as

$$\rho = R \cdot c \quad (8)$$

and according to Pollaczek-Khinchine formula, the mean waiting time for M/D/1 queue can be calculated as

$$Wq = \rho / (2(1 - \rho)) \cdot c, \text{ for } \rho < 1 \quad (9)$$

thus the expected total time of detection is

$$c + Wq = c \cdot (1 + \rho / (2(1 - \rho))) \quad (10)$$

In order for this to stay within 10% of c only ($\rho / (2(1 - \rho)) \leq 0.1$), we have to have $\rho \leq 0.167$, or $R \leq 0.167/c \approx 111,000$ requests per second for the investigated single core — many times greater than any expected API loads for the simulated backend, pointing to the fact that the detection process will unlikely be the bottleneck for the system.

VI. COMPARISON WITH EXISTING SYSTEMS

The differences are summarized in Table IV. The system does not claim superior performance over existing Isolation Forest-SHAP systems regarding anomaly detection. Instead, the goal of the current system is to focus on a set of different pre-defined classes of critical vulnerabilities with fast, deterministic checks and using machine learning to provide reasons for the occurrence and not to detect anomalies themselves.

TABLE IV
COMPARISON WITH EXISTING SYSTEM

Aspect	Existing System	Proposed System
Detection basis	General anomaly score	Five named OWASP categories
Detection timing	Periodic batch scoring	Real-time, inline per request
Explanation scope	Every flagged anomaly	On-demand, per analyst request
Authorization violations	Not addressed	Explicit rule-based checks
Detection-path ML inference	Required	Not required (rule-based)

VII. LIMITATIONS

Certain limitations have to be recognized. First, instead of using real-world production traffic, evaluation was done using simulated traffic; attackers may not behave the same way as envisioned based on the values used in the study. Next, detection is restricted only to the five chosen categories. An attack falling outside this range, for example, an injection attack, would not be considered, as there is no general-purpose anomaly score on the detection route. Third, the category thresholds, such as request-rate limits, are viewed as constants; these can lead to a decrease in the precision when assessing business flow abuse based on the results of the evaluation done under compressed time, as illustrated in Section V-C; while no false positives were found in real life.

ACKNOWLEDGMENT

Fig. 1 in this paper was generated using Claude (Anthropic), an AI tool. All other content, including the implementation, experiments, and results, is the authors' own work.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we introduced a novel API threat detection system that ensures real-time detection and is based on OWASP API Security Top 10 principles. The proposed system encompasses a combination of conventional detection methodologies based on API-specific behavioral and authorization practices with the use of SHAP-based explanations for cases when analysts want to investigate any specific incident. We conducted experiments to prove that our detection methodology worked correctly and instantly detected malicious activities in all five categories. The experiments confirmed that our system successfully detect

anomalies in traffic flow, however, in terms of detection of authorization issues like BOLA, we emphasize that traffic anomaly techniques alone cannot detect breaches thus leading us to hybrid approach that we suggested in this paper. Future work will be focused on calibration of adaptive thresholds, development of analyst feedback system for marking incidents as resolved, provision of real-time reports, involvement of token behavior in detection process, provision of token ciphering system as separate topic and testing of our system in real setups.

REFERENCES

- [1] A. Mittal, P. Keshap, and A. Hosabettu, "AI-driven real-time API security: Explainable threat detection for cloud environments," in *Proc. 2025 IEEE Int. Carnahan Conf. Security Technology (ICCST)*, Oct. 2025, pp. 1–7, doi: 10.1109/ICCST63435.2025.11293949.
- [2] OWASP Foundation, "OWASP API Security Top 10," 2023. [Online]. Available: <https://owasp.org/API-Security/>
- [3] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009.
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, Dec. 2008, pp. 413–422.
- [5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 1–39, 2012.
- [6] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, May 2000, pp. 93–104.
- [7] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [8] S. Hariri, M. C. Kind, and R. J. Brunner, "Extended isolation forest," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 4, pp. 1479–1489, 2021.
- [9] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," *arXiv preprint arXiv:1802.09089*, 2018.
- [10] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, Aug. 2016, pp. 1135–1144.
- [11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [12] S. M. Lundberg *et al.*, "Explainable AI for trees: From local explanations to global understanding," *arXiv preprint arXiv:1905.04610*, 2019.
- [13] D. Slack, S. Hilgard, E. Jia, S. Singh, and H. Lakkaraju, "Fooling LIME and SHAP: Adversarial attacks on post hoc explanation methods," in *Proc. AAAI/ACM Conf. AI, Ethics, and Society*, Feb. 2020, pp. 180–186.
- [14] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. 2010 IEEE Symp. Security and Privacy*, May 2010, pp. 305–316.
- [15] K. Veeramachaneni, I. Arnaldo, V. Korrapati, C. Bassias, and K. Li, "AI?: Training a big data machine to defend," in *Proc. 2016 IEEE 2nd Int. Conf. Big Data Security on Cloud (BigDataSecurity)*, Apr. 2016, pp. 49–54.

ORIGINALITY REPORT

6%

SIMILARITY INDEX

5%

INTERNET SOURCES

5%

PUBLICATIONS

2%

STUDENT PAPERS

PRIMARY SOURCES

1	arxiv.org Internet Source	1%
2	cloud.riverpublishers.com Internet Source	1%
3	appsec.fyi Internet Source	1%
4	Alessandra De Paola, Salvatore Gaglio, Giuseppe Lo Re, Marco Morana. "A hybrid system for malware detection on big data", IEEE INFOCOM 2018 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), 2018 Publication	1%
5	appsecsanta.com Internet Source	<1%
6	Sitanath Biswas, Sayan Chakraborty. "Frontiers in Engineering, Science and Technology", CRC Press, 2026 Publication	<1%
7	Submitted to Richfield Graduate Institute of Technology Student Paper	<1%
8	kse2025.kse-conferences.org Internet Source	<1%
9	tel.archives-ouvertes.fr Internet Source	<1%
10	www.ijrte.org Internet Source	<1%
11	Ferreira, Tomás Ramos. "Análise de vulnerabilidades em aplicações Web e APIs baseadas em OWASP.", Instituto Politecnico de Beja (Portugal) Publication	<1%
12	Sukhdeep Kaur, Ashima Kalra, Bhawna Tandon. "Smart Systems and Beyond - Redefining Computing and Robotics through AI", CRC Press, 2026 Publication	<1%
13	dspace.mit.edu Internet Source	<1%

14	porto.polito.it Internet Source	<1 %
15	www.riverpublishers.com Internet Source	<1 %
16	Ram Lal Yadava, Anil Kumar Sethi, Amanpreet Singh Saini. "Computing, Communication, and Sustainable Energy Technologies - Communication Techniques and Intelligent Systems (Volume 1)", CRC Press, 2026 Publication	<1 %

Exclude quotes	Off	Exclude matches	Off
Exclude bibliography	On		

Hardening API Authentication with Encrypted, Short-Lived Access Tokens Using JSON Web Encryption

Mrs.M.Shyni
Assistant Professor (SS)
Rajalakshmi Engineering College
Chennai, India
shyni.m@rajalakshmi.edu.in

Varsha B
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
prathikshathila@gmail.com

Tejushree Sanjeevi Kumar
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
tejushreebv@gmail.com

Valluru Varshini
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
valluruvarshini9@gmail.com

Abstract—Token authentication is used in API which helps to identify the user who logs into the system. The usage of conventional JWT means that it is not encrypted, this means that anyone who intercepts the token will be able to decode the information contained in it such as user id, token id, expiry time, privileges etc. The purpose of this research is to talk about improving the token generation mechanism in FastAPI web application by means of usage of JWE technique that uses AES 256 GCM encryption and establishes fixed time period of 20 minutes. According to the analytical model, the precise token length is given (predicted length is 241 characters against measured length). Furthermore, the maximum of forgery probability is $q \cdot 2^{-128}$ and the average duration for which the token can be intercepted is $T/2 = 10$ minutes. By conducting experiments, the attacker managed to extract data from 4 out of 4 letters in the signed JWT but did not manage to decode letters in the form of JWE. Among 2,000 tokens, related to the cases of token usage with one flipped bit [2,000 tokens with a wrong decryption key and 2,000 tokens that became invalid], only valid tokens were used. Moreover, 20,000 encryptions of the same texts resulted in obtaining 20,000 various IVs. The obtained data presented above can be compared with the previous results related to the signed JWT, which indicated a difference between the length of the text presented as a signed JWT and the text, issued in the form of the JWE.

Index Terms—API security, JSON Web Encryption, JWE, token authentication, credential confidentiality, AES-256-GCM, traffic analysis

I. INTRODUCTION

Almost all the APIs in existence nowadays provide a token after successful authentication allowing identification of the user in subsequent requests. The most common example is the JSON Web Token (JWT) which consists of three parts - header, payload, and signature. The signature guarantees that the payload has not been modified while payload remains only base64url encoded. Although implicating the token via interception or through misconfiguration or hacking won't generate another token, as anyone suffering from a mishap will be able to access the token. The claims in the token such as the user ID, token ID, validity time, and privilege flag could therefore be read by anyone who intercepts the token..

This research is part of a more extensive research project that investigates lightweight API security and includes an implementation of a real-time detection system that focuses on five categories from the OWASP API Security Top 10: broken authentication, unrestricted resource consumption, business flow abuse, broken object level authorization, and broken function level authorization (2). The detection system takes place on a simulated FastAPI backend, recognizes each incoming request in real-time through fast checks, while leaving analysis based on machine learning for after-the-fact analysis of suspicious requests. This paper is a continuation of this project, which is aimed at hardening the credential layer that the traffic relies on: access tokens generated following a successful authentication. The detection system in (2) recognizes suspicious traffic flows but does not take into account the security of tokens used in this traffic.

This issue of integrity separation, which is provided by the received JWT, is addressed by the established IETF specification called JSON Web Encryption (JWE) [1]. The main contributions of this paper are as follows:

- Development of a threat model and a development of the analytical model of the mechanism of JWE token giving the closed formulas for token size, forgery probability, key search effort, probability of IV collision and interception window for the token which has been intercepted.
- Creation of a reference implementation of the short-term encrypted token in a FastAPI backend which can be easily used in place of the signed JWT issuing without any changes needed from the client side.
- Development of a reproducible benchmark which measures in size, latency, confidentiality, tampering refusal, wrong key refusal, expiration honoring and IV unique, which can be compared with the analytical model.

- Discovery of the fact that ciphertext size allows revealing the privilege flag in correlation with the constant padding length and its overhead.

The rest of the paper is structured as follows. Related work is discussed in Section II. The threat model is presented in Section III. The mechanism is described in Section IV. The analytical model is developed in Section V. The experimental setup is described in Section VI. The results are presented in Section VII. Discussion and comparison to conventional JWT issuance is provided in Section VIII. Limitations are discussed in Section IX, and conclusions are given in Section X.

II. RELATED WORK

JSON Web Token (JWT) [3] describes a compact and URL-safe encoding for claims carried between two entities, usually digitally signed with either HMAC or RSA to ensure integrity. A digitally signed-only JWT (JWS) [7] lacks confidentiality: its payload is only base64url encoded, easily decodable without the signing key. Current best practices for using JSON Web Tokens [19] and for bearer tokens [21] in general focus on selecting an algorithm and verifying integrity rather than protecting the content of the token.

JSON Web Encryption (JWE) [1] guarantees confidentiality through a compact format who contains five different parts: protected header, encrypted key, IV, ciphertext, and authentication tag. The algorithms use by JWE are defined in JSON Web Algorithms (JWA) [8], and the keys are provided in JSON Web Keys (JWK) [9]. The scheme used in this case is AES-256-GCM, which is an authenticated encryption scheme that guarantees both confidentiality and integrity of the data by means of a single cryptographic primitive [4], [18]; AES is standardized in FIPS PUB 197 [16] and its key management is described in NIST SP 800-57 [17].

One of the categories API vulnerability is Broken Authentication, and the examples include credential stuffing, lack of proper token generation, and the case when the contents of the token are exposed to malicious third parties. Most of the important recommendation deals with detection of stolen tokens abuse, while there is not much information available about what the tokens are capable of revealing to the attacker until the abuse has already taken place.

A wider array of standards associated with OAuth 2.0 provides additional context. The framework covering OAuth 2.0 [6], its threat model [10], and the security protocol analysis [11] describe the types of vulnerabilities in authorization flows apart from keeping the confidentiality of the token. PKCE [12] and Token Introspection [13] address the problems that are related but different than those mentioned above while OpenID Connect [14] provides identity claims. Protection against token interception at the transport layer is achieved by using TLS 1.3 [15] and this protection complements the protection of the payload since the token is store in a decrypted form in proxy servers, logs, as well as browser storage. Short-lived credentials are the well-known design principle that limits the lifetime of stolen credentials [6]. The passwords that are hashed using bcrypt [20] are purposely expensive, which cost is referred to in Section VII when determining token overhead. Unlike the usual token recommendation, this work compares the present solution against traffic-level detector [2] at the same API testbed.

III. THREAT MODEL AND SECURITY GOALS

The asset consists of the four claims which the token carries: sub, jti, exp, and is_admin. Let F be the set containing these assets and $|F| = 4$. The server stores only one 256-bit symmetric key which never leaves the server. There are three security objectives formulated: G1, ensuring the secrecy of F from an adversary which does not have access to the key; G2, providing the integrity of the token so that any modification of the token is detected; and G3, a limited useful lifetime for a compromised token. The adversaries considered are summarized in Table I.

TABLE I
THREAT MODEL

Adversary	Capability	Goal
A1 Passive interceptor	Observes tokens in transit, in logs, or at rest; no key	G1
A2 Active tamperer	Modifies any part of a token and replays it	G2
A3 Key guesser	Tries to decrypt or forge tokens by guessing the key	G1, G2
A4 Late replayer	Replays a captured, unmodified token after a delay	G3

A token replayed unmodified within its lifetime is accepted, because a bearer token is not bound to a client. The mechanism bounds this risk through the short expiry (G3) but does not eliminate it.

IV. PROPOSED MECHANISM

A. Token Issuance

After successful authentication, the server will create the claim object with 4 attributes. The given claim object is then converted into JSON format and encrypted using the JWE using the dir key wrapping mechanism (in this mechanism, the symmetric key is used directly without any wrapping of keys) and the encryption algorithm used is A256GCM. The issuance is given in Algorithm 1.

Algorithm 1: Token issuance

Input: user id u , privilege flag a , server key K , lifetime T .

1. $jti \leftarrow$ new random UUID.
2. $exp \leftarrow$ current time + T .
3. $claims \leftarrow \{sub: u, jti: jti, exp: exp, is_admin: a\}$.
4. $IV \leftarrow$ fresh random value.
5. $(C, tag) \leftarrow \text{AES-256-GCM-Encrypt}(K, IV, \text{JSON}(claims))$.
6. Return $header.empty().IV.C.tag, \text{base64url-encoded}$.

B. Token Verification

The token is decrypted at the server using the same key for every request. In the case of unsuccessful decryption or signature verification, the request would be denied. On the other hand, if decryption was done successfully but the token has expired, then the request still would be denied. Algorithm two provides a summary for verification.

Algorithm 2: Token verification

Input: token, server key K .

1. Split token into header, key, IV , C , tag .
2. Decrypt C under K , IV ; verify tag; else reject.
3. Parse JSON claims; on failure, reject.
4. If exp is not later than now, reject.
5. Return claims.

C. Short Expiry

Tokens have a lifespan of $T=20$ minutes. This restricts the period in which any intercepted tokens will remain valid, regardless of when the interception is noticed. The precise period is examined in Section V-D.

D. Fixed-Length Claim Padding (Extension)

The claim has its contents encoded, but its size is transparent. This is because the size of JSON text representing a privileged token is different from the size of the JSON text that represents a common token. To alleviate this issue, the serialized claims are padded with an additional field so that the plaintext is always of size P . The complete process is shown in Fig. 1.

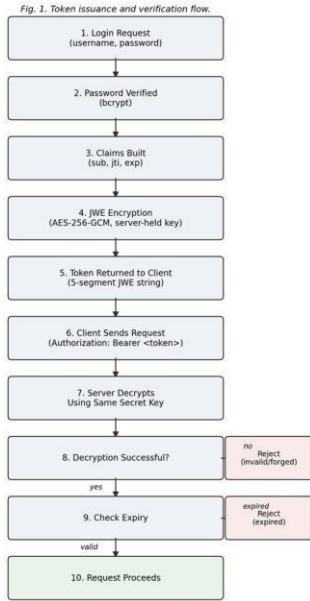


Fig. 1. Token issuance and verification flow.

V. ANALYTICAL MODEL

A. Token Size

The unpadded base64url representation of n bytes yields the character length given by

$$L(n) = \lceil 4n/3 \rceil \quad (1)$$

characters. A compact JSON Web Encryption (JWE) token consists of a header of h bytes, an encrypted key of k bytes (with k equal to 0 when the encryption is performed by means of a symmetric key), an initialization vector (IV) of v bytes, an encrypted message of m bytes (where $m = p$ since Galois/Counter Mode (GCM) retains the size of plaintext), and a tag of τ bytes, as represented in the following equation (note that the token parts are divided by four dots):

$$S_{JWE} = L(h) + L(k) + L(v) + L(m) + L(\tau) + 4 \quad (2)$$

In turn, a signed JSON Web Token (JWT) using the HMAC SHA256 algorithm (HS256) includes a header of h' bytes, a payload of P bytes, a 32-byte signature, and two dots

$$S_{JWS} = L(h') + L(p) + L(32) + 2 \quad (3)$$

Therefore, the relative size of overhead of encryption may be defined as

$$\delta S = (S_{JWE} - S_{JWS}) / S_{JWS} \quad (4)$$

B. Forgery and Key Search

For a t -bit authentication tag, the probability a single forged token is accepted is on the order of 2^{-t} [4], so an adversary making q forgery attempts succeeds with probability at most

$$P_{\text{forge}}(q) \leq q \cdot 2^{-t} \quad (5)$$

For $t = 128$ and $q = 240$ attempts, P_{forge} is at most 2^{-88} , about 3.2×10^{-27} . Exhaustive search of a k -bit key needs on average 2^{k-1} trials, so at R trials per second

$$T_{\text{search}} = 2^{k-1} / R \quad (6)$$

For $k = 256$ and $R = 1012$ trials/s, T_{search} is about 5.8×10^{64} seconds, or about 1.8×10^{57} years.

C. IV Collision

Each token uses a fresh random IV of b bits. By the birthday bound, the probability that any two of n tokens share an IV is approximately

$$p_{IV}(n) \approx n(n-1) / 2^{b+1} \quad (7)$$

and the number of tokens issuable under one key before this probability reaches a target p is

$$n^*(p) = \sqrt{2^{b+1} \cdot p} \quad (8)$$

With the measured $b = 128$, $p_{IV}(20,000)$ is about 5.9×10^{-31} , and $n^*(2^{-32})$ is about 4.0×10^{14} tokens, so key lifetime is not limited by IV collisions at this system's scale.

D. Exposure Window

Suppose a token of lifetime T is intercepted at a time u distributed uniformly over its lifetime. The remaining useful time is $W = T - u$, so

$$E[W] = T/2 \quad (9a)$$

$$P(\text{valid after delay } d) = \max(0, 1 - d/T) \quad (9b)$$

For $T = 20$ minutes, $E[W] = 10$ minutes and the worst case is 20 minutes. Relative to a reference lifetime T_{ref} , the exposure window is reduced by the factor T_{ref}/T ; for a 24-hour token this factor is 72.

E. Confidentiality Metrics

Let $1[f]$ equal 1 if field f in F can be recovered without the key. The exposure ratio is

$$E_x = (1/|F|) \sum_{f \in F} 1[f] \quad (10)$$

For a length-based classifier predicting privilege class C from token length Λ , with $N(l, c)$ tokens of length l and class c among N tokens, accuracy and leaked information are

$$A = (1/N) \sum_l \max_c N(l, c) \quad (11a)$$

$$I(\Lambda; C) = H(C) - H(C|\Lambda) \quad (11b)$$

For balanced classes, $H(C) = 1$ bit and chance accuracy is 0.5. Padding every plaintext to a fixed length P gives a padding field of length

$$\pi = P - m_0 \geq 0 \quad (12)$$

where m_0 is the unpadded plaintext length, making Λ constant, so $I(\Lambda; C) = 0$.

VI. EXPERIMENTAL SETUP

Every experiment utilizes the same token module call, the same key, and a backend lifetime of 20 minutes, and a single script is executed to recreate the numbers with the same random seed of 42. The baseline uses the HS256 JWT signature with four identical claims. All measurements are based on $n = 10,000$ measurements being made while 300 warmed-up executions are used; results include mean, median, standard deviation, and 95th and 99th percentiles. The experiments were conducted in a single-core environment using Python 3.12 and python-jose 3.3.0. The mean bcrypt verification time from five runs is utilized as the reference cost for login. For the rejection experiment, $N = 2,000$ tokens were used for the tampering experiment in which one random bit was manipulated in every header/IV/ciphertext/tag (500 instances). The wrong-key decryption was conducted using a random key of 32 characters and an expiry experiment was done with tokens with an expiry of between 1 second and 24 hours (rejection) as well as within the lifetime (for acceptance) along with one live 2 seconds lifetime token. An IV length experiment used each claim and was encrypted 20,000 times, and a length leak experiment was conducted with 1,000 normal tokens and 1,000 administrator tokens.

VII. RESULTS

A. Structure and Size

Every token contained five segments that can be separated by a dot while signed JWTs had three segments. In the section about key encryption, there was nothing since it was the case of dir. Table II provides information on the real size compared to the theoretical values of the size described by Eqs. (2) and (3). The real and estimated size precisely match (JWE equals 241, JWT equals 201) leading to the token size of 40 characters (Eq. 4).

TABLE II
TOKEN SIZE: PREDICTED VS. MEASURED

Component	Bytes	Chars
JWE header	29	39
JWE encrypted key	0	0
JWE IV	16	22
JWE ciphertext	115	154
JWE tag	16	22
JWE total (pred./meas.)	—	241 / 241
JWS header	27	36
JWS payload	90	120
JWS signature	32	43
JWS total (pred./meas.)	—	201 / 201

B. Latency

The timings can be observed from Table III, while Figure 2(b) depicts the timings through graphical representation. The generating of JWE consumed 54.14 μ s versus 23.42 μ s of time for the signed JWT, thus giving a lag of around 31 μ s. The verification of JWE consumed 35.82 μ s as opposed to 39.86 μ s for signed JWT, meaning that it does not take longer; however, it should not be treated as the benefit of encryption since different groups of libraries were used along with claim validation for JWT. The single verification takes 36 μ s, which means that measurements amount to around 27,900 verifications per second per core.

TABLE III
LATENCY (MS, $N = 10,000$)

Operation	Mean	Median	P95	P99
JWS issue	23.42	20.90	34.82	53.53
JWE issue	54.14	47.92	81.40	113.11
JWS verify	39.86	35.86	59.58	81.53
JWE verify	35.82	31.98	54.77	81.52
bcrypt (ms)	297.33	—	—	—

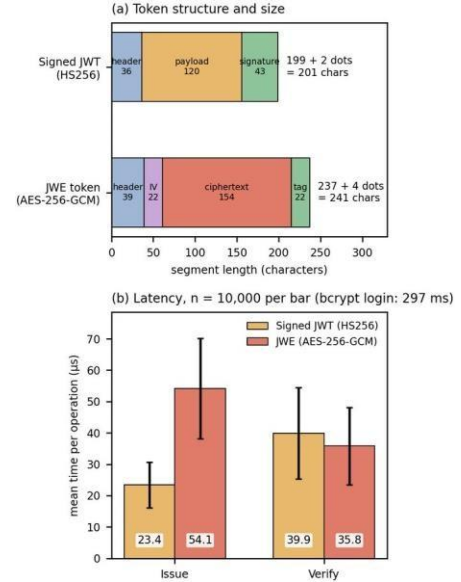


Fig. 2. (a) Token structure and size; (b) issuance/verification latency.

C. Security Experiments

The results of the security tests are depicted in Table IV. The interception of four out of four claims made by each signed JWTs took place ($Ex = 1.00$), but there were no claims recovered from JWE tokens ($Ex = 0.00$) in the experiments carried out. All malformed, invalid, and expired tokens were rejected. Notice that all valid tokens were accepted. Out of 20,000 of the identical claims being encrypted using the same secret key, each encryption produced a different pair of IV and ciphertext, thus confirming that $p_{IV} \approx 5.9 \times 10^{-31}$, as shown in Equation (7).

TABLE IV
SECURITY EXPERIMENTS

Experiment	N	Result
Claim fields recoverable, JWT	1,000	4 of 4 ($Ex=1.00$)
Claim fields recoverable, JWE	1,000	0 of 4 ($Ex=0.00$)
Bit flip (header/IV/CT/tag)	2,000	2,000 rejected
Bit flip, JWT payload	2,000	2,000 rejected
Wrong-key decryption	2,000	2,000 rejected
Expired tokens	2,000	2,000 rejected
Valid tokens	2,000	2,000 accepted
Live 2-s expiry test	1	ok@0s, rej.@3s
Distinct IVs / ciphertexts	20,000	20,000 / 20,000

D. Length Leakage

The JWE payload has been encrypted, which makes it unreadable. However, this process does not hide the length of tokens. The average token length is measured at 241 bytes, while the admin token length is 239 bytes because of the noticeable difference in length between the string values true and false. A simple classification method based on token length was able to classify the privilege type of tokens correctly at a 100% rate (Eq. 11a). Thus, the rate of information leakage is 1

bit per token.

Once the tokens were padded to a constant length $P=192$ bytes, their length grew to 343 bytes, which caused the accuracy of the classifier to decrease to the level of 50%, which is equivalent to blind guessing. In this case, the information leakage would equal 0 bits.

TABLE V
LENGTH LEAKAGE OF THE PRIVILEGE FLAG

Variant	Chars	Lengths	Accuracy
Unpadded	241 / 239	2	100%
Padded ($P=192$)	343	1	50%

VIII. DISCUSSION AND COMPARISON

The results confirm the goals of confidentiality and integrity, G1 and G2: For instance, the token does not display any of its four claim fields for parties not possessing the key, while all alterations are blocked in all experiments. The overhead is low in absolute terms, as the token takes up 19.9% more space, requires 31 μ s for issuance, does nearly the same for verification, which pales in comparison with the 297 ms necessary to hash a password during logging in. The length experiment proves that the confidentiality of content differs from the confidentiality of metadata: An attacker, while unable to obtain the value of the privilege field, may learn it by its length. Length padding removes this vulnerability with a minor cost. The analysis of the exposure window resolves the task described in goal G3: If T is set to be 20 minutes, the intercepted token can be expected to have a remaining useful life of 10 minutes, which is 72 times smaller than for a 24-hour token.

TABLE VI
COMPARISON WITH CONVENTIONAL SIGNED JWT

Aspect	JWS	JWE (proposed)
Segments	3	5
Fields readable w/o key	4 of 4	0 of 4
Tamper detection	Yes (sig.)	Yes (auth. tag)
Privilege inferable	Yes	If unpadded
Size (chars)	201	241 (343 padded)
Issue time (μ s)	23.42	54.14
Verify time (μ s)	39.86	35.82
Client integration	—	None (drop-in)
Exposure window	Depends	10 min ($T=20$ min)

IX. LIMITATIONS

There are many constraints that need to be considered. First of all, a single symmetric key is used that does not change, and this means that if the key is revealed, then all tokens signed by the key will be revealed as well. Secondly, there is no refresh-token implementation, and therefore the users have to authenticate again every 20 minutes. Thirdly, even though the library creates IVs that are 128 bits long, it should have created 96-bit long IVs according to standard [8] for AES-GCM. However, in this case, it is acceptable since only one server signs and validates tokens. Fourthly, the benchmark was conducted on the padded version of the algorithm, and this version was not used on the endpoint yet. Moreover, every bearer token can be used more than one time within its validity period since the tokens do not correspond to any specific client. Finally,

the tampering test only alters one random bit to make it hardly resemble the real-life attack by an ordinary script.

timing depends on hardware and is done locally, all experiments were in-process too.

X. CONCLUSION AND FUTURE WORK

In this paper it has been proposed to use encrypted and temporary API access tokens created with JWE and AES-256-GCM whose life span is 20 minutes. An analytical model was created to determine the optimal size for the token (241 characters). During the research, it was noted that 0 of 4 claims are revealed to the interceptor in case of using the token while it is equal to 4 of 4 in case of using a signed JWT. More than 2000 counterfeit tokens were tested and rejected as well as 2000 tokens created using wrong key. Also, the number of rejected tokens created by using the expired tokens amounted to 2000. Finding the length of ciphertext made it known that figuring out the privilege bit can be achieved successfully every time. Since utilization of fixed-length padding made this impossible adding 102 characters makes this issue unthinkable. This paper synthesized while doing research on lightweight API security along with the companion detection system. Subsequent work will include adding padding to the deployed endpoint, key rotation, refresh tokens, 96-bit IVs to ensure interoperability, client binding in order to prevent replay attacks, and storing token usage data in the companion detection system.

ACKNOWLEDGMENT

Fig. 1 and Fig. 2 in this paper were generated using Claude (Anthropic), an AI tool. All other content, including the implementation, experiments, and results, is the authors' own work.

REFERENCES

- [1] M. Jones and J. Hildebrand, "JSON Web Encryption (JWE)," Internet Eng. Task Force, RFC 7516, 2015.
- [2] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Eng. Task Force, RFC 7519, 2015.
- [3] D. McGrew and J. Viega, "The Galois/counter mode of operation (GCM)," in *Submission to NIST Modes of Operation Process*, vol. 20, 2004, pp. 0278–0070.
- [4] S. J. García Zambrano, D. A. Terán Piedra, E. F. Paredes Nacipucha, and D. A. Dávila Cárdenas, "Diseño de una metodología de Pentesting para APIs RESTful basada en los cinco principales riesgos del OWASP API Security Top 10 (2023)," Doctoral dissertation, UIDE, Quito, Ecuador, 2026.
- [5] D. Hardt, "The OAuth 2.0 authorization framework," Internet Eng. Task Force, RFC 6749, 2012.
- [6] M. Jones, "JSON Web Signature (JWS) unencoded payload option," Internet Eng. Task Force, RFC 7797, 2016.
- [7] M. Jones, "JSON Web Algorithms (JWA)," Internet Eng. Task Force, RFC 7518, May 2015.
- [8] M. Hofmeier and W. Hommel, "Enabling the JSON web signature format to support complex and identity-oriented non-web processes," in *Proc. Int. Conf. Innov. Community Services*, Cham, Switzerland: Springer Nature Switzerland, Sep. 2023, pp. 29–47.
- [9] T. Lodderstedt, M. McGloin, and P. Hunt, "OAuth 2.0 threat model and security considerations," Internet Eng. Task Force, RFC 6819, 2013.
- [10] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in *Proc. 2016 ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1204–1215.
- [11] N. Sakimura, J. Bradley, and N. Agarwal, "Proof key for

- code exchange by OAuth public clients,” Internet Eng. Task Force, RFC 7636, 2015.
- [12] J. Richer, “OAuth 2.0 token introspection,” Internet Eng. Task Force, RFC 7662, 2015.
 - [13] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, “OpenID Connect Core 1.0 incorporating errata set 1,” OpenID Foundation, Specification, 2014.
 - [14] E. Rescorla, “The transport layer security (TLS) protocol version 1.3,” Internet Eng. Task Force, RFC 8446, 2018.
 - [15] NIST, “Advanced encryption standard (AES),” Nat. Inst. Stand. Technol., FIPS Pub. 197, 2001.

KL KL

Paper2_IEEE_Formatted_Fixed_title_updated

 Research PB

 PB

 A'Sharqiyah University, Oman

Document Details

Submission ID

trn:oid::1:3666803744

Submission Date

Oct 2, 2026, 12:01 PM GMT+4

Download Date

Oct 2, 2026, 12:04 PM GMT+4

File Name

Paper2_IEEE_Formatted_Fixed_title_updated.docx

File Size

224.6 KB

7 Pages

4,250 Words

21,576 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Hardening API Authentication with Encrypted, Short-Lived Access Tokens Using JSON Web Encryption

Mrs. Shyni

Assistant Professor (SS)
Rajalakshmi Engineering College
Chennai, India
shyni.m@rajalakshmi.edu.in

Varshabala

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
prathikshathila@gmail.com

Tejushree Sanjeevi Kumar

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
tejushreebv@gmail.com

Valluru Varshini

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
valluruvarshini9@gmail.com

Abstract—Token authentication is used in API which helps to identify the user who logs into the system. The usage of conventional JWT means that it is not encrypted, this means that anyone who intercepts the token will be able to decode the information contained in it such as user id, token id, expiry time, privileges etc. The purpose of this research is to talk about improving the token generation mechanism in FastAPI web application by means of usage of JWE technique that uses AES 256 GCM encryption and establishes fixed time period of 20 minutes. According to the analytical model, the precise token length is given (predicted length is 241 characters against measured length). Furthermore, the maximum of forgery probability is $q \cdot 2^{-128}$ and the average duration for which the token can be intercepted is $T/2 = 10$ minutes. By conducting experiments, the attacker managed to extract data from 4 out of 4 letters in the signed JWT but did not manage to decode letters in the form of JWE. Among 2,000 tokens, related to the cases of token usage with one flipped bit [2,000 tokens with a wrong decryption key and 2,000 tokens that became invalid], only valid tokens were used. Moreover, 20,000 encryptions of the same texts resulted in obtaining 20,000 various IVs. The obtained data presented above can be compared with the previous results related to the signed JWT, which indicated a difference between the length of the text presented as a signed JWT and the text, issued in the form of the JWE.

Index Terms—API security, JSON Web Encryption, JWE, token authentication, credential confidentiality, AES-256-GCM, traffic analysis

I. INTRODUCTION

Almost all the APIs in existence nowadays provide a token after successful authentication allowing identification of the user in subsequent requests. The most common example is the JSON Web Token (JWT) which consists of three parts - header, payload, and signature. The signature guarantees that the payload has not been modified while payload remains only base64url encoded. Although implicating the token via interception or through misconfiguration or hacking won't generate another token, as anyone suffering from a mishap will be able to access the token. The claims in the token such as the user ID, token ID, validity time, and privilege flag could therefore be read by anyone who intercepts the token..

This research is part of a more extensive research project that investigates lightweight API security and includes an implementation of a real-time detection system that focuses on five categories from the OWASP API Security Top 10: broken authentication, unrestricted resource consumption, business flow abuse, broken object level authorization, and broken function level authorization (2). The detection system takes place on a simulated FastAPI backend, recognizes each incoming request in real-time through fast checks, while leaving analysis based on machine learning for after-the-fact analysis of suspicious requests. This paper is a continuation of this project, which is aimed at hardening the credential layer that the traffic relies on: access tokens generated following a successful authentication. The detection system in (2) recognizes suspicious traffic flows but does not take into account the security of tokens used in this traffic.

This issue of integrity separation, which is provided by the received JWT, is addressed by the established IETF specification called JSON Web Encryption (JWE) [1]. The main contributions of this paper are as follows:

- Development of a threat model and a development of the analytical model of the mechanism of JWE token giving the closed formulas for token size, forgery probability, key search effort, probability of IV collision and interception window for the token which has been intercepted.
- Creation of a reference implementation of the short-term encrypted token in a FastAPI backend which can be easily used in place of the signed JWT issuing without any changes needed from the client side.
- Development of a reproducible benchmark which measures in size, latency, confidentiality, tampering refusal, wrong key refusal, expiration honoring and IV unique, which can be compared with the analytical model.

- Discovery of the fact that ciphertext size allows revealing the privilege flag in correlation with the constant padding length and its overhead.

The rest of the paper is structured as follows. Related work is discussed in Section II. The threat model is presented in Section III. The mechanism is described in Section IV. The analytical model is developed in Section V. The experimental setup is described in Section VI. The results are presented in Section VII. Discussion and comparison to conventional JWT issuance is provided in Section VIII. Limitations are discussed in Section IX, and conclusions are given in Section X.

II. RELATED WORK

JSON Web Token (JWT) [3] describes a compact and URL-safe encoding for claims carried between two entities, usually digitally signed with either HMAC or RSA to ensure integrity. A digitally signed-only JWT (JWS) [7] lacks confidentiality: its payload is only base64url encoded, easily decodable without the signing key. Current best practices for using JSON Web Tokens [19] and for bearer tokens [21] in general focus on selecting an algorithm and verifying integrity rather than protecting the content of the token.

JSON Web Encryption (JWE) [1] guarantees confidentiality through a compact format who contains five different parts: protected header, encrypted key, IV, ciphertext, and authentication tag. The algorithms use by JWE are defined in JSON Web Algorithms (JWA) [8], and the keys are provided in JSON Web Keys (JWK) [9]. The scheme used in this case is AES-256-GCM, which is an authenticated encryption scheme that guarantees both confidentiality and integrity of the data by means of a single cryptographic primitive [4], [18]; AES is standardized in FIPS PUB 197 [16] and its key management is described in NIST SP 800-57 [17].

One of the categories API vulnerability is Broken Authentication, and the examples include credential stuffing, lack of proper token generation, and the case when the contents of the token are exposed to malicious third parties. Most of the important recommendation deals with detection of stolen tokens abuse, while there is not much information available about what the tokens are capable of revealing to the attacker until the abuse has already taken place.

A wider array of standards associated with OAuth 2.0 provides additional context. The framework covering OAuth 2.0 [6], its threat model [10], and the security protocol analysis [11] describe the types of vulnerabilities in authorization flows apart from keeping the confidentiality of the token. PKCE [12] and Token Introspection [13] address the problems that are related but different than those mentioned above while OpenID Connect [14] provides identity claims. Protection against token interception at the transport layer is achieved by using TLS 1.3 [15] and this protection complements the protection of the payload since the token is store in a decrypted form in proxy servers, logs, as well as browser storage. Short-lived credentials are the well-known design principle that limits the lifetime of stolen credentials [6]. The passwords that are hashed using bcrypt [20] are purposely expensive, which cost is referred to in Section VII when determining token overhead. Unlike the usual token recommendation, this work compares the present solution against traffic-level detector [2] at the same API testbed.

III. THREAT MODEL AND SECURITY GOALS

The asset consists of the four claims which the token carries: sub, jti, exp, and is_admin. Let F be the set containing these assets and $|F| = 4$. The server stores only one 256-bit symmetric key which never leaves the server. There are three security objectives formulated: G1, ensuring the secrecy of F from an adversary which does not have access to the key; G2, providing the integrity of the token so that any modification of the token is detected; and G3, a limited useful lifetime for a compromised token. The adversaries considered are summarized in Table I.

TABLE I
THREAT MODEL

Adversary	Capability	Goal
A1 Passive interceptor	Observes tokens in transit, in logs, or at rest; no key	G1
A2 Active tamperer	Modifies any part of a token and replays it	G2
A3 Key guesser	Tries to decrypt or forge tokens by guessing the key	G1, G2
A4 Late replayer	Replays a captured, unmodified token after a delay	G3

A token replayed unmodified within its lifetime is accepted, because a bearer token is not bound to a client. The mechanism bounds this risk through the short expiry (G3) but does not eliminate it.

IV. PROPOSED MECHANISM

A. Token Issuance

After successful authentication, the server will create the claim object with 4 attributes. The given claim object is then converted into JSON format and encrypted using the JWE using the dir key wrapping mechanism (in this mechanism, the symmetric key is used directly without any wrapping of keys) and the encryption algorithm used is A256GCM. The issuance is given in Algorithm 1.

Algorithm 1: Token issuance

Input: user id u , privilege flag a , server key K , lifetime T .

1. $jti \leftarrow$ new random UUID.
2. $exp \leftarrow$ current time + T .
3. $claims \leftarrow \{sub: u, jti: jti, exp: exp, is_admin: a\}$.
4. $IV \leftarrow$ fresh random value.
5. $(C, tag) \leftarrow \text{AES-256-GCM-Encrypt}(K, IV, \text{JSON}(claims))$.
6. Return $header.empty().IV.C.tag, \text{base64url-encoded}$.

B. Token Verification

The token is decrypted at the server using the same key for every request. In the case of unsuccessful decryption or signature verification, the request would be denied. On the other hand, if decryption was done successfully but the token has expired, then the request still would be denied. Algorithm two provides a summary for verification.

Algorithm 2: Token verification

Input: token, server key K .

1. Split token into header, key, IV, C, tag.
2. Decrypt C under K, IV ; verify tag; else reject.
3. Parse JSON claims; on failure, reject.
4. If exp is not later than now, reject.
5. Return claims.

C. Short Expiry

Tokens have a lifespan of $T=20$ minutes. This restricts the period in which any intercepted tokens will remain valid, regardless of when the interception is noticed. The precise period is examined in Section V-D.

D. Fixed-Length Claim Padding (Extension)

The claim has its contents encoded, but its size is transparent. This is because the size of JSON text representing a privileged token is different from the size of the JSON text that represents a common token. To alleviate this issue, the serialized claims are padded with an additional field so that the plaintext is always of size P . The complete process is shown in Fig. 1.

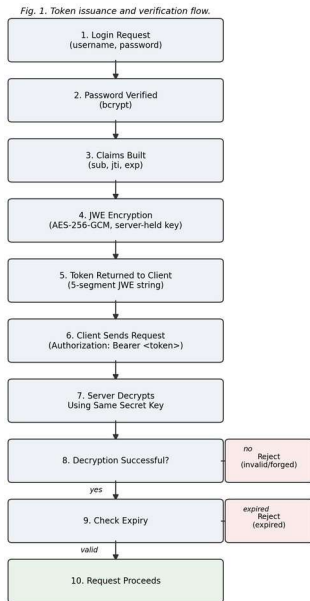


Fig. 1. Token issuance and verification flow.

V. ANALYTICAL MODEL

A. Token Size

The unpadded base64url representation of n bytes yields the character length given by

$$L(n) = \lceil 4n / 3 \rceil \quad (1)$$

characters. A compact JSON Web Encryption (JWE) token consists of a header of h bytes, an encrypted key of k bytes (with k equal to 0 when the encryption is performed by means of a symmetric key), an initialization vector (IV) of v bytes, an encrypted message of m bytes (where $m = p$ since Galois/Counter Mode (GCM) retains the size of plaintext), and a tag of τ bytes, as represented in the following equation (note that the token parts are divided by four dots):

$$S_{JWE} = L(h) + L(k) + L(v) + L(m) + L(\tau) + 4 \quad (2)$$

In turn, a signed JSON Web Token (JWT) using the HMAC SHA256 algorithm (HS256) includes a header of h' bytes, a payload of P bytes, a 32-byte signature, and two dots

$$S_{JWS} = L(h') + L(p) + L(32) + 2 \quad (3)$$

Therefore, the relative size of overhead of encryption

$$\text{overhead} = (S_{JWE} - S_{JWS}) / S_{JWS} \quad (4)$$

B. Forgery and Key Search

For a t -bit authentication tag, the probability a single forged token is accepted is on the order of 2^{-t} [4], so an adversary making q forgery attempts succeeds with probability at most

$$P_{\text{forge}}(q) \leq q \cdot 2^{-t} \quad (5)$$

For $t = 128$ and $q = 240$ attempts, P_{forge} is at most 2^{-88} , about 3.2×10^{-27} . Exhaustive search of a k -bit key needs on average $2^k - 1$ trials, so at R trials per second

$$T_{\text{search}} = 2^k / R \quad (6)$$

For $k = 256$ and $R = 1012$ trials/s, T_{search} is about 5.8×10^{64} seconds, or about 1.8×10^{57} years.

C. IV Collision

Each token uses a fresh random IV of b bits. By the birthday bound, the probability that any two of n tokens share an IV is approximately

$$p_{IV}(n) \approx n(n-1) / 2^{b+1} \quad (7)$$

and the number of tokens issuable under one key before this probability reaches a target p is

$$n^*(p) = \sqrt{2^{b+1} \cdot p} \quad (8)$$

With the measured $b = 128$, $p_{IV}(20,000)$ is about 5.9×10^{-31} , and $n^*(2^{-32})$ is about 4.0×10^{14} tokens, so key lifetime is not limited by IV collisions at this system's scale.

D. Exposure Window

Suppose a token of lifetime T is intercepted at a time u distributed uniformly over its lifetime. The remaining useful time is $W = T - u$, so

$$E[W] = T / 2 \quad (9a)$$

$$P(\text{valid after delay } d) = \max(0, 1 - d/T) \quad (9b)$$

For $T = 20$ minutes, $E[W] = 10$ minutes and the worst case is 20 minutes. Relative to a reference lifetime T_{ref} , the exposure window is reduced by the factor T_{ref}/T ; for a 24-hour token this factor is 72.

E. Confidentiality Metrics

Let $1[f]$ equal 1 if field f in F can be recovered without the key. The exposure ratio is

$$E_x = (1/|F|) \sum_{f \in F} 1[f] \quad (10)$$

For a length-based classifier predicting privilege class C from token length Λ , with $N(l, c)$ tokens of length l and class c among N tokens, accuracy and leaked information are

$$A = (1/N) \sum_l \max_c N(l, c) \quad (11a)$$

$$I(\Lambda; C) = H(C) - H(C|\Lambda) \quad (11b)$$

For balanced classes, $H(C) = 1$ bit and chance accuracy is 0.5. Padding every plaintext to a fixed length P gives a padding field of length

$$\pi = P - m_0 \geq 0 \quad (12)$$

where m_0 is the unpadded plaintext length, making Λ constant, so $I(\Lambda; C) = 0$.

VI. EXPERIMENTAL SETUP

Every experiment utilizes the same token module call, the same key, and a backend lifetime of 20 minutes, and a single script is executed to recreate the numbers with the same random seed of 42. The baseline uses the HS256 JWT signature with four identical claims. All measurements are based on $n = 10,000$ measurements being made while 300 warmed-up executions are used; results include mean, median, standard deviation, and 95th and 99th percentiles. The experiments were conducted in a single-core environment using Python 3.12 and python-jose 3.3.0. The mean bcrypt verification time from five runs is utilized as the reference cost for login. For the rejection experiment, $N = 2,000$ tokens were used for the tampering experiment in which one random bit was manipulated in every header/IV/ciphertext/tag (500 instances). The wrong-key decryption was conducted using a random key of 32 characters and an expiry experiment was done with tokens with an expiry of between 1 second and 24 hours (rejection) as well as within the lifetime (for acceptance) along with one live 2 seconds lifetime token. An IV length experiment used each claim and was encrypted 20,000 times, and a length leak experiment was conducted with 1,000 normal tokens and 1,000 administrator tokens.

VII. RESULTS

A. Structure and Size

Every token contained five segments that can be separated by a dot while signed JWTs had three segments. In the section about key encryption, there was nothing since it was the case of dir. Table II provides information on the real size compared to the theoretical values of the size described by Eqs. (2) and (3). The real and estimated size precisely match (JWE equals 241, JWT equals 201) leading to the token size of 40 characters (Eq. 4).

TABLE II
TOKEN SIZE: PREDICTED VS. MEASURED

Component	Bytes	Chars
JWE header	29	39
JWE encrypted key	0	0
JWE IV	16	22
JWE ciphertext	115	154
JWE tag	16	22
JWE total (pred./meas.)	—	241 / 241
JWS header	27	36
JWS payload	90	120
JWS signature	32	43
JWS total (pred./meas.)	—	201 / 201

B. Latency

The timings can be observed from Table III, while Figure 2(b) depicts the timings through graphical representation. The generating of JWE consumed 54.14 μ s versus 23.42 μ s of time for the signed JWT, thus giving a lag of around 31 μ s. The verification of JWE consumed 35.82 μ s as opposed to 39.86 μ s for signed JWT, meaning that it does not take longer; however, it should not be treated as the benefit of encryption since different groups of libraries were used along with claim validation for JWT. The single verification takes 36 μ s, which means that measurements amount to around 27,900 verifications per second per core.

TABLE III
LATENCY (MS, $N = 10,000$)

Operation	Mean	Median	P95	P99
JWS issue	23.42	20.90	34.82	53.53
JWE issue	54.14	47.92	81.40	113.11
JWS verify	39.86	35.86	59.58	81.53
JWE verify	35.82	31.98	54.77	81.52
bcrypt (ms)	297.33	—	—	—

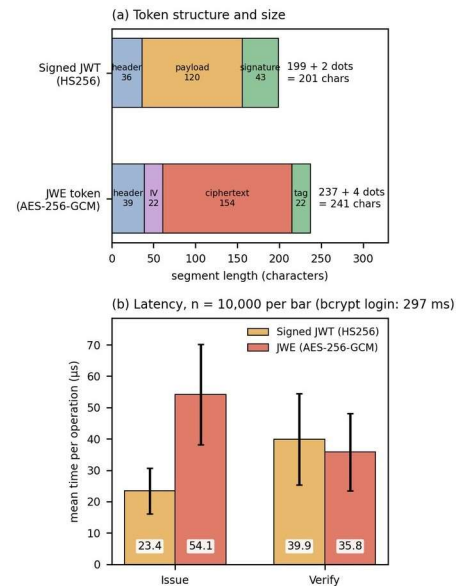


Fig. 2. (a) Token structure and size; (b) issuance/verification latency.

C. Security Experiments

The results of the security tests are depicted in Table IV. The interception of four out of four claims made by each signed JWTs took place ($Ex = 1.00$), but there were no claims recovered from JWE tokens ($Ex = 0.00$) in the experiments carried out. All malformed, invalid, and expired tokens were rejected. Notice that all valid tokens were accepted. Out of 20,000 of the identical claims being encrypted using the same secret key, each encryption produced a different pair of IV and ciphertext, thus confirming that $p_{IV} \approx 5.9 \times 10^{-31}$, as shown in Equation (7).

TABLE IV
SECURITY EXPERIMENTS

Experiment	N	Result
Claim fields recoverable, JWT	1,000	4 of 4 ($Ex=1.00$)
Claim fields recoverable, JWE	1,000	0 of 4 ($Ex=0.00$)
Bit flip (header/IV/CT/tag)	2,000	2,000 rejected
Bit flip, JWT payload	2,000	2,000 rejected
Wrong-key decryption	2,000	2,000 rejected
Expired tokens	2,000	2,000 rejected
Valid tokens	2,000	2,000 accepted
Live 2-s expiry test	1	ok@0s, rej.@3s
Distinct IVs / ciphertexts	20,000	20,000 / 20,000

D. Length Leakage

The JWE payload has been encrypted, which makes it unreadable. However, this process does not hide the length of tokens. The average token length is measured at 241 bytes, while the admin token length is 239 bytes because of the noticeable difference in length between the string values true and false. A simple classification method based on token length was able to classify the privilege type of tokens correctly at a 100% rate (Eq. 11a). Thus, the rate of information leakage is 1

bit per token.

Once the tokens were padded to a constant length $P=192$ bytes, their length grew to 343 bytes, which caused the accuracy of the classifier to decrease to the level of 50%, which is equivalent to blind guessing. In this case, the information leakage would equal 0 bits.

TABLE V
LENGTH LEAKAGE OF THE PRIVILEGE FLAG

Variant	Chars	Lengths	Accuracy
Unpadded	241 / 239	2	100%
Padded ($P=192$)	343	1	50%

VIII. DISCUSSION AND COMPARISON

The results confirm the goals of confidentiality and integrity, G1 and G2: For instance, the token does not display any of its four claim fields for parties not possessing the key, while all alterations are blocked in all experiments. The overhead is low in absolute terms, as the token takes up 19.9% more space, requires 31 μ s for issuance, does nearly the same for verification, which pales in comparison with the 297 ms necessary to hash a password during logging in. The length experiment proves that the confidentiality of content differs from the confidentiality of metadata: An attacker, while unable to obtain the value of the privilege field, may learn it by its length. Length padding removes this vulnerability with a minor cost. The analysis of the exposure window resolves the task described in goal G3: If T is set to be 20 minutes, the intercepted token can be expected to have a remaining useful life of 10 minutes, which is 72 times smaller than for a 24-hour token.

TABLE VI
COMPARISON WITH CONVENTIONAL SIGNED JWT

Aspect	JWS	JWE (proposed)
Segments	3	5
Fields readable w/o key	4 of 4	0 of 4
Tamper detection	Yes (sig.)	Yes (auth. tag)
Privilege inferable	Yes	If unpadded
Size (chars)	201	241 (343 padded)
Issue time (μ s)	23.42	54.14
Verify time (μ s)	39.86	35.82
Client integration	—	None (drop-in)
Exposure window	Depends	10 min ($T=20$ min)

IX. LIMITATIONS

There are many constraints that need to be considered. First of all, a single symmetric key is used that does not change, and this means that if the key is revealed, then all tokens signed by the key will be revealed as well. Secondly, there is no refresh-token implementation, and therefore the users have to authenticate again every 20 minutes. Thirdly, even though the library creates IVs that are 128 bits long, it should have created 96-bit long IVs according to standard [8] for AES-GCM. However, in this case, it is acceptable since only one server signs and validates tokens. Fourthly, the benchmark was conducted on the padded version of the algorithm, and this version was not used on the endpoint yet. Moreover, every bearer token can be used more than one time within its validity period since the tokens do not correspond to any specific client. Finally,

timing depends on hardware and is done locally, all experiments were in-process too.

X. CONCLUSION AND FUTURE WORK

In this paper it has been proposed to use encrypted and temporary API access tokens created with JWE and AES-256-GCM whose life span is 20 minutes. An analytical model was created to determine the optimal size for the token (241 characters). During the research, it was noted that 0 of 4 claims are revealed to the interceptor in case of using the token while it is equal to 4 of 4 in case of using a signed JWT. More than 2000 counterfeit tokens were tested and rejected as well as 2000 tokens created using wrong key. Also, the number of rejected tokens created by using the expired tokens amounted to 2000. Finding the length of ciphertext made it known that figuring out the privilege bit can be achieved successfully every time. Since utilization of fixed-length padding made this impossible adding 102 characters makes this issue unthinkable. This paper synthesized while doing research on lightweight API security along with the companion detection system. Subsequent work will include adding padding to the deployed endpoint, key rotation, refresh tokens, 96-bit IVs to ensure interoperability, client binding in order to prevent replay attacks, and storing token usage data in the companion detection system.

ACKNOWLEDGMENT

Fig. 1 and Fig. 2 in this paper were generated using Claude (Anthropic), an AI tool. All other content, including the implementation, experiments, and results, is the authors' own work.

REFERENCES

- [1] M. Jones and J. Hildebrand, "JSON Web Encryption (JWE)," Internet Eng. Task Force, RFC 7516, 2015.
- [2] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Eng. Task Force, RFC 7519, 2015.
- [3] D. McGrew and J. Viegas, "The Galois/counter mode of operation (GCM)," in *Submission to NIST Modes of Operation Process*, vol. 20, 2004, pp. 0278–0070.
- [4] S. J. García Zambrano, D. A. Terán Piedra, E. F. Paredes Nacipucha, and D. A. Dávila Cárdenas, "Diseño de una metodología de Pentesting para APIs RESTful basada en los cinco principales riesgos del OWASP API Security Top 10 (2023)," Doctoral dissertation, UIDE, Quito, Ecuador, 2026.
- [5] D. Hardt, "The OAuth 2.0 authorization framework," Internet Eng. Task Force, RFC 6749, 2012.
- [6] M. Jones, "JSON Web Signature (JWS) unencoded payload option," Internet Eng. Task Force, RFC 7797, 2016.
- [7] M. Jones, "JSON Web Algorithms (JWA)," Internet Eng. Task Force, RFC 7518, May 2015.
- [8] M. Hofmeier and W. Hommel, "Enabling the JSON web signature format to support complex and identity-oriented non-web processes," in *Proc. Int. Conf. Innov. Community Services*, Cham, Switzerland: Springer Nature Switzerland, Sep. 2023, pp. 29–47.
- [9] T. Lodderstedt, M. McGloin, and P. Hunt, "OAuth 2.0 threat model and security considerations," Internet Eng. Task Force, RFC 6819, 2013.
- [10] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in *Proc. 2016 ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1204–1215.
- [11] N. Sakimura, J. Bradley, and N. Agarwal, "Proof key for

- code exchange by OAuth public clients,” Internet Eng. Task Force, RFC 7636, 2015.
- [12] J. Richer, “OAuth 2.0 token introspection,” Internet Eng. Task Force, RFC 7662, 2015.
- [13] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, “OpenID Connect Core 1.0 incorporating errata set 1,” OpenID Foundation, Specification, 2014.
- [14] E. Rescorla, “The transport layer security (TLS) protocol version 1.3,” Internet Eng. Task Force, RFC 8446, 2018.
- [15] NIST, “Advanced encryption standard (AES),” Nat. Inst. Stand. Technol., FIPS Pub. 197, 2001.

- [16] E. Barker, W. Barker, W. Burr, W. Polk, and M. Smid, "Recommendation for key management part 1: General (revision 3)," Nat. Inst. Stand. Technol., NIST Spec. Publ. 800-57, 2012, pp. 1–147.
- [17] M. Dworkin, "Recommendation for block cipher modes of operation," Nat. Inst. Stand. Technol., NIST Spec. Publ. 800-38B, 2001.
- [18] Y. Sheffer, D. Hardt, and M. Jones, "JSON web token best current practices," Internet Eng. Task Force, RFC 8725, 2020.
- [19] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, Jun. 1999, pp. 81–91.
- [20] M. Jones and D. Hardt, "The OAuth 2.0 authorization framework: Bearer token usage," Internet Eng. Task Force, RFC 6750, 2012.

Paper2_IEEE_Formatted_Fixed_title_updated

by KL KL

Submission date: 02-Oct-2026 12:01PM (UTC+0400)

Submission ID: 3041465849

File name: Paper2_IEEE_Formatted_Fixed_title_updated.docx (224.6K)

Word count: 4250

Character count: 21576

Hardening API Authentication with Encrypted, Short-Lived Access Tokens Using JSON Web Encryption

Mrs. Shyni

¹² Assistant Professor (SS)
Rajalakshmi Engineering College
Chennai, India
shyni.m@rajalakshmi.edu.in

Varshabala

¹ Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
prathikshathila@gmail.com

¹ Tejushree Sanjeevi Kumar
Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
tejushreebv@gmail.com

Valluru Varshini

Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
valluruvarshini9@gmail.com

Abstract—Token authentication is used in API which helps to identify the user who logs into the system. The usage of conventional JWT means that it is not encrypted, this means that anyone who intercepts the token will be able to decode the information contained in it such as user id, token id, expiry time, privileges etc. The purpose of this research is to talk about improving the token generation mechanism in FastAPI web application by means of usage of JWE technique that uses AES 256 GCM encryption and establishes fixed time period of 20 minutes. According to the analytical model, the precise token length is given (predicted length is 241 characters against measured length). Furthermore, the maximum of forgery probability is $q \cdot 2^{-128}$ and the average duration for which the token can be intercepted is $T/2 = 10$ minutes. By conducting experiments, the attacker managed to extract data from 4 out of 4 letters in the signed JWT but did not manage to decode letters in the form of JWE. Among 2,000 tokens, related to the cases of token usage with one flipped bit [2,000 tokens with a wrong decryption key and 2,000 tokens that became invalid], only valid tokens were used. Moreover, 20,000 encryptions of the same texts resulted in obtaining 20,000 various IVs. The obtained data presented above can be compared with the previous results related to the signed JWT, which indicated a difference between the length of the text presented as a signed JWT and the text, issued in the form of the JWE.

Index Terms—API security, JSON Web Encryption, JWE, token authentication, credential confidentiality, AES-256-GCM, traffic analysis

I. INTRODUCTION

Almost all the APIs in existence nowadays provide a token after successful authentication allowing identification of the user in subsequent requests. The most common example is the JSON Web Token (JWT) which consists of three parts - header, payload, and signature. The signature guarantees that the payload has not been modified while payload remains only base64url encoded. Although implicating the token via interception or through misconfiguration or hacking won't generate another token, as anyone suffering from a mishap will be able to access the token. The claims in the token such as the user ID, token ID, validity time, and privilege flag could therefore be read by anyone who intercepts the token..

This research is part of a more extensive research project that investigates lightweight API security and includes an implementation of a real-time detection system that focuses on five categories from the OWASP API Security Top 10: broken authentication, unrestricted resource consumption, business flow abuse, broken object level authorization, and broken function level authorization (2). The detection system takes place on a simulated FastAPI backend, recognizes each incoming request in real-time through fast checks, while leaving analysis based on machine learning for after-the-fact analysis of suspicious requests. This paper is a continuation of this project, which is aimed at hardening the credential layer that the traffic relies on: access tokens generated following a successful authentication. The detection system in (2) recognizes suspicious traffic flows but does not take into account the security of tokens used in this traffic.

This issue of integrity separation, which is provided by the received JWT, is addressed by the established IETF specification called JSON Web Encryption (JWE) [1]. The main contributions of this paper are as follows:

- Development of a threat model and a development of the analytical model of the mechanism of JWE token giving the closed formulas for token size, forgery probability, key search effort, probability of IV collision and interception window for the token which has been intercepted.
- Creation of a reference implementation of the short-term encrypted token in a FastAPI backend which can be easily used in place of the signed JWT issuing without any changes needed from the client side.
- Development of a reproducible benchmark which measures in size, latency, confidentiality, tampering refusal, wrong key refusal, expiration honoring and IV unique, which can be compared with the analytical model.

- Discovery of the fact that ciphertext size allows revealing the privilege flag in correlation with the constant padding length and its overhead.

7 The rest of the paper is structured as follows. Related work is discussed in Section II. The threat model is presented in Section III. The mechanism is described in Section IV. The analytical model is developed in Section V. The experimental setup is described in Section VI. The results are presented in Section VII. Discussion and comparison to conventional JWT issuance is provided in Section VIII. Limitations are discussed in Section IX, and conclusions are given in Section X.

II. RELATED WORK 2

JSON Web Token (JWT) [3] describes a compact and URL-safe encoding for claims carried between two entities, usually digitally signed with either HMAC or RSA to ensure integrity. A digitally signed-only JWT (JWS) [7] lacks confidentiality: its payload is only base64url encoded, easily decodable without the signing key. Current best practices for using JSON Web Tokens [19] and for bearer tokens [21] in general focus on selecting an algorithm and verifying integrity rather than protecting the content of the token.

JSON Web Encryption (JWE) [1] guarantees confidentiality through a compact format who contains five different parts: protected header, encrypted key, IV, ciphertext, and authentication tag. The algorithms use by JWE are defined in JSON Web Algorithms (JWA) [8], and the keys are provided in JSON Web Keys (JWK) [9]. The scheme used in 4 is case is AES-256-GCM, which is an authenticated encryption scheme that guarantees both confidentiality and integrity of the data by means of a single cryptographic primitive [4], [18]; AES is standardized in FIPS PUB 197 [16] and its key management is described in NIST SP 800-57 [17].

One of the categories API vulnerability is Broken Authentication, and the examples include credential stuffing, lack of proper token generation, and the case when the contents of the token are exposed to malicious third parties. Most of the important recommendation deals with detection of stolen tokens abuse, while there is not much information available about what the tokens are capable of revealing to the attacker until the abuse has already taken place.

A wider array of standards associated with OAuth 2.0 provides additional context. The framework covering OAuth 2.0 [6], its threat model [10], and the security protocol analysis [11] describe the types of vulnerabilities in authorization flows apart from keeping the confidentiality of the token. PKCE [12] and Token Introspection [13] address the problems that are related but different than those mentioned above while OpenID Connect [14] provides identity claims. Protection against token interception at the transport layer is achieved by using TLS 1.3 [15] and this protection complements the protection of the payload since the token is store in a decrypted form in proxy servers, logs, as well as browser storage. Short-lived credentials are the well-known design principle that limits the lifetime of stolen credentials [6]. The passwords that are hashed using bcrypt [20] are purposely expensive, which cost is referred to in Section VII when determining token overhead. Unlike the usual token recommendation, this work compares the present solution against traffic-level detector [2] at the same API testbed.

III. THREAT MODEL AND SECURITY GOALS

The asset consists of the four claims which the token carries: sub, jti, exp, and is_admin. Let F be the set containing these assets and $|F| = 4$. The server stores only one 256-bit symmetric key which never leaves the server. There are three security objectives formulated: G1, ensuring the secrecy of F from an adversary which does not have access to the key; G2, providing the integrity of the token so that any modification of the token is detected; and G3, a limited useful lifetime for a compromised token. The adversaries considered are summarized in Table I.

TABLE I
THREAT MODEL

Adversary	Capability	Goal
A1 Passive interceptor	Observes tokens in transit, in logs, or at rest; no key	G1
A2 Active tamperer	Modifies any part of a token and replays it	G2
A3 Key guesser	Tries to decrypt or forge tokens by guessing the key	G1, G2
A4 Late replayer	Replays a captured, unmodified token after a delay	G3

A token replayed unmodified within its lifetime is accepted, because a bearer token is not bound to a client. The mechanism bounds this risk through the short expiry (G3) but does not eliminate it.

IV. PROPOSED MECHANISM

A. Token Issuance

After successful authentication, the server will create the claim object with 4 attributes. The given claim object is then converted into JSON format and encrypted using the JWE using the dir key wrapping mechanism (in this mechanism, the symmetric key is used directly without any wrapping of keys) and the encryption algorithm used is A256GCM. The issuance is given in Algorithm 1.

Algorithm 1: Token issuance

Input: user id u , privilege flag a , server key K , lifetime T .

1. $jti \leftarrow$ new random UUID.
2. $exp \leftarrow$ current time + T .
3. $claims \leftarrow \{sub: u, jti: jti, exp: exp, is_admin: a\}$.
4. $IV \leftarrow$ fresh random value.
5. $(C, tag) \leftarrow \text{AES-256-GCM-Encrypt}(K, IV, \text{JSON}(claims))$.
6. Return header.empty).IV.C.tag, base64url-encoded.

B. Token Verification

The token is decrypted at the server using the same key for every request. In the case of unsuccessful decryption or signature verification, the request would be denied. On the other hand, if decryption was done successfully but the token has expired, then the request still would be denied. Algorithm two provides a summary for verification.

Algorithm 2: Token verification

Input: token, server key K .

1. Split token into header, key, IV, C, tag.
2. Decrypt C under K, IV ; verify tag; else reject.
3. Parse JSON claims; on failure, reject.
4. If exp is not later than now, reject.
5. Return claims.

C. Short Expiry

Tokens have a lifespan of $T=20$ minutes. This restricts the period in which any intercepted tokens will remain valid, regardless of when the interception is noticed. The precise period is examined in Section V-D.

D. Fixed-Length Claim Padding (Extension)

The claim has its contents encoded, but its size is transparent. This is because the size of JSON text representing a privileged token is different from the size of the JSON text that represents a common token. To alleviate this issue, the serialized claims are padded with an additional field so that the plaintext is always of size P . The complete process is shown in Fig. 1.

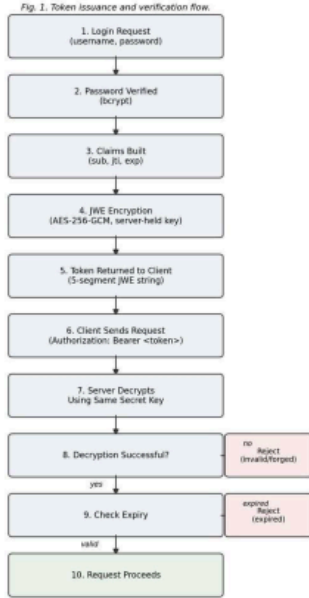


Fig. 1. Token issuance and verification flow.

V. ANALYTICAL MODEL

A. Token Size

The unpadded base64url representation of n bytes yields the character length given by

$$L(n) = \lceil 4n / 3 \rceil \quad (1)$$

characters. A compact JSON Web Encryption (JWE) token consists of a header of h bytes, an encrypted key of k bytes (with k equal to 0 when the encryption is performed by means of a symmetric key), an initialization vector (IV) of v bytes, an encrypted message of m bytes (where $m = p$ since Galois/Counter Mode (GCM) retains the size of plaintext), and a tag of τ bytes, as represented in the following equation (note that the token parts are divided by four dots):

$$S_{JWE} = L(h) + L(k) + L(v) + L(m) + L(\tau) + 4 \quad (2)$$

In turn, a signed JSON Web Token (JWT) using the HMAC SHA256 algorithm (HS256) includes a header of h' bytes, a payload of P bytes, a 32-byte signature, and two dots

$$S_{JWS} = L(h') + L(p) + L(32) + 2 \quad (3)$$

Therefore, the relative size of overhead of encryption may be defined as

$$\delta S = (S_{JWE} - S_{JWS}) / S_{JWS} \quad (4).$$

B. Forgery and Key Search

For a t -bit authentication tag, the probability a single forged token is accepted is on the order of 2^{-t} [4], so an adversary making q forgery attempts succeeds with probability at most

$$P_{\text{forge}}(q) \leq q \cdot 2^{-t} \quad (5)$$

For $t = 128$ and $q = 240$ attempts, P_{forge} is at most 2^{-88} , about 3.2×10^{-27} . Exhaustive search of a k -bit key needs on average 2^{k-1} trials, so at R trials per second

$$T_{\text{search}} = 2^{k-1} / R \quad (6)$$

For $k = 256$ and $R = 1012$ trials/s, T_{search} is about 5.8×10^{64} seconds, or about 1.8×10^{57} years.

C. IV Collision

Each token uses a fresh random IV of b bits. By the birthday bound, the probability that any two of n tokens share an IV is approximately

$$p_{IV}(n) \approx n(n-1) / 2^{b+1} \quad (7)$$

and the number of tokens issuable under one key before this probability reaches a target p is

$$n^*(p) = \sqrt{2^{b+1} \cdot p} \quad (8)$$

With the measured $b = 128$, $p_{IV}(20,000)$ is about 5.9×10^{-31} , and $n^*(2^{-32})$ is about 4.0×10^{14} tokens, so key lifetime is not limited by IV collisions at this system's scale.

D. Exposure Window

Suppose a token of lifetime T is intercepted at a time u distributed uniformly over its lifetime. The remaining useful time is $W = T - u$, so

$$E[W] = T / 2 \quad (9a)$$

$$P(\text{valid after delay } d) = \max(0, 1 - d/T) \quad (9b)$$

For $T = 20$ minutes, $E[W] = 10$ minutes and the worst case is 20 minutes. Relative to a reference lifetime T_{ref} , the exposure window is reduced by the factor T_{ref}/T ; for a 24-hour token this factor is 72.

E. Confidentiality Metrics

Let $1[f]$ equal 1 if field f in F can be recovered without the key. The exposure ratio is

$$E_x = (1/|F|) \sum_{f \in F} 1[f] \quad (10)$$

For a length-based classifier predicting privilege class C from token length Λ , with $N(l, c)$ tokens of length l and class c among N tokens, accuracy and leaked information are

$$A = (1/N) \sum_l \max_c N(l, c) \quad (11a)$$

$$I(\Lambda; C) = H(C) - H(C|\Lambda) \quad (11b)$$

For balanced classes, $H(C) = 1$ bit and chance accuracy is 0.5. Padding every plaintext to a fixed length P gives a padding field of length

$$\pi = P - m_0 \geq 0 \quad (12)$$

where m_0 is the unpadded plaintext length, making Λ constant, so $I(\Lambda; C) = 0$.

VI. EXPERIMENTAL SETUP

Every experiment utilizes the same token module call, the same key, and a backend lifetime of 20 minutes, and a single script is executed to recreate the numbers with the same random seed of 42. The baseline uses the HS256 JWT signature with four identical claims. All measurements are based on $n = 10,000$ measurements being made while 300 warmed-up executions are used; results include mean, median, standard deviation, and 95th and 99th percentiles. The experiments were conducted in a single-core environment using Python 3.12 and python-jose 3.3.0. The mean bcrypt verification time from five runs is utilized as the reference cost for login. For the rejection experiment, $N = 2,000$ tokens were used for the tampering experiment in which one random bit was manipulated in every header/IV/ciphertext/tag (500 instances). The wrong-key decryption was conducted using a random key of 32 characters and an expiry experiment was done with tokens with an expiry of between 1 second and 24 hours (rejection) as well as within the lifetime (for acceptance) along with one live 2 seconds lifetime token. An IV length experiment used each claim and was encrypted 20,000 times, and a length leak experiment was conducted with 1,000 normal tokens and 1,000 administrator tokens.

VII. RESULTS

A. Structure and Size

Every token contained five segments that can be separated by a dot while signed JWTs had three segments. In the section about key encryption, there was nothing since it was the case of dir. Table II provides information on the real size compared to the theoretical values of the size described by Eqs. (2) and (3). The real and estimated size precisely match (JWE equals 241, JWT equals 201) leading to the token size of 40 characters (Eq. 4).

TABLE II
TOKEN SIZE: PREDICTED VS. MEASURED

Component	Bytes	Chars
JWE header	29	39
JWE encrypted key	0	0
JWE IV	16	22
JWE ciphertext	115	154
JWE tag	16	22
JWE total (pred./meas.)	—	241 / 241
JWS header	27	36
JWS payload	90	120
JWS signature	32	43
JWS total (pred./meas.)	—	201 / 201

B. Latency

The timings can be observed from Table III, while Figure 2(b) depicts the timings through graphical representation. The generating of JWE consumed 54.14 μ s versus 23.42 μ s of time for the signed JWT, thus giving a lag of around 31 μ s. The verification of JWE consumed 35.82 μ s as opposed to 39.86 μ s for signed JWT, meaning that it does not take longer; however, it should not be treated as the benefit of encryption since different groups of libraries were used along with claim validation for JWT. The single verification takes 36 μ s, which means that measurements amount to around 27,900 verifications per second per core.

TABLE III
LATENCY (MS, $N = 10,000$)

Operation	Mean	Median	P95	P99
JWS issue	23.42	20.90	34.82	53.53
JWE issue	54.14	47.92	81.40	113.11
JWS verify	39.86	35.86	59.58	81.53
JWE verify	35.82	31.98	54.77	81.52
bcrypt (ms)	297.33	—	—	—

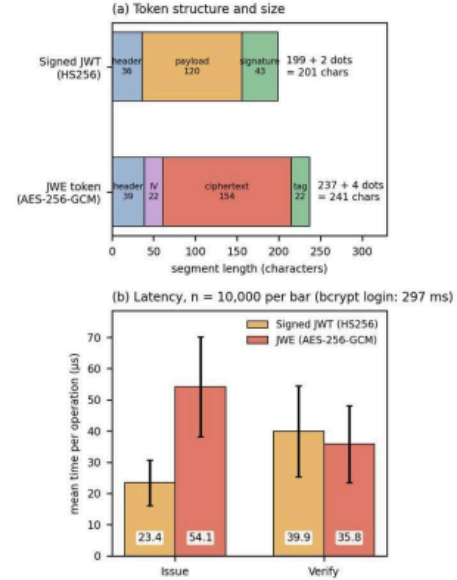


Fig. 2. (a) Token structure and size; (b) issuance/verification latency.

C. Security Experiments

The results of the security tests are depicted in Table IV. The interception of four out of four claims made by each signed JWTs took place ($Ex = 1.00$), but there were no claims recovered from JWE tokens ($Ex = 0.00$) in the experiments carried out. All malformed, invalid, and expired tokens were rejected. Notice that all valid tokens were accepted. Out of 20,000 of the identical claims being encrypted using the same secret key, each encryption produced a different pair of IV and ciphertext, thus confirming that $p_{IV} \approx 5.9 \times 10^{-31}$, as shown in Equation (7).

TABLE IV
SECURITY EXPERIMENTS

Experiment	N	Result
Claim fields recoverable, JWT	1,000	4 of 4 ($Ex=1.00$)
Claim fields recoverable, JWE	1,000	5 of 4 ($Ex=0.00$)
Bit flip (header/IV/CT/tag)	2,000	2,000 rejected
Bit flip, JWT payload	2,000	2,000 rejected
Wrong-key decryption	2,000	2,000 rejected
Expired tokens	2,000	2,000 rejected
Valid tokens	2,000	2,000 accepted
Live 2-s expiry test	1	ok@0s, rej.@3s
Distinct IVs / ciphertexts	20,000	20,000 / 20,000

D. Length Leakage

The JWE payload has been encrypted, which makes it unreadable. However, this process does not hide the length of tokens. The average token length is measured at 241 bytes, while the admin token length is 239 bytes because of the noticeable difference in length between the string values true and false. A simple classification method based on token length was able to classify the privilege type of tokens correctly at a 100% rate (Eq. 11a). Thus, the rate of information leakage is 1

bit per token.

Once the tokens were padded to a constant length $P=192$ bytes, their length grew to 343 bytes, which caused the accuracy of the classifier to decrease to the level of 50%, which is equivalent to blind guessing. In this case, the information leakage would equal 0 bits.

TABLE V
LENGTH LEAKAGE OF THE PRIVILEGE FLAG

Variant	Chars	Lengths	Accuracy
Unpadded	241 / 239	2	100%
Padded ($P=192$)	343	1	50%

VIII. DISCUSSION AND COMPARISON

The results confirm the goals of confidentiality and integrity, G1 and G2: For instance, the token does not display any of its four claim fields for parties not possessing the key, while all alterations are blocked in all experiments. The overhead is low in absolute terms, as the token takes up 19.9% more space, requires 31 μ s for issuance, does nearly the same for verification, which pales in comparison with the 297 ms necessary to hash a password during logging in. The length experiment proves that the confidentiality of content differs from the confidentiality of metadata: An attacker, while unable to obtain the value of the privilege field, may learn it by its length. Length padding removes this vulnerability with a minor cost. The analysis of the exposure window resolves the task described in goal G3: If T is set to be 20 minutes, the intercepted token can be expected to have a remaining useful life of 10 minutes, which is 72 times smaller than for a 24-hour token.

TABLE VI
COMPARISON WITH CONVENTIONAL SIGNED JWT

Aspect	JWS	JWE (proposed)
Segments	3	5
Fields readable w/o key	4 of 4	0 of 4
Tamper detection	Yes (sig.)	Yes (auth. tag)
Privilege inferable	Yes	If unpadded
Size (chars)	201	241 (343 padded)
Issue time (μ s)	23.42	54.14
Verify time (μ s)	39.86	35.82
Client integration	—	None (drop-in)
Exposure window	Depends	10 min ($T=20$ min)

IX. LIMITATIONS

There are many constraints that need to be considered. First of all, a single symmetric key is used that does not change, and this means that if the key is revealed, then all tokens signed by the key will be revealed as well. Secondly, there is no refresh-token implementation, and therefore the users have to authenticate again every 20 minutes. Thirdly, even though the library creates IVs that are 128 bits long, it should have created 96-bit long IVs according to standard [8] for AES-GCM. However, in this case, it is acceptable since only one server signs and validates tokens. Fourthly, the benchmark was conducted on the padded version of the algorithm, and this version was not used on the endpoint yet. Moreover, every bearer token can be used more than one time within its validity period since the tokens do not correspond to any specific client. Finally,

the tampering test only alters one random bit to make it hardly resemble the real-life attack by an ordinary script.

timing depends on hardware and is done locally, all experiments were in-process too.

X. CONCLUSION AND FUTURE WORK

In this paper it has been proposed to use encrypted and temporary API access tokens created with JWE and AES-256-GCM whose life span is 20 minutes. An analytical model was created to determine the optimal size for the token (241 characters). During the research, it was noted that 0 of 4 claims are revealed to the interceptor in case of using the token while it is equal to 4 of 4 in case of using a signed JWT. More than 2000 counterfeit tokens were tested and rejected as well as 2000 tokens created using wrong key. Also, the number of rejected tokens created by using the expired tokens amounted to 2000. Finding the length of ciphertext made it known that figuring out the privilege bit can be achieved successfully every time. Since utilization of fixed-length padding made this impossible adding 102 characters makes this issue unthinkable. This paper synthesized while doing research on lightweight API security along with the companion detection system. Subsequent work will include adding padding to the deployed endpoint, key rotation, refresh tokens, 96-bit IVs to ensure interoperability, client binding in order to prevent replay attacks, and storing token usage data in the companion detection system.

ACKNOWLEDGMENT

Fig. 1 and Fig. 2 in this paper were generated using Claude (Anthropic), an AI tool. All other content, including the implementation, experiments, and results, is the authors' own work.

REFERENCES

- [1] M. Jones and J. Hildebrand, "JSON Web Encryption (JWE)," Internet Eng. Task Force, RFC 7516, 2015.
- [2] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Eng. Task Force, RFC 7519, 2015.
- [3] D. McGrew and J. Viega, "The Galois/counter mode of operation (GCM)," in *Submission to NIST Modes of Operation Process*, vol. 20, 2004, pp. 0278–0070.
- [4] S. J. García Zambrano, D. A. Terán Piedra, E. F. Paredes Nacipucha, and D. A. Dávila Cárdenas, "Diseño de una metodología de Pentesting para APIs RESTful basada en los cinco principales riesgos del OWASP API Security Top 10 (2023)," Doctoral dissertation, UIDE, Quito, Ecuador, 2026.
- [5] D. Hardt, "The OAuth 2.0 authorization framework," Internet Eng. Task Force, RFC 6749, 2012.
- [6] M. Jones, "JSON Web Signature (JWS) unencoded payload option," Internet Eng. Task Force, RFC 7797, 2016.
- [7] M. Jones, "JSON Web Algorithms (JWA)," Internet Eng. Task Force, RFC 7518, May 2015.
- [8] M. Hofmeier and W. Hommel, "Enabling the JSON web signature format to support complex and identity-oriented non-web processes," in *Proc. Int. Conf. Innov. Community Services*, Cham, Switzerland: Springer Nature Switzerland, Sep. 2023, pp. 29–47.
- [9] T. Lodderstedt, M. McGloin, and P. Hunt, "OAuth 2.0 threat model and security considerations," Internet Eng. Task Force, RFC 6819, 2013.
- [10] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in *Proc. 2016 ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1204–1215.
- [11] N. Sakimura, J. Bradley, and N. Agarwal, "Proof key for

- code exchange by OAuth public clients,” Internet Eng. Task Force, RFC 7636, 2015.
- [12] J. Richer, “OAuth 2.0 token introspection,” Internet Eng. Task Force, RFC 7662, 2015.
- [13] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore, “OpenID Connect Core 1.0 incorporating errata set 1,” OpenID Foundation, Specification, 2014.
- [14] E. Rescorla, “The transport layer security (TLS) protocol version 1.3,” Internet Eng. Task Force, RFC 8446, 2018.
- [15] NIST, “Advanced encryption standard (AES),” Nat. Inst. Stand. Technol., FIPS Pub. 197, 2001.

- [16] E. Barker, W. Barker, W. Burr, W. Polk, and M. Smid, "Recommendation for key management part 1: General (revision 3)," Nat. Inst. Stand. Technol., NIST Spec. Publ. 800-57, 2012, pp. 1–147.
- [17] M. Dworkin, "Recommendation for block cipher modes of operation," Nat. Inst. Stand. Technol., NIST Spec. Publ. 800-38B, 2001.
- [18] Y. Sheffer, D. Hardt, and M. Jones, "JSON web token best current practices," Internet Eng. Task Force, RFC 8725, 2020.
- [19] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, Jun. 1999, pp. 81–91.
- [20] M. Jones and D. Hardt, "The OAuth 2.0 authorization framework: Bearer token usage," Internet Eng. Task Force, RFC 6750, 2012.

ORIGINALITY REPORT

4%

SIMILARITY INDEX

4%

INTERNET SOURCES

2%

PUBLICATIONS

0%

STUDENT PAPERS

PRIMARY SOURCES

1	cloud.riverpublishers.com	Internet Source	1%
2	www.finalroundai.com	Internet Source	1%
3	chorus.scs.carleton.ca	Internet Source	<1%
4	cybok.org	Internet Source	<1%
5	Submitted to University of Pittsburgh	Student Paper	<1%
6	V. Sekhar, P. Ajay Kumar Reddy, K. Logesh, T. Raghunadha Reddy, Dara Raju. "Next-Generation Smart Systems - Bridging Sustainability and Computational Intelligence", CRC Press, 2026	Publication	<1%
7	ns2.thinkmind.org	Internet Source	<1%
8	www.flyriver.com	Internet Source	<1%
9	www.ijert.org	Internet Source	<1%
10	Ram Lal Yadava, Anil Kumar Sethi, Amanpreet Singh Saini. "Computing, Communication, and Sustainable Energy Technologies - Sustainable Electronics and Emerging Energy Technologies (Volume 2)", CRC Press, 2026	Publication	<1%
11	datatracker.ietf.org	Internet Source	<1%
12	www.riverpublishers.com	Internet Source	<1%
13	Pita, Pedro Henrique Mendes. "Guia Técnico para Apoio no Desenvolvimento de API RESTFul Seguras", Instituto Politecnico de Beja (Portugal), 2025	Publication	<1%

Exclude quotes

Off

Exclude matches

Off

Exclude bibliography

On

Author Console

Please click [here](#) to view Welcome Message & Instructions.

+ Create new submission...Upcoming Deadlines :

1 - 2 of 21100AllClear All Filters

Paper ID	Title	Track	Files	Actions
Clear	Clear	Clear		
36	Lightweight API Security Monitoring and Anomaly Detection: Real-Time Detection of Five OWASPBased Vulnerability Categories Show abstract	Track 4: Cybersecurity and Data Privacy	Submission files: 📎 final year research paper 1.pdf	Submission: ☒ Edit Submission ☒ Edit Conflicts ✖ Delete Submission
37	Hardening API Authentication with Encrypted, Short-Lived Access Tokens Using JSON Web Encryption Show abstract	Track 4: Cybersecurity and Data Privacy	Submission files: 📎 final year research paper 2.pdf	Submission: ☒ Edit Submission ☒ Edit Conflicts ✖ Delete Submission